

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Definición y desarrollo
de Solución Informática
para el aprendizaje del
lenguaje musical

Curso: 2025/2026

Alumno/a:

José Luis Garrido Castro

Director/es:

Javier Criado Rodríguez

AGRADECIMIENTOS

Me gustaría dar las gracias, en primer lugar, a la Escuela de Música de Huércal de Almería, al Real Conservatorio Profesional de Música de Almería y la Escuela de Música Manuel Sánchez Alonso de Canjáyar por formarme durante mis 17 últimos años, poner en mi vida este maravilloso arte que es la música y por prestarme su valiosa ayuda para este proyecto.

En segundo lugar, a mi familia ya que siempre me han apoyado a lo largo de toda mi vida y se que se sienten muy orgullosos de mis logros, en especial, los conseguidos a lo largo de este grado de Ingeniería Informática. Me han dado todo y no me cabe duda que siempre me lo darán.

Después, cabe mencionar a mis amigos que siempre estaban ahí para animarme en momentos malos y sacarme de casa cuando me hacía falta.

En particular, quería destacar a mi compañero Jesús, con el cual llevo coincidiendo desde el instituto, manteniendo una estrecha amistad que se ha fortalecido durante el grado pasando grandes momentos y ayudándonos de forma recíproca en lo que hemos necesitado cada uno.

A mi pareja que hasta hoy no ha quedado día que no me haya preguntado como me iba con todos mis proyectos (como este) y que siempre consigue mandarme un empujón y sacarme una sonrisa para seguir llevando todo a cabo.

Por último, a todos los profesores que me han ayudado a lo largo de todo el grado y que me han transmitido todos los conocimientos posibles así como permitirme desarrollar la gran cantidad de nuevas habilidades que he conseguido durante estos años.

ÍNDICE GENERAL

	Página
1. Introducción	1
1.1. Motivación	1
1.2. Proceso de desarrollo de la aplicación	2
1.3. Estructura de la memoria	2
2. Contexto y estado del arte	4
2.1. Contexto	4
2.2. Digitalización de la enseñanza musical	5
2.3. Particularidades del aprendizaje del lenguaje musical	6
2.4. Estado del arte	6
2.5. Justificación de la solución propuesta	7
2.6. Tecnologías y arquitectura de la solución	8
2.6.1. Frontend	8
2.6.2. Backend	9
2.6.3. Base de datos	9
2.6.4. Servicios externos	10
2.6.5. Integración y despliegue continuo	10
2.6.6. Arquitectura global de la aplicación	10
3. Objetivos	12
3.1. Objetivo general	12
3.2. Objetivos específicos	12
3.3. Planificación	13
3.3.1. Metodología de trabajo	13
3.3.2. Cronograma del proyecto	14
4. Análisis, definición y arquitectura de la solución	16
4.1. Investigación	16
4.1.1. Introducción	16

4.1.2.	Análisis y comparación de las respuestas de los profesores	17
4.1.3.	Análisis y comparación de las respuestas de los alumnos	18
4.1.4.	Herramientas actuales	20
4.1.5.	Necesidades detectadas	21
4.2.	Estudio bibliográfico	21
4.2.1.	Introducción	21
4.2.2.	Mapping Literature Review	22
4.2.2.1.	Definición la cadena de búsqueda y criterios	22
4.2.2.2.	Selección de documentación relevante	23
4.2.2.3.	Resultados obtenidos	24
4.2.2.4.	Conclusiones	26
4.3.	Especificación formal de la solución	27
4.4.	Objetivos funcionales del sistema	28
4.5.	Diagrama UML de casos de uso del sistema	29
4.6.	Definición del diagrama de casos de uso	30
4.6.1.	Actores	30
4.6.2.	Casos de uso	31
4.6.2.1.	Definición general	31
4.6.2.2.	Requisitos funcionales	34
4.7.	Requisitos no funcionales	59
4.8.	Arquitectura de la solución	62
4.8.1.	Aspectos comunes de la implementación	62
4.8.1.1.	Principios de diseño comunes	63
4.8.1.2.	Comunicación e integración entre módulos	63
4.8.1.3.	Seguridad y gestión de la autenticación	64
4.8.1.4.	Calidad del software y despliegue	66
4.8.2.	Backend	67
4.8.2.1.	Arquitectura y organización del backend	67
4.8.2.2.	Modelado de datos	68
4.8.2.3.	Implementación de la lógica de negocio	70
4.8.2.4.	Integración con servicios externos	71
4.8.2.5.	Servicios de soporte	71
4.8.2.6.	Testing	73
4.8.2.7.	Despliegue	73
4.8.3.	Frontend	74

4.8.3.1.	Arquitectura y organización del frontend	74
4.8.3.2.	Implementación	75
4.8.3.3.	Diseño y experiencia de usuario	79
4.8.3.4.	Testing	81
4.8.3.5.	Despliegue	82
4.8.4.	Integración continua del sistema	83
4.8.4.1.	Coordinación del pipeline	83
4.8.4.2.	Automatización del flujo de trabajo	84
4.8.4.3.	Resultado del proceso	84
5.	Flujos de trabajo	85
5.1.	Flujo de trabajo de cuestionarios	85
5.1.1.	Creación del cuestionario	85
5.1.2.	Resolución del cuestionario	86
5.1.3.	Consulta de resultados	87
5.2.	Flujo de trabajo de tareas	88
5.2.1.	Creación de la tarea	88
5.2.2.	Entrega de la tarea	89
5.2.3.	Evaluación de la tarea	90
5.2.4.	Consulta de la calificación	90
5.3.	Flujo de trabajo de calificaciones	91
5.3.1.	Calificación general por parte del profesor	91
5.3.2.	Consulta de calificaciones por parte del alumno	92
5.4.	Conclusión	92
6.	Conclusiones y líneas de trabajo futuro	93
6.1.	Conclusiones	93
6.2.	Líneas de trabajo futuro	94
6.2.1.	Plan de implantación y despliegue en entornos reales	94
6.2.2.	Extensión de funcionalidades e innovación	95
6.2.3.	Acceso sin autenticación y apertura a usuarios externos	95
6.2.4.	Creación de la aplicación móvil	96
7.	Anexo	97
7.1.	Repositorio del proyecto	97
7.2.	Estructura del proyecto	97



7.2.1. Estructura del backend	98
7.2.2. Estructura del frontend	98
7.3. Capturas adicionales de la aplicación	99
7.4. Entornos y herramientas de desarrollo	101
7.4.1. Entorno de desarrollo	101
7.4.2. Gestión y visualización de la base de datos	101
7.4.3. Herramientas de apoyo adicionales	102

ÍNDICE DE FIGURAS

2.1. Elementos del lenguaje musical	4
2.2. Tecnologías frontend: Ionic, Angular, Jasmine y Karma	8
2.3. Tecnologías backend: Node.js, Express.js, JWT, Swagger, Jest y Supertest	9
2.4. MongoDB	10
2.5. Servicios externos	10
2.6. Tecnologías de integración y despliegue continuo	10
2.7. Arquitectura global de la aplicación	11
3.1. Cronograma del proyecto	15
4.1. Entrevistas	17
4.2. Encuesta al alumnado	18
4.3. Respuestas a las preguntas 1, 3 y 6	19
4.4. Respuestas a las pregunta 4	19
4.5. Flujograma de la Mapping Review	23
4.6. Diagrama UML de casos de uso del sistema	29
4.7. Flujo general de comunicación entre frontend y backend durante una operación del sistema.	64
4.8. Modelo conceptual de datos del sistema	69
4.9. Interfaz de documentación de la API generada mediante Swagger.	72
4.10. Pipeline de integración y despliegue continuo del backend.	74
4.11. Vista de inicio de sesión en dispositivos móviles y escritorio.	80
4.12. Menú lateral y barra de pestañas utilizadas en la aplicación.	80
4.13. Ejemplo de utilización de componentes modales en la gestión de calificaciones.	80
4.14. Vista representativa de la interfaz desarrollada.	81
4.15. Proceso de despliegue del frontend.	82
5.1. Creación de un nuevo cuestionario.	86
5.2. Mensaje de notificación al alumno tras la creación del cuestionario.	86

5.3. Resolución de un cuestionario por parte del alumno utilizando las pistas disponibles. . .	87
5.4. Consulta de un cuestionario realizado y revisión de las respuestas.	87
5.5. Creación de una nueva tarea.	88
5.6. Confirmación de la creación de la tarea.	89
5.7. Entrega de una tarea por parte del alumno.	89
5.8. Mensaje de notificación al profesor tras la entrega de la tarea.	89
5.9. Evaluación de una tarea por parte del profesor.	90
5.10. Calificaciones en detalles de tarea.	90
5.11. Consulta de la tarea evaluada por parte del alumno.	91
5.12. Calificación general por parte del profesor.	91
5.13. Consulta de las calificaciones por parte del alumno.	92
7.1. Vista de cada rama de la aplicación	99
7.2. Vista de administración	100
7.3. Vista de mensajería	100

ÍNDICE DE TABLAS

4.1. Documentación recogida	24
4.2. Actor 1	30
4.3. Actor 2	30
4.4. Actor 3	30
4.5. Actor 4	30
4.6. Actor 5	30
4.7. Caso de uso: Iniciar sesión	31
4.8. Caso de uso: Recuperar contraseña	31
4.9. Caso de uso: Crear profesor	31
4.10. Caso de uso: Crear alumno	31
4.11. Caso de uso: Gestionar grupos	31
4.12. Caso de uso: Enviar mensaje	32
4.13. Caso de uso: Gestionar material de cada rama	32
4.14. Caso de uso: Gestionar tareas	32
4.15. Caso de uso: Gestionar entregas	32
4.16. Caso de uso: Gestionar entregas	32
4.17. Caso de uso: Gestionar calificaciones del alumnado	32
4.18. Caso de uso: Ver mensajes	33
4.19. Caso de uso: Cambiar contraseña	33
4.20. Caso de uso: Interactuar con el material de cada rama	33
4.21. Caso de uso: Revisar calificaciones	33
4.22. Caso de uso: Interactuar con tareas	33
4.23. Caso de uso: Interactuar con cuestionarios	33
4.24. Caso de uso: Interactuar con el metrónomo	33
4.25. Caso de uso: Interactuar con el piano	34
4.26. Requisito funcional: RF-01 Iniciar sesión	34
4.27. Requisito funcional: RF-02 Recuperar contraseña	35

4.28. Requisito funcional: RF-03 Crear profesor	35
4.29. Requisito funcional: RF-04 Validar y generar automáticamente las credenciales	36
4.30. Requisito funcional: RF-05 Crear alumno	37
4.31. Requisito funcional: RF-06 Crear grupo	37
4.32. Requisito funcional: RF-07 Listar grupos	38
4.33. Requisito funcional: RF-08 Seleccionar grupo	38
4.34. Requisito funcional: RF-09 Enviar mensaje	39
4.35. Requisito funcional: RF-10 Ver libro del curso	39
4.36. Requisito funcional: RF-11 Subir o reemplazar libro del curso	40
4.37. Requisito funcional: RF-12 Eliminar libro del curso	41
4.38. Requisito funcional: RF-13 Revisar tareas	41
4.39. Requisito funcional: RF-14 Crear tarea	42
4.40. Requisito funcional: RF-15 Modificar tarea	43
4.41. Requisito funcional: RF-16 Cerrar tarea	44
4.42. Requisito funcional: RF-17 Eliminar tarea	44
4.43. Requisito funcional: RF-18 Revisar cuestionarios	45
4.44. Requisito funcional: RF-19 Crear cuestionario	46
4.45. Requisito funcional: RF-20 Modificar cuestionario	47
4.46. Requisito funcional: RF-21 Cerrar cuestionario	48
4.47. Requisito funcional: RF-22 Eliminar cuestionario	48
4.48. Requisito funcional: RF-23 Revisar entregas	49
4.49. Requisito funcional: RF-24 Calificar entrega	50
4.50. Requisito funcional: RF-25 Revisar calificaciones de entregas	51
4.51. Requisito funcional: RF-26 Revisar calificaciones generales del alumno	52
4.52. Requisito funcional: RF-27 Asignar calificación general al alumno	53
4.53. Requisito funcional: RF-28 Ver mensajes	54
4.54. Requisito funcional: RF-29 Entregar tarea	55
4.55. Requisito funcional: RF-30 Rellenar y entregar cuestionario	56
4.56. Requisito funcional: RF-31 Cambiar contraseña	57
4.57. Requisito funcional: Configurar tempo del metrónomo	57
4.58. Requisito funcional: Iniciar y detener metrónomo	58
4.59. Requisito funcional: Utilizar piano de referencia	58

4.60. Requisito funcional transversal: RFT-1 Enviar notificaciones del sistema	59
4.61. Requisito no funcional: RNF-01 Escalabilidad y mantenibilidad	59
4.62. Requisito no funcional: RNF-02 Usabilidad y accesibilidad	60
4.63. Requisito no funcional: RNF-03 Diseño visual atractivo y motivador	60
4.64. Requisito no funcional: RNF-04 Portabilidad	60
4.65. Requisito no funcional: RNF-05 Seguridad	61
4.66. Requisito no funcional: RNF-06 Consistencia pedagógica	61
4.67. Requisito no funcional: RNF-07 Adaptabilidad del contenido	62

1 INTRODUCCIÓN

El proyecto aborda la necesidad de recursos software en el lenguaje musical o tradicionalmente conocido como solfeo. Se trata de una asignatura fundamental en los centros educativos donde se aprende a tocar un instrumento. Este trabajo se enmarca en el proyecto “Definición y desarrollo de una solución informática para el aprendizaje del lenguaje musical”.

1.1 MOTIVACIÓN

La motivación para la realización de este trabajo surge de la combinación de dos ámbitos que han formado parte de mi trayectoria personal y académica: la música y la informática. Mi formación musical comenzó en la Escuela Municipal de Música de Huércal de Almería, donde descubrí el lenguaje musical como la base fundamental para el aprendizaje de cualquier instrumento [1]. Posteriormente, continué mi formación durante seis años en el Conservatorio Profesional de Música de Almería, una etapa que me permitió profundizar en esta disciplina y conocer de cerca su metodología de enseñanza.

Durante este recorrido pude comprobar la importancia del lenguaje musical en la formación de cualquier músico, pero también observar que el proceso de aprendizaje sigue apoyándose principalmente en recursos tradicionales, como libros de teoría, cuadernos de ejercicios o explicaciones en el aula. Aunque en los últimos años la tecnología ha adquirido un papel cada vez más relevante en el ámbito educativo, la oferta de aplicaciones específicas para el aprendizaje del lenguaje musical continúa siendo limitada y, en muchos casos, no responde completamente a las necesidades del alumnado.

Esta experiencia personal despertó el interés por desarrollar una herramienta informática que pudiera servir como complemento a la enseñanza convencional, ofreciendo un entorno interactivo que facilitara la práctica autónoma y reforzara los conocimientos adquiridos en clase. La posibilidad de recibir retroalimentación inmediata, realizar ejercicios de forma dinámica y disponer de un recurso accesible desde cualquier lugar puede contribuir a mejorar la experiencia de aprendizaje y aumentar la motivación del estudiante.

Además, este Trabajo Fin de Grado representa una oportunidad para integrar los conocimientos adquiridos durante el Grado en Ingeniería Informática con una afición que ha formado parte de mi vida desde una edad temprana. El desarrollo de esta aplicación no solo supone un reto técnico, sino también la posibilidad de aportar una solución basada en una necesidad identificada a través de mi propia experiencia como estudiante de música.

1.2 PROCESO DE DESARROLLO DE LA APLICACIÓN

El desarrollo de la aplicación se ha llevado a cabo siguiendo un proceso iterativo [2] compuesto por diferentes fases, con el objetivo de diseñar una solución que respondiera a las necesidades reales de la enseñanza del lenguaje musical y garantizara una implementación sólida y mantenible.

La primera fase consistió en la investigación y análisis del problema. Para ello, se estudió el contexto de la enseñanza del lenguaje musical y se identificaron las necesidades del profesorado y del alumnado mediante diferentes instrumentos de recopilación de información, como encuestas y entrevistas. Paralelamente, se realizó un estudio de las herramientas informáticas existentes y una revisión bibliográfica que permitieron conocer el estado actual de este tipo de soluciones, identificar sus fortalezas y limitaciones y obtener una base sobre la que definir una propuesta propia.

A partir de la información recopilada, se procedió a la definición de la solución, estableciendo las funcionalidades que debía ofrecer la aplicación y los requisitos necesarios para satisfacer las necesidades detectadas. Esta fase incluyó el modelado del sistema y el diseño de la arquitectura, definiendo la organización de sus componentes y las tecnologías más adecuadas para su desarrollo.

Una vez diseñada la solución, se inició la fase de implementación, desarrollando de forma progresiva los diferentes módulos que componen la aplicación. Se construyeron tanto los servicios encargados de la lógica de negocio y la gestión de datos como la interfaz de usuario, integrando todas las funcionalidades previstas en el diseño inicial y garantizando la comunicación entre los distintos componentes del sistema.

Durante el desarrollo se realizaron pruebas continuas sobre las funcionalidades implementadas con el fin de detectar y corregir posibles errores desde las primeras etapas de construcción. Finalmente, se configuraron procesos de integración y despliegue continuo que permitieron automatizar la validación y publicación de la aplicación, facilitando su mantenimiento y evolución.

1.3 ESTRUCTURA DE LA MEMORIA

La memoria se organiza en seis capítulos principales, cada uno de ellos dedicado a un aspecto específico del trabajo realizado.

El primer capítulo, *Introducción*, presenta el contexto del proyecto, la motivación que ha dado lugar a su desarrollo, un resumen del proceso seguido para la creación de la aplicación y la organización de la propia memoria.

El segundo capítulo, *Contexto y estado del arte*, describe los conceptos fundamentales relacionados con el lenguaje musical, así como su relación con la informática, y analiza las principales herramientas informáticas existentes en este ámbito, proporcionando el marco teórico necesario para el desarrollo del proyecto.

En el tercer capítulo, *Objetivos*, se definen los objetivos generales y específicos del proyecto, estableciendo el alcance de la solución propuesta y los requisitos que debe satisfacer. De igual forma, se presenta la planificación del trabajo y la metodología seguida durante el desarrollo del proyecto.

El cuarto capítulo, *Análisis, definición y desarrollo de la solución*, constituye el núcleo de la memoria. En él se describen las fases de investigación realizadas para identificar las necesidades del profesorado y del alumnado, el estudio bibliográfico llevado a cabo y el proceso de recopilación de información mediante diferentes instrumentos de investigación. A partir de este análisis se presenta la definición de la solución, el modelado de la aplicación, el diseño de su arquitectura, la selección de las tecnologías empleadas y el desarrollo de los diferentes componentes que la integran. Asimismo, se exponen los mecanismos de validación, las pruebas realizadas y los procesos de integración y despliegue continuo implementados durante el desarrollo.

El quinto capítulo, *Flujos de trabajo*, muestra algunos de los principales escenarios de utilización de la aplicación, describiendo el funcionamiento de sus principales funcionalidades y el recorrido que puede seguir un usuario durante la interacción con el sistema.

El sexto capítulo, *Conclusiones y líneas de trabajo futuro*, recoge una valoración global del trabajo realizado, analizando el grado de consecución de los objetivos planteados, las limitaciones encontradas durante el desarrollo y las posibles mejoras y ampliaciones que podrán abordarse en futuras versiones de la aplicación.

Por último, el séptimo capítulo, *Anexo*, expone aspectos importantes del proyecto que no se pueden encuadrar en ninguna de las fases de redacción de la memoria.

2 CONTEXTO Y ESTADO DEL ARTE

2.1 CONTEXTO

El lenguaje musical, como se ha mencionado, es una asignatura enseñada en la gran mayoría de centros de estudio de la música, pero, como su propio nombre indica, también es un lenguaje o "idioma" (es por ello que la asignatura en la que se estudia tiene el mismo nombre).

Al igual que ocurre con otros idiomas, tanto en la comunicación escrita como oral, es el método que se usa para transmitir algo mediante la música. Por tanto, al igual que existen las letras, las palabras o los distintos signos de puntuación, en este caso, se tendrán una serie de elementos para establecer la comunicación tal y como se muestra en la Figura 2.1 [3].

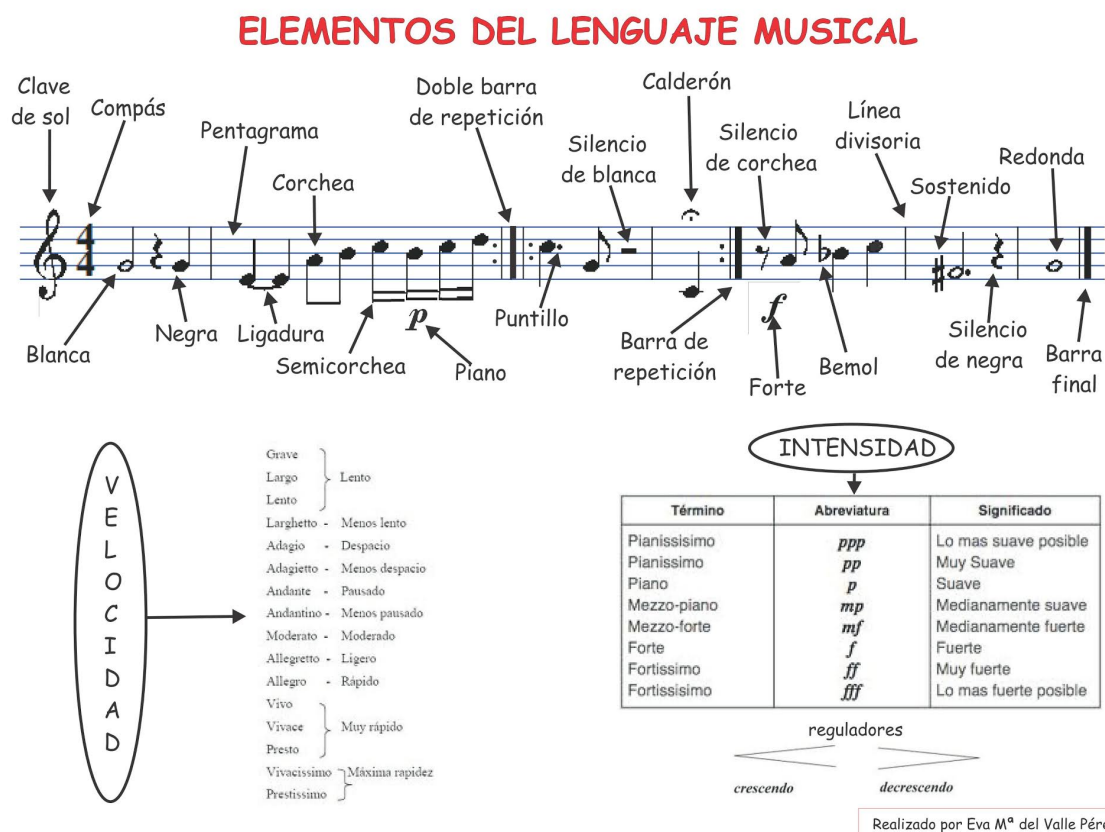


Figura 2.1: Elementos del lenguaje musical [4]

Para aprender a utilizar todos estos elementos tanto de forma escrita como oral es necesario estudiar las siguientes cuatro ramas:

- **Ritmo:** Es la parte del lenguaje musical que consiste en la enseñanza de la lectura de partituras con fluidez.
- **Entonación:** Esta rama se encarga del estudio de la entonación de partituras con fluidez.
- **Audición:** La audición es otra de las ramas del lenguaje musical y en ella se entrena el oído para ser capaz de entender cualquier fragmento musical con el fin de poder plasmarlo (principalmente de forma escrita).
- **Teoría:** Por último, existe una parte en el lenguaje musical que consiste en la enseñanza teórica de la misma.

A través del estudio de las anteriores, se puede llegar a conseguir un dominio completo en el uso del lenguaje musical, por lo tanto, es importante tenerlo en cuenta de cara a la construcción de una solución.

Otro aspecto fundamental es la diferenciación entre escuelas de música y conservatorios. Los primeros, habitualmente, están gestionados por asociaciones y/o pequeñas empresas y, aunque pueden beneficiarse de alguna subvención local, son por tanto privados, lo que significa que cada uno puede tener una manera muy diferente de enfocar la asignatura por lo que las necesidades pueden ser distintas, en cambio, los conservatorios son públicos por lo que su manera de enseñar la asignatura está mucho más racionalizada y será muy parecida entre todos los centros.

2.2 DIGITALIZACIÓN DE LA ENSEÑANZA MUSICAL

La incorporación de las Tecnologías de la Información y la Comunicación (TIC) ha supuesto una transformación significativa en los procesos de enseñanza y aprendizaje. La aparición de nuevas herramientas digitales ha permitido complementar los métodos tradicionales mediante recursos que facilitan el acceso a los contenidos, fomentan la participación del alumnado y amplían las posibilidades de aprendizaje fuera del aula [5].

La enseñanza musical también ha experimentado esta evolución. En la actualidad es habitual encontrar en los centros educativos proyectores, equipos de sonido, pizarras digitales o plataformas para compartir materiales docentes. Sin embargo, esta digitalización se ha producido principalmente desde el punto de vista de la infraestructura, mientras que la disponibilidad de aplicaciones específicas para el aprendizaje del lenguaje musical continúa siendo relativamente limitada.

Como consecuencia, una parte importante del proceso de enseñanza sigue apoyándose en materiales tradicionales como libros, cuadernos de ejercicios o apuntes elaborados por el profesorado, reduciendo las posibilidades de interacción del alumnado y dificultando la realización de actividades autónomas fuera del entorno presencial.

En este contexto, el desarrollo de soluciones software específicas representa una oportunidad para complementar la enseñanza convencional, ofreciendo herramientas que permitan reforzar el aprendizaje mediante actividades interactivas, un acceso permanente a los contenidos y una mayor adaptación al ritmo de estudio de cada estudiante.

2.3 PARTICULARIDADES DEL APRENDIZAJE DEL LENGUAJE MUSICAL

El aprendizaje del lenguaje musical presenta una serie de características que lo diferencian de otras disciplinas académicas. No se limita a la adquisición de conocimientos teóricos, sino que combina el estudio conceptual con el desarrollo de habilidades prácticas relacionadas con la lectura, la interpretación y la percepción auditiva [6].

Las cuatro ramas que componen la asignatura requieren metodologías de aprendizaje diferentes, aunque estrechamente relacionadas. Mientras que la teoría puede reforzarse mediante ejercicios escritos o cuestionarios, el ritmo, la entonación y la audición necesitan una práctica constante basada en la repetición, la interpretación y la escucha activa.

Esta naturaleza práctica convierte la continuidad en el estudio en un elemento fundamental para la consolidación del aprendizaje. Sin embargo, el tiempo disponible durante las clases suele resultar insuficiente para realizar todo el entrenamiento necesario, por lo que disponer de recursos complementarios accesibles desde cualquier lugar puede aportar un valor significativo al proceso educativo.

Las aplicaciones informáticas ofrecen ventajas especialmente interesantes en este ámbito, permitiendo proporcionar corrección inmediata, registrar el progreso del alumnado, adaptar el nivel de dificultad o facilitar la realización de ejercicios de forma autónoma. De esta manera, la tecnología no pretende sustituir la labor del profesorado, sino convertirse en una herramienta de apoyo que complemente los métodos tradicionales de enseñanza.

2.4 ESTADO DEL ARTE

La incorporación de herramientas informáticas en la enseñanza de la música ha experimentado un crecimiento significativo durante los últimos años. Actualmente existe una amplia variedad de aplicaciones y servicios que permiten complementar el aprendizaje mediante recursos interactivos, actividades prácticas y contenidos multimedia. Sin embargo, la mayoría de estas soluciones están orientadas a cubrir necesidades muy específicas, siendo necesario combinar varias herramientas diferentes para abarcar la totalidad de los contenidos del lenguaje musical.

Dentro de las aplicaciones más extendidas se encuentran los editores de partituras, entre los que destaca *MuseScore* [7], utilizado tanto por profesorado como por alumnado para la creación, edición y reproducción de partituras digitales. Del mismo modo, plataformas como *Noteflight* [8] trasladan estas funcionalidades a un entorno web, facilitando el trabajo colaborativo y el acceso desde distintos dispositivos.

En el ámbito de la creación y edición de audio también existen herramientas consolidadas como *Audacity* [9], que permiten trabajar aspectos relacionados con la audición y el análisis sonoro, mientras que aplicaciones como *Hydrogen* o *Band-in-a-Box* ofrecen recursos para la creación de patrones rítmicos y melódicos que pueden utilizarse como apoyo en el aprendizaje.

Por otro lado, la digitalización de la educación ha impulsado el uso de plataformas de aprendizaje generalistas como *Kahoot* [10] o *Quizizz*, ampliamente utilizadas para la realización de cuestionarios interactivos y actividades de evaluación. Estas herramientas aportan una mayor participación del alumnado y una retroalimentación inmediata, características especialmente interesantes en el contexto educativo.

Además de estas aplicaciones de carácter general, también existen soluciones específicamente orientadas a la enseñanza musical. Entre ellas destaca *EstudioMusica* [11], plataforma utilizada en distintos centros de enseñanza musical como apoyo a la asignatura y que proporciona recursos para la realización de actividades y ejercicios. Asimismo, pueden encontrarse aplicaciones como *Leemusica*, desarrollada con el objetivo de introducir el lenguaje musical en edades tempranas mediante una interfaz adaptada al alumnado infantil, o sistemas más amplios como *Soft Way to Mozart*, centrados en el aprendizaje instrumental mediante recursos interactivos.

No obstante, el análisis de las soluciones existentes pone de manifiesto una característica común: la mayoría se centran en funcionalidades concretas o en una única rama del aprendizaje musical, obligando a combinar diferentes aplicaciones para trabajar la teoría, el ritmo, la audición o la entonación. Además, muchas de ellas están orientadas a la creación de contenidos por parte del profesorado o requieren conocimientos previos que dificultan su utilización como herramienta principal de aprendizaje.

Por tanto, más que una ausencia de recursos informáticos, el estado actual presenta un ecosistema de herramientas especializadas pero poco integradas entre sí. Esta situación justifica el desarrollo de una aplicación que reúna en un único entorno las funcionalidades más relevantes para la enseñanza del lenguaje musical, proporcionando una experiencia homogénea tanto para el profesorado como para el alumnado y sirviendo como complemento a los métodos tradicionales de enseñanza.

2.5 JUSTIFICACIÓN DE LA SOLUCIÓN PROPUESTA

El análisis del contexto educativo y del estado actual de las herramientas disponibles pone de manifiesto la existencia de una carencia de recursos informáticos que aborden el aprendizaje del lenguaje musical de forma global. La mayoría de las aplicaciones existentes están orientadas a una única funcionalidad o requieren el uso combinado de diferentes plataformas para cubrir las distintas ramas de la asignatura.

Por este motivo, el presente trabajo plantea el desarrollo de una aplicación informática que reúna en un mismo entorno las funcionalidades necesarias para complementar la enseñanza del lenguaje musical, proporcionando un recurso accesible, intuitivo y adaptado tanto a las necesidades del profesorado como del alumnado.

La solución propuesta no pretende sustituir la metodología tradicional empleada en los centros de enseñanza musical, sino ofrecer una herramienta que facilite el trabajo dentro y fuera del aula, fomente

el aprendizaje autónomo y aproveche las posibilidades que ofrecen las tecnologías actuales para mejorar la experiencia educativa.

2.6 TECNOLOGÍAS Y ARQUITECTURA DE LA SOLUCIÓN

Con el objetivo de desarrollar una aplicación web moderna, mantenible y fácilmente escalable, se ha optado por una arquitectura basada en un modelo cliente-servidor [12] con una separación clara entre frontend y backend, apoyada en servicios REST [13]. Esta decisión responde tanto a criterios técnicos como a condicionantes generales del proyecto, tales como la viabilidad en un contexto académico, el uso de tecnologías gratuitas o con planes de uso sin coste durante el desarrollo, y la necesidad de facilitar la evolución futura del sistema.

La arquitectura modular adoptada permite integrar servicios externos, desplegar la solución en entornos *cloud* y automatizar su ciclo de vida, garantizando una experiencia de usuario accesible y consistente, así como una gestión eficiente de los recursos y los datos. Además, favorece aspectos como la mantenibilidad, la seguridad, la portabilidad y la adaptabilidad, que constituyen objetivos fundamentales del desarrollo de la aplicación.

A continuación, se detallan las tecnologías seleccionadas para cada una de las capas del sistema, junto con la justificación de su elección.

2.6.1 Frontend

La interfaz de usuario se desarrolla como una aplicación web con el objetivo de maximizar la accesibilidad desde distintos dispositivos y plataformas. En consecuencia, se adopta un enfoque híbrido que permite, como línea de trabajo futura, exportar la solución a aplicaciones móviles nativas sin necesidad de rediseñar el sistema.

Para ello, se ha seleccionado el framework **Ionic** [14] junto con **Angular** [15]. Ionic facilita el desarrollo multiplataforma y la reutilización de código, mientras que Angular aporta una arquitectura robusta y modular. En particular, Angular sigue el patrón **Modelo-Vista-Controlador (MVC)** [16], lo que favorece la separación de responsabilidades, la mantenibilidad del código y la escalabilidad del sistema.

Además, este enfoque arquitectónico facilita la aplicación de buenas prácticas de desarrollo y la incorporación de estándares de accesibilidad (como WCAG [17]), al permitir una estructuración clara de las vistas, la lógica de negocio y la gestión de estados.

Para garantizar la calidad del frontend, se emplean herramientas de testing como **Jasmine** [18] y **Karma** [19], que permiten realizar pruebas unitarias y de integración, contribuyendo a la fiabilidad del sistema ante futuras modificaciones.



Figura 2.2: Tecnologías frontend: Ionic, Angular, Jasmine y Karma

La elección de estas tecnologías se justifica por su madurez, amplia documentación y comunidad activa, lo que refuerza la viabilidad técnica del proyecto dentro de un entorno académico y facilita su evolución futura.

2.6.2 Backend

El backend se implementa siguiendo una arquitectura RESTful, lo que permite una comunicación desacoplada con el frontend y facilita la integración con servicios externos. Esta decisión responde tanto a criterios técnicos como a la necesidad de mantener una arquitectura clara, extensible y fácilmente mantenible.

La tecnología base escogida es **Node.js** [20] junto con **Express.js** [21]. Node.js proporciona un entorno eficiente basado en JavaScript, coherente con el stack del frontend, mientras que Express.js ofrece la flexibilidad necesaria para construir APIs ligeras y escalables.

En materia de seguridad, se integra **JSON Web Tokens (JWT)** [22] para la autenticación y autorización de usuarios. Este mecanismo garantiza la identidad de los usuarios mediante tokens firmados, evitando accesos no autorizados y alineándose con buenas prácticas de seguridad ampliamente aceptadas [23].

Para mejorar la mantenibilidad y facilitar la comprensión del sistema, se utiliza **Swagger** [24] como herramienta de documentación de la API. Swagger permite generar documentación interactiva de forma automática, lo que resulta especialmente útil en contextos académicos y en futuras ampliaciones del sistema.

La calidad del backend se refuerza mediante el uso de herramientas de testing como **Jest** [25] y **Supertest** [26]. Jest permite realizar pruebas unitarias de la lógica de negocio, mientras que Supertest facilita la validación de los endpoints HTTP, contribuyendo a la fiabilidad y robustez del sistema.



Figura 2.3: Tecnologías backend: Node.js, Express.js, JWT, Swagger, Jest y Supertest

2.6.3 Base de datos

Se opta por una base de datos documental, concretamente **MongoDB** [27], debido a su flexibilidad en la gestión de datos y su capacidad de escalar horizontalmente. Esta elección permite adaptarse a posibles cambios en el modelo de datos sin necesidad de rediseños complejos.

MongoDB se integra de forma nativa con Node.js, facilitando el desarrollo y la gestión de la información relativa a usuarios, grupos, tareas y demás entidades del sistema.



Figura 2.4: MongoDB

Para su despliegue, se ha optado por **MongoDB Atlas**, un servicio gestionado en la nube que ofrece una solución escalable y segura.

2.6.4 Servicios externos

Con el objetivo de optimizar el desarrollo y mejorar la seguridad del sistema, se integran servicios externos que permiten reutilizar soluciones especializadas:

- **Mailjet** [28] para la gestión del envío de correos electrónicos y notificaciones automáticas.
- **API de generación de contraseñas** [29] para asegurar la creación de credenciales robustas sin desarrollar esta funcionalidad desde cero.



Figura 2.5: Servicios externos

2.6.5 Integración y despliegue continuo

El ciclo de vida del desarrollo se apoya en un flujo de integración y despliegue continuo mediante **GitHub Actions** [30], que automatiza la ejecución de pruebas y el despliegue del sistema tras cada cambio relevante en el repositorio.

El despliegue para el backend se realiza en la plataforma **Render** [31] integrada directamente con GitHub y para el frontend con **GitHub Pages**, lo que permite una actualización automática del entorno de producción. Esta decisión responde a criterios de viabilidad y gratuidad, adecuados al contexto académico del proyecto.



Figura 2.6: Tecnologías de integración y despliegue continuo

2.6.6 Arquitectura global de la aplicación

La arquitectura global presentada en la Figura 2.7 sigue un enfoque modular y desacoplado, facilitando la escalabilidad, mantenibilidad y adaptación futura del sistema.

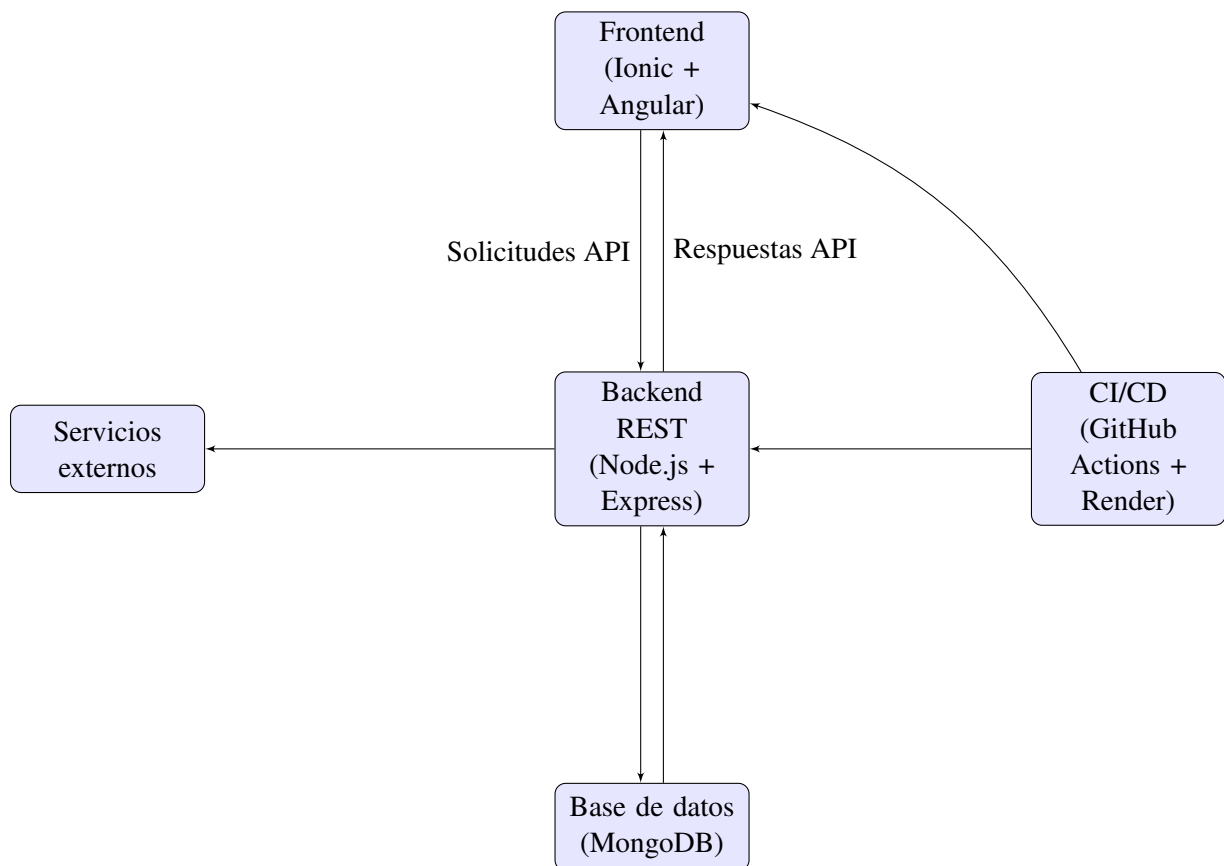


Figura 2.7: Arquitectura global de la aplicación

La combinación de esta arquitectura y de las tecnologías seleccionadas proporciona una base sólida para el desarrollo de la aplicación, facilitando su evolución, mantenimiento y posible ampliación en futuras versiones.

3 OBJETIVOS

En este capítulo se presentan los objetivos que guían el desarrollo del presente Trabajo Fin de Grado. En primer lugar, se define el objetivo general del proyecto, que establece la finalidad principal de la solución propuesta. Posteriormente, se detallan los objetivos específicos cuya consecución permite alcanzar dicho propósito y estructurar las diferentes fases del trabajo.

3.1 OBJETIVO GENERAL

El objetivo general de este Trabajo Fin de Grado es **desarrollar una aplicación informática que contribuya a mejorar el proceso de enseñanza y aprendizaje del lenguaje musical mediante el uso de herramientas digitales accesibles e interactivas.**

Para alcanzar esta meta, no solo es necesario implementar una solución software funcional, sino también comprender el contexto educativo en el que será utilizada, analizar las necesidades reales del profesorado y del alumnado, estudiar las posibilidades que ofrecen las tecnologías actuales y diseñar una herramienta que complemente los métodos tradicionales de enseñanza.

La aplicación resultante debe constituir un recurso de apoyo que facilite el aprendizaje autónomo, fomente la práctica continuada y sirva como punto de encuentro entre los contenidos impartidos en el aula y el trabajo realizado por el alumnado fuera de ella.

3.2 OBJETIVOS ESPECÍFICOS

Con el fin de alcanzar el objetivo general, se han definido los siguientes objetivos específicos:

- **Analizar las necesidades del profesorado y del alumnado de lenguaje musical.**

Este objetivo persigue conocer de primera mano las dificultades, necesidades y expectativas presentes en el proceso de enseñanza y aprendizaje de la asignatura. Para ello, resulta fundamental recopilar información procedente de los potenciales usuarios de la aplicación, garantizando que la solución propuesta responda a problemas reales y aporte un valor añadido dentro del ámbito educativo.

- **Estudiar la aplicación de la informática en la enseñanza del lenguaje musical.**

Antes de plantear cualquier propuesta tecnológica, es necesario conocer el estado actual de las herramientas disponibles y las metodologías empleadas en este ámbito. Este objetivo incluye el

análisis de soluciones existentes, trabajos relacionados y recursos tecnológicos que permitan identificar oportunidades de mejora y establecer una base sólida sobre la que construir la aplicación.

- **Definir una solución informática que responda a las necesidades detectadas.**

A partir de la información obtenida durante las fases de investigación y análisis, se pretende diseñar una solución que cubra las carencias identificadas. Esta definición comprende tanto los requisitos funcionales y no funcionales como el modelado de la aplicación, la arquitectura del sistema y las decisiones de diseño necesarias para garantizar su viabilidad y escalabilidad.

- **Desarrollar e integrar la solución propuesta.**

Una vez definida la arquitectura, el siguiente objetivo consiste en implementar los distintos componentes software que forman la aplicación, integrar sus funcionalidades y verificar su correcto funcionamiento mediante pruebas que permitan garantizar la calidad y estabilidad del sistema.

- **Utilizar tecnologías gratuitas y de libre acceso durante el desarrollo.**

Finalmente, se busca que la totalidad de la solución pueda ser desarrollada, desplegada y mantenida utilizando herramientas y tecnologías gratuitas. Este planteamiento favorece la sostenibilidad del proyecto, reduce los costes asociados a su implantación y facilita tanto su posible adopción por parte de centros educativos como su evolución y mantenimiento por futuros desarrolladores.

La consecución de estos objetivos específicos permitirá desarrollar una solución informática alineada con las necesidades detectadas en el ámbito de la enseñanza del lenguaje musical, cumpliendo así el objetivo general planteado y proporcionando una herramienta que complemente y refuerce los métodos tradicionales de aprendizaje.

3.3 PLANIFICACIÓN

3.3.1 Metodología de trabajo

Durante la fase inicial de planificación se planteó la posibilidad de desarrollar el proyecto siguiendo una metodología ágil inspirada en Scrum. Este enfoque resultaba especialmente atractivo al permitir dividir el trabajo en pequeñas iteraciones, validar de forma continua las funcionalidades implementadas y adaptar el producto a las necesidades que fueran surgiendo durante su desarrollo.

Sin embargo, una vez iniciada la ejecución del proyecto, se comprobó que esta planificación inicial no podía mantenerse de forma realista. La componente de investigación adquirió un peso mucho mayor del previsto y muchas de las decisiones relacionadas con el diseño y la implementación dependían directamente de resultados que todavía no se habían obtenido. Del mismo modo, la naturaleza individual del proyecto dificultaba la aplicación de prácticas propias de Scrum, como la planificación de sprints, la gestión de un backlog o las reuniones periódicas de seguimiento.

Por este motivo, el desarrollo se fue adaptando progresivamente hacia un enfoque más flexible basado en un modelo iterativo e incremental. En lugar de seguir una planificación cerrada de iteraciones, el trabajo se organizó en grandes bloques funcionales que fueron evolucionando conforme avanzaba el proyecto. Cada fase de investigación, análisis o implementación permitió revisar las decisiones tomadas anteriormente e introducir mejoras antes de continuar con el siguiente bloque de trabajo.

Aunque este enfoque no responde a una metodología ágil aplicada de manera estricta, sí mantiene algunos de sus principios fundamentales, como la construcción progresiva de la solución, la validación continua de los resultados obtenidos y la capacidad de adaptación ante nuevos requisitos o conocimientos adquiridos durante el desarrollo.

En consecuencia, la metodología finalmente seguida puede entenderse como una adaptación pragmática de la planificación inicialmente prevista, priorizando la evolución natural del proyecto y la calidad de la solución sobre el cumplimiento estricto de un marco metodológico determinado.

3.3.2 Cronograma del proyecto

Una vez establecido el enfoque metodológico, se definió una planificación temporal que sirviera como guía para organizar las distintas fases del proyecto y distribuir el esfuerzo de manera coherente.

El cronograma elaborado estructura el trabajo en grandes bloques que abarcan desde la investigación inicial y el análisis del problema hasta la definición de la solución, su desarrollo, validación y documentación final. Aunque esta planificación se estableció al inicio del proyecto, su carácter flexible permitió ajustar la duración y el orden de determinadas tareas cuando fue necesario, manteniendo en todo momento el cumplimiento de los objetivos previstos.

La Figura 3.1 representa la distribución temporal de las principales actividades realizadas, mostrando la secuencia seguida durante el desarrollo del proyecto y la relación existente entre las distintas fases que lo componen.

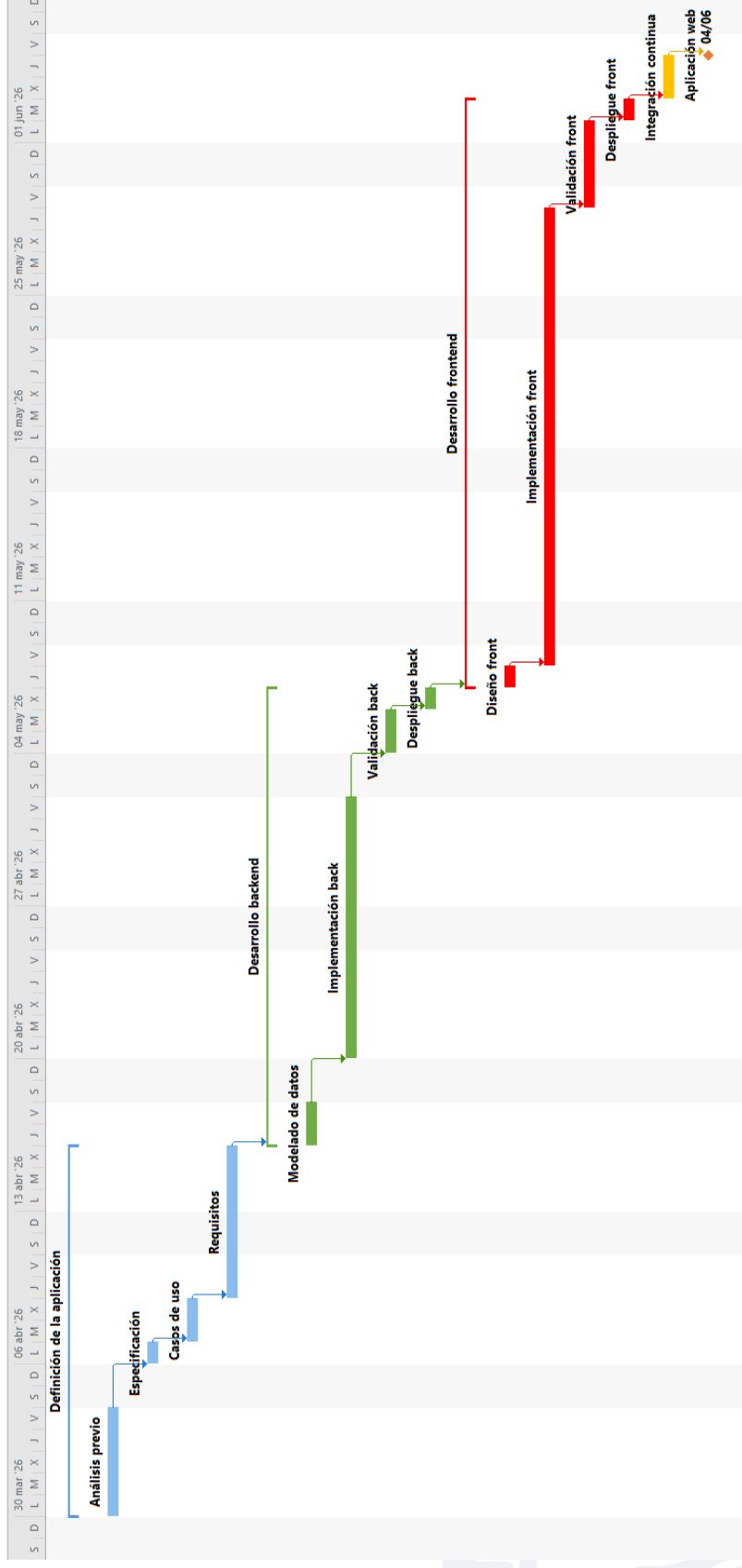


Figura 3.1: Cronograma del proyecto

4 ANÁLISIS, DEFINICIÓN Y ARQUITECTURA DE LA SOLUCIÓN

4.1 INVESTIGACIÓN

4.1.1 Introducción

Como se ha mencionado, se ha llevado a cabo una investigación alrededor de los centros de enseñanza musical, para ello, se han diseñado una serie de encuestas y entrevistas enfocadas tanto al profesorado como al alumnado de cara a comprender sus necesidades. Lo primero que se ha tenido en cuenta son esas diferencias que pueda haber entre los centros públicos y privados, por lo que ha sido necesario visitar dos entornos diferentes, el Real Conservatorio Profesional de Música de Almería (público) y la Escuela de Música de Huércal de Almería (privado).

En ambos centros se ha tenido cita con un profesor de la asignatura para tener una entrevista y entregarles una encuesta para que más tarde fueran devueltas y rellenas por todo el alumnado al cargo de los profesores mencionados. Tras ello, han sido analizadas todas las respuestas y comparadas las del público con las del privado (tanto las entrevistas como las encuestas) que, como se podrá apreciar más tarde, las preguntas también han tenido ligeras modificaciones entre las cuestionadas en un centro y otro.

A través de este método, se han podido conocer las herramientas informáticas usadas actualmente, lo cual ha sido de ayuda para el estudio bibliográfico del capítulo 4.2, y así poder poner el foco sobre las funcionalidades más interesantes.

Por último, se han extraído también del análisis las necesidades del profesorado y el alumnado actuales, lo cual ha sido de gran ayuda para poder definir correctamente las posibles soluciones a la problemática.

4.1.2 Análisis y comparación de las respuestas de los profesores

Las preguntas y respuestas realizadas en las entrevistas de ambos profesores han sido las siguientes las que aparecen en la Figura 4.1

- | | |
|---|---|
| <ol style="list-style-type: none"> 1. ¿Cómo se llama? Juan Vicente García García 2. ¿Qué edad tiene? 61 años 3. ¿Qué formación tiene? (Carreras, títulos, etcétera...) Grado superior de guitarra, grado superior de solfeo y teoría de la música, grado profesional de composición y orquestación y grado profesional de piano 4. ¿Se maneja bien con la informática? Lo justo para trabajar 5. ¿Cree que los recursos informáticos del conservatorio a nivel de Software son suficientes o considera necesaria la creación de más recursos? Hay falta porque, aunque se está sustituyendo el formato tradicional por el moderno se avanza a un ritmo lento. 6. ¿Utiliza los recursos informáticos mencionados con frecuencia en la asignatura? Sí. 7. ¿Es difícil para usted manejarlos? No, son sencillos. 8. ¿Ha invertido mucho tiempo en aprender a usarlos? No 9. ¿Considera oportuna la creación de más recursos informáticos para la asignatura? Sí, mientras sean herramientas útiles son necesarias. 10. ¿Haría un esfuerzo en aprender a usar el nuevo software si cree que merece la pena, o seguiría dando sus clases como lo hacía hasta ahora? Sí 11. ¿Conoces Duolingo o Moodle y lo has usado en alguna ocasión? Sí 12. ¿Qué utilidades como profesor cree que son necesarias en el software creado fijándonos, por ejemplo, en las aplicaciones mencionadas antes? Incluir alguna funcionalidad tipo "cuestionario" ya que el Software actual no lo incluye. 13. ¿Qué ramas considera que requieren de más apoyo informático? Ritmo y audición 14. ¿Por qué? Porque hay menos recursos dedicados a ellas 15. ¿Cree que se debería buscar la creación de un software bonito y atractivo para sus alumnos o mantener el estilo serio y profesional que caracteriza al ámbito académico? Atractivas sobre todo enfocándolo a los alumnos bastante pequeños que la asignatura suele tener 16. ¿Por qué? El alumnado está muy apático actualmente y necesita que la herramienta llame la atención. | <ol style="list-style-type: none"> 1. ¿Cómo se llama? Natalia Botella Galiana 2. ¿Qué edad tiene? 32 años 3. ¿Qué formación tiene? (Carreras, títulos, etcétera...) Grado de pedagogía musical del clarinete 4. ¿Se maneja bien con la informática? Sí. 5. ¿Cree que los recursos informáticos del centro a nivel de Software son suficientes o considera necesaria la creación de más recursos? Hay falta, sobre todo en la teoría y la audición existen carencias de recursos. 6. ¿Cómo se han obtenido los recursos de la escuela? ¿Han sido muy costosos a nivel de tiempo de instalación y dinero? La licencia del Software venía con la compra de los libros que eran bastante caros, en cambio, la instalación fue bastante sencilla. 7. ¿Cree que el organismo encargado de la educación de esta asignatura (Junta de Andalucía) debería ordenar subvenciones o encargarse de crear y obtener algunos recursos informáticos para las escuelas privadas como esta? Sí 8. ¿Utiliza los recursos informáticos mencionados con frecuencia en la asignatura? Sí 9. ¿Es difícil para usted manejarlos? No 10. ¿Ha invertido mucho tiempo en aprender a usarlos? No 11. ¿Considera oportuna la creación de más recursos informáticos para la asignatura? Sí 12. ¿Haría un esfuerzo en aprender a usar el nuevo software si cree que merece la pena, o seguiría dando sus clases como lo hacía hasta ahora? Sí 13. ¿Qué ramas considera que requieren de más apoyo informático? Entonación y audición. 14. ¿Por qué? Porque, aunque teoría tampoco está muy cubierta, la escuela enseña a alumnos en una etapa muy inicial en la cual es más complicado el estudio de la entonación y la audición, por lo que cree que es más necesario fijarse en estas partes. 15. ¿Cree que se debería buscar la creación de un software bonito y atractivo para sus alumnos o mantener el estilo serio y profesional que caracteriza al ámbito académico? Más atractivo enfocado a un alumnado de muy corta edad. 16. ¿Por qué? Porque les vendrá bien que el servicio les llame la atención. |
|---|---|

(a) Profesor del conservatorio

(b) Profesora de la escuela de música

Figura 4.1: Entrevistas

En primer lugar, se puede apreciar que se han cambiado dos preguntas entre entrevistas, esto es debido a que en el ámbito privado era importante obtener información relativa a la obtención de los recursos (lo cual está más claro en lo público) y, por el contrario, era interesante preguntar sobre herramientas de la Junta como Moodle al profesor del Conservatorio (Duolingo era para dar un ejemplo diferente). Tras ver estas respuestas se pueden concluir los siguientes aspectos:

1. A pesar de la diferencia de edad, ambos coinciden en que tienen la habilidad y predisposición suficiente para aprender a usar un nuevo servicio informático siempre y cuando sea intuitivo debido a la escasez y necesidad de recursos en ambos sectores (público y privado).
2. Ambos le dan bastante uso a los recursos que tienen actualmente.

3. También coinciden que la audición está menos cubierta, en cambio, en lo público parece que en el ritmo existe una mayor necesidad y en lo privado en la entonación. Aquí es importante señalar que las escuelas de música solo se suelen encargar de alumnos en etapas iniciadas mientras en los conservatorios sí se profundiza más en la asignatura, por tanto, si los alumnos necesitan más ayuda en la entonación en una etapa tan primeriza es normal que se ponga el foco más en ese aspecto mientras que en unas enseñanzas más avanzadas quizás no sea tan necesario.
4. Por último, también están de acuerdo en que la herramienta debería ser atractiva. Hay que poner atención a que se mencionó la necesidad de una funcionalidad tipo **cuestionario**.

4.1.3 Análisis y comparación de las respuestas de los alumnos

La encuesta realizada para los alumnos ha sido la misma para ambos centros, ya que no se ha considerado que personas de una edad tan temprana puedan apreciar diferencias en función del centro. Las preguntas realizadas aparecen en la próxima Figura 4.2

Encuesta al alumnado:

1. ¿Tiene un buen manejo con las herramientas informáticas que dispone su centro para el lenguaje musical (aplicaciones de móvil, programas de ordenador o páginas web que apoyen el aprendizaje de dicha asignatura)?

Sí ☐No ☐

2. Del 1 al 10 ¿Qué grado de dificultad presenta el uso de estas aplicaciones? (Siendo 1 extremadamente fácil y 10 extremadamente difícil):

3. ¿Consideras necesario que el centro disponga de más herramientas informáticas para la asignatura mencionada?

Sí ☐No ☐

4. Si ha respondido "sí" a la pregunta anterior, ¿Cuál o cuáles de las cuatro partes que se estudian cree que tienen una mayor falta de aplicaciones?

Ritmo ☐Entonación ☐Teoría ☐Audición ☐

5. ¿Por qué cree que las opciones que ha marcado en la pregunta anterior requieren de apoyo informático?

6. ¿Cree que la herramienta informática debería verse colorida, bonita y atractiva? ¿O piensa que debería de ser más seria y profesional? (siendo esto último más propio en este ámbito académico)

Colorida y atractiva ☐Seria y profesional ☐

7. ¿Por qué?

!!!!Gracias por responder a la encuesta!!!!

Figura 4.2: Encuesta al alumnado

Se exponen en las gráficas de la Figura 4.3 las respuestas a las preguntas 1, 3 y 6 en primer lugar, se debe tener en cuenta que el total de respuestas de cada pregunta de ambas gráficas varía un poco debido a que, en algunos casos, dicha respuesta estaba en blanco o no era adecuada.

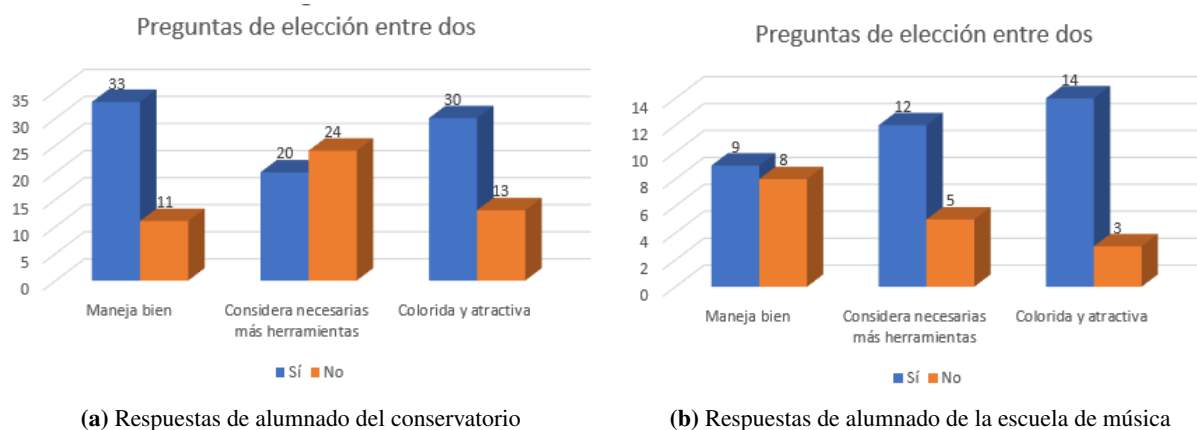


Figura 4.3: Respuestas a las preguntas 1, 3 y 6

Las respuestas de la pregunta 4 también han sido reflejadas en las gráficas de la Figura 4.4

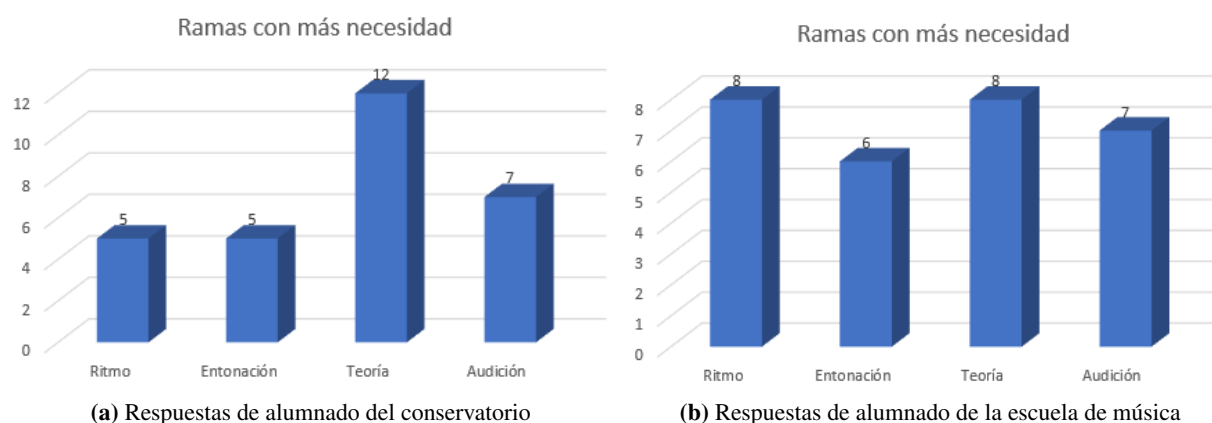


Figura 4.4: Respuestas a las pregunta 4

Cabe destacar que la media en la pregunta 2 ha sido de **4.68** para el conservatorio y de **2.58** para la escuela de música.

Para las preguntas 5 y 7 se han seleccionado las mejores respuestas:

- Mejores respuestas de la pregunta 5:
 - (Marcó todas las opciones en la pregunta 4) “Ninguna tiene apoyo informático y es necesario poder acceder al material desde casa”.
 - (Marcó teoría en la pregunta 4) “Porque se debe poder acceder al contenido de clases anteriores para que no caiga en el olvido”.
 - (Marcó teoría en la pregunta 4) “Porque en las calificaciones de las actividades de teoría actualmente no se puede saber en qué te has equivocado”.



■ Mejores respuestas de la pregunta 7:

- (Marcó ambas en la pregunta 6) “Creo que podría ser ambas cosas”.
- (Marcó colorida y atractiva en la pregunta 6) “Al llamar más la atención se crea un ambiente más llevadero y también es importante disfrutar un poco de nuestras responsabilidades”.

Por lo tanto, se sacan las siguientes conclusiones:

1. En la escuela de música están mucho más de acuerdo en querer más herramientas informáticas y, a su vez, les cuesta menos el uso de las herramientas actuales, muy posiblemente sea debido a que, en el conservatorio, el alumnado es generalmente mayor que en la escuela de música además de que, por experiencia personal, suele haber apuntados más adultos y es muy posible que estén menos dispuestos al uso de nuevos servicios.
2. Están bastante de acuerdo en que el software debe ser colorido y atractivo.
3. En contradicción con los profesores, creen más necesario reforzar los recursos informáticos dedicados a la teoría. También encuentran necesidad en la audición y el ritmo mientras que la entonación quedaría un peldaño por debajo.
4. Algunos aspectos referentes a la funcionalidad de software mencionados en las respuestas son la necesidad de poder acceder a material antiguo y desde casa y poder saber por qué se ha cometido un error en las correcciones de teoría.

4.1.4 Herramientas actuales

Ambos profesores se sirven de la aplicación web **EstudioMusica** [11] y se observa que las escuelas de música generalmente también usan herramientas incluidas con el material físico (libros de texto por ejemplo).



4.1.5 Necesidades detectadas

Mezclando las necesidades de profesorado y alumnado, se llega a las siguientes conclusiones:

1. Es muy necesario apoyar a los centros de educación musical en cuanto a recursos informáticos debido a su escasez.
2. En general, profesorado y alumnado está a favor de incentivar la informática en las aulas.
3. Es de vital importancia que las aplicaciones sean coloridas y atractivas.
4. Las ramas más importantes son la auditiva por parte del profesorado y la teórica por parte del alumnado. También se ha mencionado que podría ser muy útil reforzar la rama de la entonación en las etapas iniciales del estudio de la asignatura.
5. Es necesario crear una funcionalidad tipo cuestionario, en la cual, se puedan añadir las correcciones para que sea autocorregido. También podría ser útil que el servicio fuera accesible desde casa y el material antiguo se mantuviera en el mismo.

4.2 ESTUDIO BIBLIOGRÁFICO

4.2.1 Introducción

El siguiente aspecto a tratar es la bibliografía. Una vez definidas unas conclusiones de la investigación previa, es necesario conocer qué documentación existe relacionada con el ámbito abarcado para extraer y sintetizar una información de interés de cara a la construcción de la solución. Por lo tanto, es necesario establecer unos pasos a seguir que permitan obtener la documentación necesaria, de manera que, se ha elegido hacer uso de la MLR (Mapping Literature Review [32]), ya que establece una metodología marcada propia del dominio de la informática.

Los pasos a seguir son los siguientes:

1. Definición de la pregunta de interés y los criterios de inclusión y exclusión de los estudios.
2. Localización y selección de la documentación relevante.
3. Síntesis de la información de la selección.
4. Análisis e interpretación de los resultados presentados.

4.2.2 Mapping Literature Review

Definición la cadena de búsqueda y criterios

En primer lugar, se deben tener en cuenta los siguientes componentes clave:

- Población específica y contexto: Áreas de conocimiento del software enfocado a la enseñanza del lenguaje musical.
- Intervención: Funcionalidades y características interesantes desde el punto de vista pedagógico del lenguaje musical.
- Comparación: Otros software dedicados a la enseñanza del lenguaje musical.
- Resultado: Necesidades y conclusiones para una solución.

Tras ello, se formulan una serie de preguntas en base a lo anterior:

1. ¿Qué softwares son más usados en la enseñanza del lenguaje musical?
2. ¿Qué características debe tener un software para que sea pedagógico desde el punto de vista musical?
3. ¿Qué características son más importantes en un software para la asignatura?
4. ¿Qué necesidades no están cubiertas por el software actual?

Una vez identificadas las preguntas, es necesario crear una cadena de búsqueda que se encargue de encontrar documentación dedicada tan solo a responderlas. La cadena sería la siguiente:

("software" OR "system" OR "application" OR "service") AND ("musical language teaching" OR "musical language education" OR "musical language pedagogy") AND ("requirements" OR "characteristics" OR "functionality")

Pero debido a que ninguna biblioteca daba resultados (excepto Google Scholar por ser muy genérica), se han tenido que disminuir en gran medida los términos de la cadena de búsqueda finalmente:

("software" OR "system" OR "application" OR "service") AND ("pedagogy" OR "teaching") AND "musical language"

Esto es consecuencia de que los temas tratados son muy dispares y es muy posible que exista muy poca documentación en la que se encuentren relacionados.

Por último, se han seleccionado los siguientes criterios de exclusión:

- No es accesible (documento privado o de pago).
- No abarca cualquiera de los dos grandes temas (informática y/o lenguaje musical) y/o no los relaciona en ningún punto.
- No está escrito en inglés o español.



Selección de documentación relevante

Los motores de búsqueda utilizados han sido: Google Scholar, Scopus, Association for Computing Machinery (ACM) Library y UVAdoc, este último, porque al hacer búsquedas previas en crudo para analizar los campos, en la universidad de Valladolid (a la cual pertenece el repositorio mencionado), ya se habían hecho algunos TFGs similares. Para el motor de búsqueda de Google se han analizado tan solo los primeros 50 resultados ya que, a partir de ahí, comenzaban a tener muy poca relevancia. Todo esto aparece esquematizado en el flujograma de la Figura 4.5

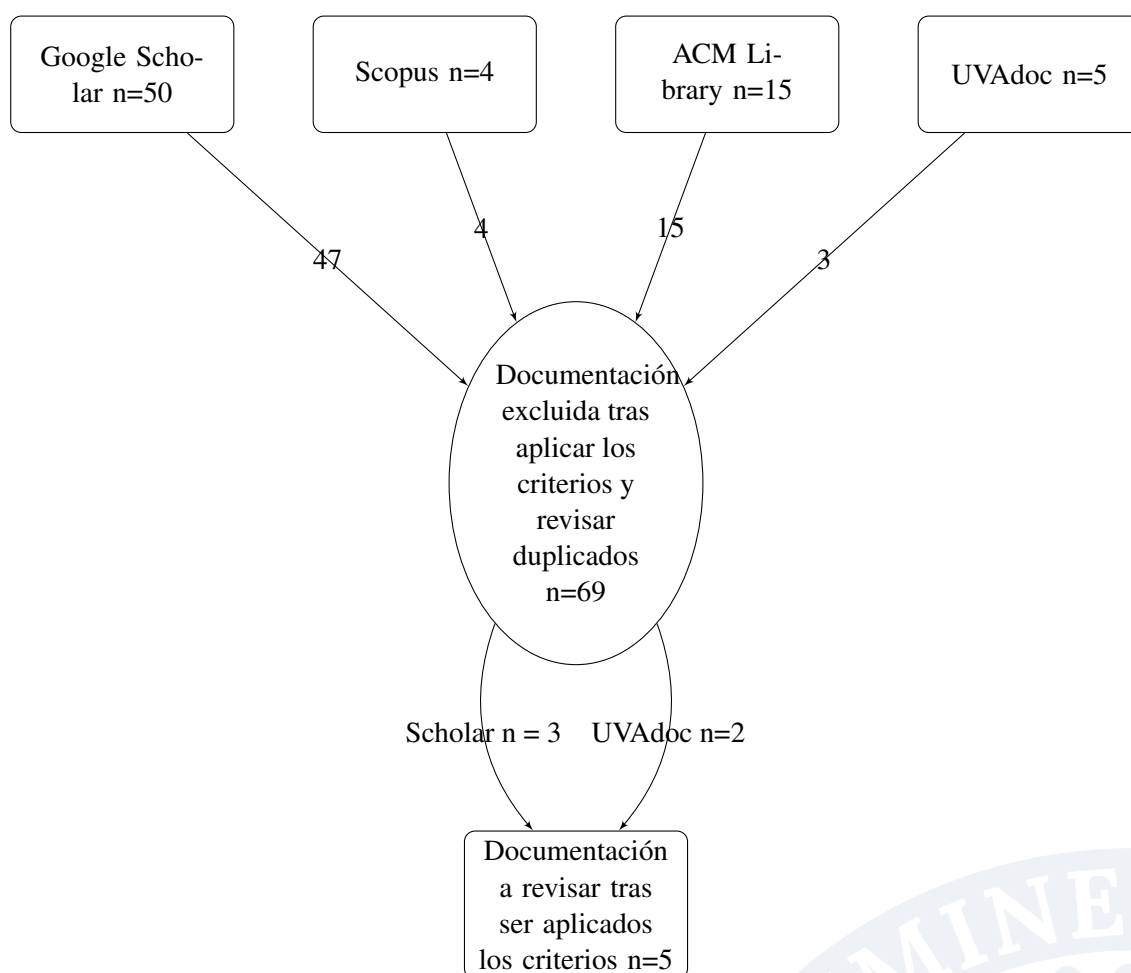


Figura 4.5: Flujograma de la Mapping Review

A pesar de la baja cantidad de documentos a revisar, abrir los términos de búsqueda para obtener más resultados no es una buena opción puesto que, el número de documentos irrelevantes ya es grande y solo se obtendrían aún más resultados innecesarios. Esto se debe a que, el marco teórico establecido que combina la informática con la música está bastante poco explorado aún.

Se puede comprobar la documentación obtenida (ordenada de por fecha de publicación descendente) en la Figura 4.1.

Fecha	Documentación recogida		
Datos	Sitio de publicación	Tipo	Título
2022	UVAdoc	Trabajo técnico	El lenguaje musical a través de las TIC en educación primaria
2018	AtlantisPress	Artículo	Music Computer Technologies and Interactive Systems of Education in Digital Age School
2017	UVAdoc	Trabajo técnico	Aplicación informática para la introducción del lenguaje musical (Leemusica) en dispositivos móviles
2015	Hejmec Journal	Artículo	Online Gaming to Learn Music and English Language in Music and Ballet School Solfeggio Education
2014	Life Science Journal	Artículo	Multimedia technologies as a tool for teaching supervision over the students' skills (within the course on "music theory training")

Tabla 4.1: Documentación recogida

Resultados obtenidos

Como se ha descrito al principio de la revisión, el objetivo es encontrar soluciones o respuestas a una serie de problemáticas concretas que nos permitirán deducir una serie de conclusiones válidas para el proyecto. Para ello, se hará una breve síntesis de cada uno de los documentos enfocada en responder a las preguntas expuestas.

El lenguaje musical a través de las TIC en educación primaria Este primer documento es un Trabajo de Fin de Grado de la Universidad de Valladolid que consiste en la creación de un blog que funcione como apoyo para la enseñanza del lenguaje musical, así pues, se apoya mucho a su vez, en otro tipo de herramientas centrando su proyecto en la innovación, el ámbito pedagógico y el método de aprendizaje que deben seguir los alumnos sobre todo teniendo en cuenta que son en su mayoría de edades muy tempranas. Las herramientas en las que hace énfasis y en las que se podría poner el foco son editores de partituras como MuseScore o Noteflight, editores de audio como Audacity y aplicaciones de encuestas o juegos como Kahoot o Quizizz, en consecuencia, se deduce que las funcionalidades más interesantes que expone son las que aportan dichas herramientas.

También, al tocar la parte de la innovación, insiste en el punto de que es contradictorio el hecho de que la asignatura sea impartida desde un punto de vista tan teórico, puesto que, es muy importante para el músico tener una formación práctica siendo esta, además, una vía mucho más amena, por ello, es importante por la parte pedagógica que el alumno interaccione por su cuenta y tenga forma de trabajar los contenidos sin necesidad del estudio de memorización establecido actualmente. Por último, destaca que es importante llevar a cabo este proceso poco a poco, pues hay aún muchos profesores que están habituados a un sistema muy teórico y tradicional sin apoyarse demasiado en las TIC y que sería imposible realizar una adaptación de golpe, debido a ello, es importante encontrar un balance entre las innovaciones y el sistema actual [33].

Music Computer Technologies and Interactive Systems of Education in Digital Age School En segundo lugar, se encuentra un artículo a modo de análisis de la influencia y presencia de la tecnología en el aprendizaje de la música así como sus ventajas e inconvenientes. Además, se comenta sobre un nuevo software llamado "Soft Way to Mozart System", el cual, trata de paliar esas necesidades que actualmente se presentan a la hora de aprender a tocar un instrumento. Al parecer, en este caso, trata más del aprendizaje del instrumento en sí que del lenguaje musical, aún así se nos aporta la información de que, mediante el software, existe una posibilidad muy grande de apoyar la enseñanza de la música en general. Además, cabe destacar, que una de las claves es que los alumnos, al ser de edades tempranas, es muy importante que puedan interactuar con las herramientas y no sea el profesor el único que lo haga [34].

Aplicación informática para la introducción del lenguaje musical En el siguiente documento, se presenta otro Trabajo de Fin de Grado de la Universidad de Valladolid que consiste en el desarrollo de una aplicación para la enseñanza del lenguaje musical (más enfocado al desarrollo que el anterior) en edades tempranas como educación infantil. Vuelve a nombrar una serie de herramientas compatibles con el aprendizaje de la asignatura como MuseScore, Audacity o creadores de encuestas/actividades e incluso algunas nuevas como Hydrogen que sirve para crear patrones de percusión con los que trabajar el ritmo con los alumnos y Band-in-a-Box para patrones melódicos.

En este caso, el objetivo de la herramienta es que los alumnos sean partícipes de la herramienta puesto que, para alumnos de educación infantil, los recursos creados hasta ahora los tiene que usar el profesor debido a su dificultad. Para ello, centra mucho su aplicación en la pedagogía a través del diseño, no solo haciéndolo llamativo, sino también usando colores para diferenciar notas entre otros recursos psicológicos. Por último, vuelve a hacerse énfasis en que la innovación no debe llegar de forma brusca, en este caso, debido a que la tecnología es un recurso complementario y no puede condicionar los contenidos que se dan en la asignatura, así como quitar demasiado tiempo al profesor para la enseñanza de los mismos [35].

Online Gaming to Learn Music and English Language in Music and Ballet School Solfeggio Education A continuación, se trata un artículo que se centra en el enriquecimiento que ganan los alumnos de música y otras artes en general al jugar videojuegos exponiendo que, a día de hoy, es muy importante desarrollar más recursos TIC enfocados en la música. El documento aporta una serie de páginas web con juegos on-line musicales que se pueden usar como software de referencia, estas son "Classics for Kids Games" y "New York Philharmonic Kidzone" [36].

Multimedia technologies as a tool for teaching supervision over the students' skills (within the course on “music theory training”) Por último, se revisa este artículo que consiste en la búsqueda de métodos para poder evaluar las habilidades del alumno de música, y parece ser que el mejor para ello es las tecnologías multimedia. También incide en las competencias a evaluar y la metodología. Para el proyecto actual, tan solo es importante tomar en cuenta lo mencionado al principio sobre que es muy importante establecer la posibilidad de que se pueda llevar una supervisión y evaluación del alumno mediante los recursos TIC [37].

Conclusiones

Tras analizar los diferentes documentos se sacan las siguientes conclusiones:

1. Los software que se recomiendan, en general, son herramientas de apoyo que no pueden abarcar el grueso del temario de la asignatura y, además, son recursos muy completos que no pueden ser introducidos en este trabajo ya que por sí solos son un proyecto muy completo (línea de trabajo futura), exceptuando los juegos y cuestionarios en los que sí podría ser interesante poner el foco.
2. Se insiste mucho en la pedagogía en todos los documentos ya que la asignatura es impartida a personas de edades muy tempranas (desde 8 a 14 años generalmente), de hecho, no es importante tan solo crear un diseño llamativo sino que también la propia discriminación de colores u otras funcionalidades del tipo pueden ayudar a obtener una serie de enseñanzas concretas al alumnado, además de ayudar a personas con distintos tipos de problemas visuales u otras patologías y, sobre todo, que sea sencillo de usar para que pueda interactuar el alumnado y para que pueda ser gestionada por el profesorado sin demasiada formación en las TIC.
3. Las características señaladas, por tanto, podrían ser, el desarrollo de cuestionarios, atender al diseño responsive, usabilidad y seguimiento y evaluación del alumnado. También sería interesante que, para hacer la experiencia más completa, cada rama tenga alguna funcionalidad especial ajustada para apoyar su ciclo de aprendizaje consiguiendo que los usuarios no requieran de otros recursos para abarcar los contenidos fuera del dominio de la solución software.
4. Actualmente, el principal problema de las herramientas no es la falta de funcionalidades, sino el hecho de que no pueden abarcar la asignatura por sí solas, en consecuencia, es necesario aunar las funcionalidades más básicas para la asignatura en una sola herramienta que, además, permita establecer un sistema de evaluación para que pueda ser el epicentro de la enseñanza de la asignatura y, adicionalmente, sea fácil de manejar por todo tipo de usuarios.

4.3 ESPECIFICACIÓN FORMAL DE LA SOLUCIÓN

El sistema permitirá el acceso mediante un mecanismo de inicio de sesión en el que podrán autenticarse tres tipos de usuarios: administrador, profesor y alumno. Cada usuario tendrá acceso únicamente a las funcionalidades correspondientes a su rol, sin posibilidad de utilizar las funciones asignadas a otros.

No existirá un proceso de registro. El administrador será creado de forma automática y, una vez dentro de la aplicación, tendrá la capacidad de crear las cuentas de los profesores manualmente. Para ello, deberá introducir los siguientes datos: nombre, primer apellido, segundo apellido y dirección de correo electrónico. Se realizará una comprobación de validez de los datos introducidos (formato correcto del nombre, de los apellidos y del correo electrónico) y una verificación de unicidad del correo electrónico en la base de datos.

En el proceso de creación, el sistema generará automáticamente un nombre de usuario a partir de la primera letra del nombre y de los apellidos. Por ejemplo, para un usuario denominado José Luis Garrido Castro, el identificador generado será jgc. En caso de que existan coincidencias con identificadores ya presentes en la base de datos, se añadirá un número consecutivo equivalente al total de usuarios con la misma raíz más uno, de modo que si ya existen 40 usuarios con identificadores iniciados en jgc, el nuevo identificador será jgc41. La contraseña será generada automáticamente mediante el uso de un servicio externo de generación de contraseñas seguras. Una vez creadas las credenciales, estas serán enviadas por correo electrónico al profesor.

El profesor, a su vez, tendrá la capacidad de crear cuentas de alumnos manualmente mediante un procedimiento equivalente al descrito para la creación de manual de profesores.

Además, el profesor podrá crear grupos de clase y añadir alumnos a los mismos. Un alumno podrá pertenecer a varios grupos de forma simultánea.

También podrá enviar mensajes a unos alumnos o grupos completos determinados y podrá adjuntar las calificaciones generales del curso seleccionando a un alumno en concreto.

El sistema dispondrá de un área denominada administración en la que estarán disponibles las operaciones mencionadas anteriormente.

Fuera de esta sección, la aplicación contará con cuatro apartados denominados **Ritmo, Entonación, Audición y Teoría**. Cada apartado estará compuesto por dos secciones diferenciadas:

1. **Libro del curso:** Permitirá al profesor subir un único archivo en formato PDF correspondiente al curso seleccionado. Si se sube un nuevo archivo, el anterior será sustituido. El profesor podrá eliminar o reemplazar el archivo en cualquier momento.
2. **Tareas a realizar:** Permitirá al profesor crear tareas asociadas a un curso determinado. Cada tarea constará de una descripción y, opcionalmente, de un material de apoyo en formato PDF (o mp3 en caso de estar en la rama de entonación). También se definirá una fecha límite de entrega. El profesor podrá calificar y añadir una observación a la entrega individualmente, modificarse y/o eliminarse la tarea en cualquier momento, pero no se eliminarán automáticamente al llegar la fecha de entrega; se mantendrán hasta la finalización del curso. En el apartado de Teoría, las tareas

podrán sustituirse (o no) por un cuestionario tipo test. Dicho cuestionario permitirá la creación de preguntas con opciones de respuesta, y cada pregunta podrá ir acompañada de un archivo en formato MP3.

Asimismo, los alumnos podrán acceder a los mismos cuatro apartados (Ritmo, Entonación, Audición y Teoría). En cada apartado se mostrarán las mismas dos secciones, y en algunos casos tres:

- En la sección Libro del curso, los alumnos podrán visualizar el archivo PDF correspondiente y descargarlo, pero no modificarlo.
- En la sección Tareas a realizar, los alumnos podrán consultar la lista de tareas disponibles y realizar las entregas correspondientes. Los formatos permitidos serán PDF, MP3 o MP4, y/o bien un comentario textual dentro de la propia aplicación. En el caso de los cuestionarios, el alumno podrá acceder de forma específica a cada uno para completarlos. El cuestionario no se considerará entregado hasta que el alumno pulse expresamente el botón de envío.
- Esta sección es solo visible por los alumnos y cambiará en función de la rama. En la rama de entonación aparecerá una funcionalidad tipo metrónomo y en la rama de audición aparecerá un pequeño piano virtual. Esta sección no aparecerá en las otras dos ramas

Los alumnos y profesores dispondrán de un área específica de mensajes. En ella recibirán avisos cuando se cree, se modifique, se califique (alumno) o se entregue (profesor) una tarea o cuestionario, cuando reciban una calificación general (mensajes del sistema) o un mensaje del profesor. Por último, ambos perfiles de usuario podrán cambiar su contraseña desde la aplicación.

4.4 OBJETIVOS FUNCIONALES DEL SISTEMA

El objetivo principal del sistema es facilitar y mejorar la enseñanza del lenguaje musical mediante una plataforma que proporcione funcionalidades específicas tanto para profesores como para alumnos. Estos objetivos funcionales guían el desarrollo del sistema y aseguran que las necesidades pedagógicas y administrativas se cubran de forma eficiente y adaptada a las diferentes ramas del aprendizaje musical. Los objetivos funcionales que se persiguen son los siguientes:

- Permitir al profesorado crear, gestionar y visualizar grupos de alumnos, facilitando la organización y seguimiento académico.
- Gestionar el contenido educativo estructurado según las ramas del lenguaje musical: teoría, entonación, ritmo y audición.
- Proporcionar herramientas para que el profesor pueda crear, modificar, cerrar y eliminar tareas y cuestionarios específicos para cada rama.
- Facilitar la interacción del alumnado con las tareas, cuestionarios y las distintas funcionalidades de las ramas, incluyendo la entrega de trabajos y la realización de pruebas con autocorrección cuando corresponda.

- Permitir al profesorado revisar, calificar y gestionar las entregas y calificaciones de los alumnos de forma clara y accesible.
- Implementar un sistema de mensajería interna que facilite la comunicación entre profesores y alumnos, con notificaciones automáticas vinculadas a eventos relevantes.
- Permitir la gestión del material didáctico asociado a cada rama del aprendizaje musical, con capacidad de adaptación y personalización según las necesidades específicas de cada grupo de alumnos.

Estos objetivos funcionales proporcionan un marco claro para la posterior definición de los casos de uso y requisitos funcionales, asegurando que el sistema responda adecuadamente a las necesidades del entorno educativo para el que está diseñado.

4.5 DIAGRAMA UML DE CASOS DE USO DEL SISTEMA

Una vez definida la solución mediante lenguaje natural, se da el siguiente paso hacia el modelado de casos de uso.

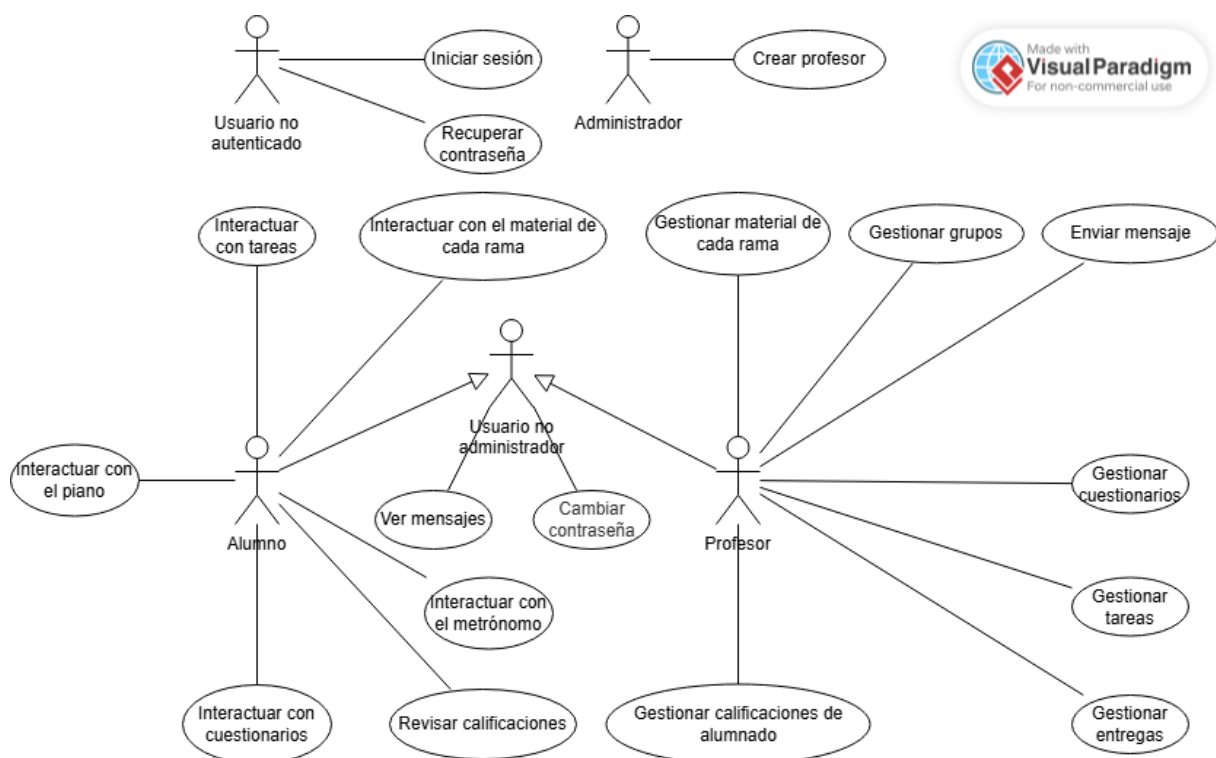


Figura 4.6: Diagrama UML de casos de uso del sistema

4.6 DEFINICIÓN DEL DIAGRAMA DE CASOS DE USO

El diagrama de casos de uso aporta una visión muy genérica y poco detallada y, aunque es mucho más objetiva que el lenguaje natural, se encuentra en un nivel de abstracción muy alto, por lo tanto, es necesario entrar en más detalle para cada uno de los elementos del diagrama.

4.6.1 Actores

Se pueden observar cuatro actores principales, cada uno con su respectivo rol: Además, el diagrama

ID del actor	ACT-1
Nombre del actor	Usuario no autenticado
Rol	Usuario que accede al sistema con el único objetivo de iniciar sesión.

Tabla 4.2: Actor 1

ID del actor	ACT-2
Nombre del actor	Administrador
Rol	Usuario autenticado con el objetivo de crear a los usuarios con rol de profesor en el sistema.

Tabla 4.3: Actor 2

ID del actor	ACT-3
Nombre del actor	Profesor
Rol	Usuario autenticado con el objetivo de crear usuarios con el rol alumno y gestionar el contenido académico dirigido a los mismos en el sistema.

Tabla 4.4: Actor 3

ID del actor	ACT-4
Nombre del actor	Alumno
Rol	Usuario autenticado con el objetivo de acceder al contenido e interactuar con el mismo a través del sistema.

Tabla 4.5: Actor 4

muestra un actor común que aglutina los casos de uso de los actores profesor y alumno.

ID del actor	ACT-5
Nombre del actor	Usuario no administrador
Rol	Usuario autenticado ficticio con el objetivo de representar las funcionalidades comunes a los actores profesor y alumno.

Tabla 4.6: Actor 5

4.6.2 Casos de uso

Para definir los casos de uso del sistema la mejor forma de hacerlo es, como en cualquier otro proyecto de desarrollo de software, mediante los requisitos funcionales que lo conforman.

Definición general

Nombre	Iniciar sesión
Descripción	Permitir al usuario no autenticado iniciar sesión en el sistema proporcionando sus credenciales y, en caso de éxito, tendrá acceso a las funcionalidades del actor que tenga asignado en el sistema.
Actor	ACT-1
Requisitos funcionales	RF-01

Tabla 4.7: Caso de uso: Iniciar sesión

Nombre	Recuperar contraseña
Descripción	Permitir al usuario recuperar el acceso a su cuenta mediante la verificación de su dirección de correo.
Actor	ACT-1
Requisitos funcionales	RF-02

Tabla 4.8: Caso de uso: Recuperar contraseña

Nombre	Crear profesor
Descripción	Permitir al administrador crear nuevos usuarios con rol de profesor en el sistema, proporcionando los datos necesarios y validando la información.
Actor	ACT-2
Requisitos funcionales	RF-03

Tabla 4.9: Caso de uso: Crear profesor

Nombre	Crear alumno
Descripción	Permitir al profesor crear nuevos usuarios con rol de alumno en el sistema, proporcionando los datos necesarios y validando la información.
Actor	ACT-3
Requisitos funcionales	RF-05 RF-04

Tabla 4.10: Caso de uso: Crear alumno

Nombre	Gestionar grupos
Descripción	Permitir al profesor crear nuevos grupos, revisar los que ha creado y elegir un grupo para trabajar con él.
Actor	ACT-3
Requisitos funcionales	RF-06 RF-07 RF-08

Tabla 4.11: Caso de uso: Gestionar grupos

Nombre	Enviar mensaje
Descripción	Permitir al profesor enviar mensajes a los alumnos de un grupo seleccionado.
Actor	ACT-3
Requisitos funcionales	RF-09

Tabla 4.12: Caso de uso: Enviar mensaje

Nombre	Gestionar material de cada rama
Descripción	Permitir al profesor gestionar el material de cada rama de la asignatura.
Actor	ACT-3
Requisitos funcionales	RF-10 RF-11 RF-12

Tabla 4.13: Caso de uso: Gestionar material de cada rama

Nombre	Gestionar tareas
Descripción	Permitir al profesor gestionar las tareas de cada rama de la asignatura.
Actor	ACT-3
Requisitos funcionales	RF-13 RF-14 RF-15 RF-16 RF-17

Tabla 4.14: Caso de uso: Gestionar tareas

Nombre	Gestionar cuestionarios
Descripción	Permitir al profesor gestionar los cuestionarios de la rama de teoría de la asignatura.
Actor	ACT-3
Requisitos funcionales	RF-18 RF-19 RF-20 RF-21 RF-22

Tabla 4.15: Caso de uso: Gestionar entregas

Nombre	Gestionar entregas
Descripción	Permitir al profesor gestionar las entregas de las tareas de los alumnos.
Actor	ACT-3
Requisitos funcionales	RF-23 RF-24 RF-25

Tabla 4.16: Caso de uso: Gestionar entregas

Nombre	Gestionar calificaciones del alumnado
Descripción	Permitir al profesor gestionar las calificaciones generales del curso de los alumnos.
Actor	ACT-3
Requisitos funcionales	RF-26 RF-27

Tabla 4.17: Caso de uso: Gestionar calificaciones del alumnado

Nombre	Ver mensajes
Descripción	Permitir al alumno y al profesor ver los mensajes.
Actor	ACT-5
Requisitos funcionales	RF-28

Tabla 4.18: Caso de uso: Ver mensajes

Nombre	Cambiar contraseña
Descripción	Permitir al alumno y al profesor cambiar su contraseña.
Actor	ACT-5
Requisitos funcionales	RF-31

Tabla 4.19: Caso de uso: Cambiar contraseña

Nombre	Interactuar con el material de cada rama
Descripción	Permitir al alumno interactuar con el material de cada rama.
Actor	ACT-4
Requisitos funcionales	RF-10

Tabla 4.20: Caso de uso: Interactuar con el material de cada rama

Nombre	Revisar calificaciones
Descripción	Permitir al alumno revisar las calificaciones.
Actor	ACT-4
Requisitos funcionales	RF-25

Tabla 4.21: Caso de uso: Revisar calificaciones

Nombre	Interactuar con tareas
Descripción	Permitir al alumno interactuar con las tareas.
Actor	ACT-4
Requisitos funcionales	RF-13 RF-29

Tabla 4.22: Caso de uso: Interactuar con tareas

Nombre	Interactuar con cuestionarios
Descripción	Permitir al alumno interactuar con los cuestionarios.
Actor	ACT-4
Requisitos funcionales	RF-18 RF-30

Tabla 4.23: Caso de uso: Interactuar con cuestionarios

Nombre	Interactuar con el metrónomo
Descripción	Permitir al alumno interactuar con el metrónomo.
Actor	ACT-4
Requisitos funcionales	RF-32 RF-33

Tabla 4.24: Caso de uso: Interactuar con el metrónomo

Nombre	Interactuar con el piano
Descripción	Permitir al alumno interactuar con el piano.
Actor	ACT-4
Requisitos funcionales	RF-34

Tabla 4.25: Caso de uso: Interactuar con el piano**Requisitos funcionales**

RF-01	Iniciar sesión
Descripción	Permitir al usuario no autenticado iniciar sesión en el sistema proporcionando sus credenciales y, en caso de éxito, tendrá acceso a las funcionalidades del actor que tenga asignado en el sistema.
Precondición	El usuario no debe estar autenticado en el sistema.
Secuencia de pasos	1. El usuario accede a la pantalla de inicio de sesión. 2. El usuario introduce su nombre de usuario y contraseña. 3. El sistema valida las credenciales introducidas. 4. Si las credenciales son correctas, el sistema autentica al usuario. 5. El sistema asigna al usuario el rol correspondiente y concede acceso a las funcionalidades habilitadas. 6. El usuario es redirigido a la página principal o panel correspondiente a su rol.
Postcondición	El usuario queda autenticado en el sistema con acceso a las funcionalidades correspondientes a su rol.
Excepciones	■ El usuario introduce credenciales incorrectas. ■ Problemas de conexión con la base de datos.

Tabla 4.26: Requisito funcional: RF-01 Iniciar sesión

RF-02	Recuperar contraseña
Descripción	Permitir al usuario recuperar el acceso a su cuenta mediante la verificación de su dirección de correo.
Precondición	El usuario debe tener acceso al correo electrónico asociado a su cuenta y no estar autenticado en el sistema.
Secuencia de pasos	1. El usuario accede a la opción "Recuperar contraseña" desde la pantalla de inicio de sesión. 2. El usuario introduce su dirección de correo electrónico registrada. 3. El sistema verifica que el correo electrónico existe en la base de datos. 4. El sistema actualiza la contraseña. 5. El sistema envía la contraseña por correo y notifica al usuario del éxito.
Postcondición	El usuario puede iniciar sesión con la nueva contraseña establecida.
Excepciones	■ El correo electrónico proporcionado no está registrado en el sistema. ■ El sistema no puede conectarse a la base de datos o servicio de correo.

Tabla 4.27: Requisito funcional: RF-02 Recuperar contraseña

RF-03	Crear profesor
Descripción	Permitir al administrador crear nuevos usuarios con rol de profesor en el sistema, proporcionando los datos necesarios.
Precondición	El administrador debe estar autenticado en el sistema.
Secuencia de pasos	1. El administrador introduce los datos requeridos del profesor (nombre, apellidos y correo). 2. El sistema valida los datos mediante RF-04. 3. El sistema guarda el nuevo profesor. 4. El sistema notifica al administrador que el profesor ha sido creado correctamente.
Postcondición	El nuevo profesor queda registrado en el sistema y puede iniciar sesión con las credenciales asignadas.
Excepciones	■ Datos incompletos o inválidos introducidos por el administrador. ■ Problemas de conexión con la base de datos.

Tabla 4.28: Requisito funcional: RF-03 Crear profesor

RF-04	Validar y generar automáticamente las credenciales
Descripción	Validar la información de creación de un nuevo usuario comprobando la unicidad del correo, formatos específicos para nombre, apellidos y correo, y generar automáticamente el username y la contraseña.
Precondición	Se reciben datos completos del profesor para validar y generar credenciales.
Secuencia de pasos	1. El sistema comprueba que el correo no está registrado. 2. El sistema valida que el correo tenga formato tipo ejemplo@ejemplo.com. 3. El sistema valida que el nombre y apellidos solo contengan letras sin espacios ni caracteres inválidos. 4. El sistema genera el username a partir de las siglas de nombre y apellidos. 5. Si el username generado ya existe, se añade un identificador numérico para diferenciarlo. 6. El sistema solicita a una API externa la generación de una contraseña segura. 7. El sistema asocia el username y contraseña generados a la cuenta del profesor.
Postcondición	Se dispone de credenciales únicas y válidas para el nuevo profesor, listas para su uso en el sistema.
Excepciones	■ El correo proporcionado ya está registrado. ■ Los datos no cumplen con el formato requerido. ■ Fallo en la comunicación con la API externa para generación de contraseña. ■ Problemas técnicos al generar o asignar las credenciales.

Tabla 4.29: Requisito funcional: RF-04 Validar y generar automáticamente las credenciales

RF-05	Crear alumno
Descripción	Permitir al profesor crear nuevos usuarios con rol de alumno en el sistema, proporcionando los datos necesarios.
Precondición	El profesor debe estar autenticado en el sistema.
Secuencia de pasos	1. El profesor introduce los datos requeridos del alumno (nombre, apellidos y correo). 2. El sistema valida los datos mediante RF-04. 3. El sistema guarda el nuevo alumno. 4. El sistema notifica al profesor que el alumno ha sido creado correctamente.
Postcondición	El nuevo alumno queda registrado en el sistema y puede iniciar sesión con las credenciales asignadas.
Excepciones	■ Datos incompletos o inválidos introducidos por el profesor. ■ Problemas de conexión con la base de datos.

Tabla 4.30: Requisito funcional: RF-05 Crear alumno

RF-06	Crear grupo
Descripción	Permitir al profesor crear nuevos grupos proporcionando la información necesaria (nombre del grupo y alumnos).
Precondición	El profesor debe estar autenticado en el sistema y tener usuarios creados.
Secuencia de pasos	1. El profesor accede a la sección de administración. 2. El profesor introduce los datos requeridos para el grupo. 3. El sistema valida la información introducida. 4. El sistema guarda el nuevo grupo. 5. El sistema crea las ramas de la asignatura vacías (sin ningún material aún) y las guarda. 6. El sistema notifica al profesor que el grupo ha sido creado correctamente.
Postcondición	El grupo queda registrado en el sistema asociado al profesor y los alumnos incluidos.
Excepciones	■ Datos inválidos introducidos por el profesor. ■ Problemas de conexión con la base de datos.

Tabla 4.31: Requisito funcional: RF-06 Crear grupo

RF-07	Revisar grupos
Descripción	Permitir al profesor visualizar la lista de grupos que ha creado previamente.
Precondición	El profesor debe estar autenticado en el sistema y tener grupos creados.
Secuencia de pasos	1. El profesor accede a la sección de grupos. 2. El sistema recupera y muestra la lista de grupos asociados al profesor. 3. El profesor revisa los grupos mostrados.
Postcondición	El profesor puede ver y seleccionar cualquier grupo de su lista para gestionar.
Excepciones	■ No existen grupos creados por el profesor. ■ Problemas de conexión con la base de datos.

Tabla 4.32: Requisito funcional: RF-07 Listar grupos

RF-08	Seleccionar grupo
Descripción	Permitir al profesor seleccionar un grupo creado para realizar acciones específicas o trabajar con su contenido.
Precondición	El profesor debe estar autenticado y haber creado al menos un grupo.
Secuencia de pasos	1. El profesor visualiza la lista de grupos creados mediante RF-07. 2. El profesor selecciona un grupo de la lista. 3. El sistema carga los datos y funcionalidades correspondientes al grupo seleccionado. 4. El profesor puede comenzar a trabajar con el grupo.
Postcondición	El sistema queda configurado para trabajar con el grupo seleccionado.
Excepciones	■ El grupo seleccionado no existe. ■ Problemas de carga o acceso a los datos del grupo.

Tabla 4.33: Requisito funcional: RF-08 Seleccionar grupo

RF-09	Enviar mensaje
Descripción	Permitir al profesor enviar mensajes a los alumnos de un grupo seleccionado.
Precondición	El profesor debe estar autenticado y tener un grupo seleccionado.
Secuencia de pasos	1. El profesor accede a administración o a perfil y entra en la herramienta de creación de mensajes. 2. El profesor redacta el mensaje, el asunto y selecciona los alumnos del grupo seleccionado como destinatarios. 3. El profesor envía el mensaje. 4. El sistema valida los datos. 5. El sistema guarda el mensaje y confirma el envío exitoso del mensaje al profesor.
Postcondición	Los alumnos del grupo y el profesor reciben el mensaje en su bandeja de entrada dentro de la plataforma.
Excepciones	■ No se han seleccionado alumnos y/o el mensaje y/o el asunto no son válidos. ■ Problemas de conexión a la base de datos.

Tabla 4.34: Requisito funcional: RF-09 Enviar mensaje

RF-10	Ver libro del curso
Descripción	Permitir al profesor y al alumno visualizar el libro asociado a una rama específica de la asignatura.
Precondición	El profesor o el alumno deben estar autenticados, tener un grupo elegido y existir un libro asociado a la rama seleccionada.
Secuencia de pasos	1. El usuario selecciona la rama de la asignatura. 2. El sistema muestra el libro actual asociado a dicha rama. 3. El usuario revisa el contenido del libro.
Postcondición	El usuario visualiza correctamente el libro asociado a la rama seleccionada pudiendo revisarlo desde el propio sistema o descargarlo.
Excepciones	■ No existe libro asociado a la rama seleccionada. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.35: Requisito funcional: RF-10 Ver libro del curso

RF-11	Subir o reemplazar libro del curso
Descripción	Permitir al profesor subir un nuevo libro o reemplazar el libro existente asociado a una rama de la asignatura.
Precondición	El profesor debe estar autenticado y tener un grupo seleccionado.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El sistema muestra el libro actual asociado, si existe. 3. El profesor selecciona un archivo local para subir o reemplazar el libro. 4. El sistema valida el formato y tamaño del archivo. 5. El sistema actualiza el libro asociado a la rama con el nuevo archivo. 6. El sistema informa de la actualización exitosa.
Postcondición	El libro asociado a la rama queda actualizado con el nuevo archivo.
Excepciones	■ Archivo no válido (formato o tamaño). ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.36: Requisito funcional: RF-11 Subir o reemplazar libro del curso

RF-12	Eliminar libro del curso
Descripción	Permitir al profesor eliminar el libro asociado a una rama de la asignatura.
Precondición	El profesor debe estar autenticado, tener un grupo seleccionado y existir un libro asociado a la rama seleccionada.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El sistema muestra el libro actual asociado a la rama. 3. El profesor solicita eliminar el libro. 4. El sistema solicita confirmación de la acción. 5. El profesor confirma la eliminación. 6. El sistema elimina el libro asociado a la rama. 7. El sistema informa de la eliminación exitosa.
Postcondición	El libro asociado a la rama es eliminado y ya no está disponible para consulta.
Excepciones	■ No se ha elegido ningún grupo. ■ No existe libro para eliminar en la rama seleccionada. ■ Problemas de conexión a la base de datos.

Tabla 4.37: Requisito funcional: RF-12 Eliminar libro del curso

RF-13	Revisar tareas
Descripción	Permitir al profesor y al alumno revisar la lista de tareas creadas para una rama específica de la asignatura.
Precondición	El profesor o el alumno deben estar autenticados, tener un grupo seleccionado y existir al menos una tarea creada en la rama seleccionada.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El sistema muestra la lista de tareas disponibles. 3. El profesor puede seleccionar una tarea para ver sus detalles.
Postcondición	El profesor visualiza correctamente la información de las tareas existentes.
Excepciones	■ No se ha elegido ningún grupo. ■ No existen tareas en la rama seleccionada. ■ Problemas de conexión a la base de datos.

Tabla 4.38: Requisito funcional: RF-13 Revisar tareas

RF-14	Crear tarea
Descripción	Permitir al profesor crear nuevas tareas para una rama específica de la asignatura.
Precondición	El profesor debe estar autenticado y tener un grupo seleccionado.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor accede al formulario para crear una tarea. 3. El profesor introduce los detalles de la tarea (título, descripción, fecha límite, alumnos dentro del grupo a los que se dirige la tarea y un archivo de apoyo opcional). 4. El profesor confirma la creación de la tarea. 5. El sistema valida los datos introducidos, el archivo de apoyo si lo hubiera (formato correcto) y que haya alumnos seleccionados. 6. El sistema guarda la tarea, envía un mensaje del sistema a los alumnos e informa al profesor de la creación exitosa.
Postcondición	La nueva tarea queda registrada y disponible en la lista de tareas de la rama y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ No se ha elegido ningún grupo. ■ Datos incompletos o inválidos en el formulario. ■ Problemas de conexión a la base de datos.

Tabla 4.39: Requisito funcional: RF-14 Crear tarea

RF-15	Modificar tarea
Descripción	Permitir al profesor modificar las tareas existentes en una rama específica de la asignatura.
Precondición	El profesor debe estar autenticado, tener un grupo seleccionado y existir la tarea que desea modificar.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona la tarea a modificar. 3. El sistema muestra los datos actuales de la tarea. 4. El profesor edita los detalles necesarios. 5. El profesor confirma la modificación. 6. El sistema valida los datos introducidos, el archivo de apoyo si lo hubiera (formato correcto) y que haya alumnos seleccionados. 7. El sistema actualiza la tarea, envía un mensaje del sistema a los alumnos e informa al profesor de la modificación exitosa.
Postcondición	La tarea queda actualizada con los nuevos datos proporcionados y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ No se ha elegido ningún grupo. ■ Datos incompletos o inválidos en el formulario. ■ Problemas de conexión a la base de datos.

Tabla 4.40: Requisito funcional: RF-15 Modificar tarea

RF-16	Cerrar tarea
Descripción	Permitir al profesor cerrar una tarea, impidiendo que los alumnos sigan entregándola.
Precondición	El profesor debe estar autenticado, tener un grupo seleccionado y la tarea debe existir y estar activa (fecha de cierre no cumplida o ya cerrada antes).
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona la tarea que desea cerrar. 3. El sistema muestra la información de la tarea. 4. El profesor solicita cerrar la tarea. 5. El sistema cierra la tarea e informa de la acción exitosa.
Postcondición	La tarea queda cerrada y no acepta nuevas entregas de los alumnos.
Excepciones	■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.41: Requisito funcional: RF-16 Cerrar tarea

RF-17	Eliminar tarea
Descripción	Permitir al profesor eliminar tareas existentes de una rama de la asignatura.
Precondición	El profesor debe estar autenticado tener un grupo seleccionado y la tarea debe existir.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona la tarea que desea eliminar. 3. El sistema solicita confirmación para eliminar la tarea. 4. El profesor confirma la eliminación. 5. El sistema elimina la tarea e informa de la acción exitosa.
Postcondición	La tarea queda eliminada y ya no está disponible para los usuarios.
Excepciones	■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.42: Requisito funcional: RF-17 Eliminar tarea

RF-18	Revisar cuestionarios
Descripción	Permitir al profesor y al alumno revisar la lista de cuestionarios de la rama de teoría de la asignatura.
Precondición	El profesor o el alumno deben de estar autenticados, tener un grupo seleccionado y existir al menos un cuestionario creado en la rama de teoría.
Secuencia de pasos	1. El usuario selecciona la rama de teoría de la asignatura. 2. El usuario accede a la sección de cuestionarios. 3. El sistema muestra la lista de cuestionarios disponibles. 4. El profesor puede seleccionar un cuestionario para ver sus detalles.
Postcondición	El profesor visualiza correctamente la información de los cuestionarios existentes.
Excepciones	■ No existen cuestionarios en la rama seleccionada. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.43: Requisito funcional: RF-18 Revisar cuestionarios

RF-19	Crear cuestionario
Descripción	Permitir al profesor crear nuevos cuestionarios para la rama de teoría de la asignatura.
Precondición	El profesor debe estar autenticado y tener un grupo seleccionado.
Secuencia de pasos	1. El profesor selecciona la rama de teoría de la asignatura. 2. El usuario accede a la sección de cuestionarios. 3. El profesor accede al formulario para crear un cuestionario. 4. El profesor introduce los detalles del cuestionario (título, alumnos, fecha de cierre y preguntas tipo test, a su vez, cada pregunta podrá tener un material de apoyo opcional). 5. El profesor confirma la creación del cuestionario. 6. El sistema valida los datos introducidos, las preguntas y sus correspondientes materiales de apoyo (formato), si lo hubiese. 7. El sistema guarda el cuestionario, envía un mensaje del sistema a los alumnos e informa al profesor de la creación exitosa.
Postcondición	El nuevo cuestionario queda registrado y disponible en la lista de cuestionarios de la rama y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ Datos incompletos o inválidos en el formulario. ■ Problemas de conexión a la base de datos.

Tabla 4.44: Requisito funcional: RF-19 Crear cuestionario

RF-20	Modificar cuestionario
Descripción	Permitir al profesor modificar los cuestionarios existentes en la rama de teoría de la asignatura.
Precondición	El profesor debe estar autenticado, debe de haber un grupo seleccionado y existir el cuestionario que se desea modificar.
Secuencia de pasos	1. El profesor selecciona la rama de teoría de la asignatura. 2. El usuario accede a la sección de cuestionarios. 3. El profesor selecciona el cuestionario a modificar. 4. El sistema muestra los datos actuales del cuestionario. 5. El profesor edita los detalles necesarios. 6. El profesor confirma la modificación. 7. El sistema valida los datos introducidos, las preguntas y sus correspondientes materiales de apoyo (formato), si lo hubiese. 8. El sistema actualiza el cuestionario, envía un mensaje del sistema a los alumnos e informa al profesor de la modificación exitosa.
Postcondición	El cuestionario queda actualizado con los nuevos datos proporcionados y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ No se ha elegido ningún grupo. ■ Datos inválidos o incompletos durante la modificación. ■ Problemas de conexión a la base de datos.

Tabla 4.45: Requisito funcional: RF-20 Modificar cuestionario

RF-21	Cerrar cuestionario
Descripción	Permitir al profesor cerrar un cuestionario, impidiendo que los alumnos sigan respondiéndolo.
Precondición	El profesor debe estar autenticado, debe de haber un grupo seleccionado y el cuestionario debe existir y estar activo.
Secuencia de pasos	1. El profesor selecciona la rama de teoría de la asignatura. 2. El profesor selecciona el cuestionario que desea cerrar. 3. El sistema solicita confirmación para cerrar el cuestionario. 4. El profesor confirma el cierre. 5. El sistema cierra el cuestionario e informa de la acción exitosa.
Postcondición	El cuestionario queda cerrado y no acepta nuevas respuestas de los alumnos.
Excepciones	■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.46: Requisito funcional: RF-21 Cerrar cuestionario

RF-22	Eliminar cuestionario
Descripción	Permitir al profesor eliminar cuestionarios existentes en la rama de teoría de la asignatura.
Precondición	El profesor debe estar autenticado, debe de haber un grupo seleccionado y el cuestionario debe existir.
Secuencia de pasos	1. El profesor selecciona la rama de teoría de la asignatura. 2. El profesor selecciona el cuestionario que desea eliminar. 3. El sistema solicita confirmación para eliminar el cuestionario. 4. El profesor confirma la eliminación. 5. El sistema elimina el cuestionario y confirma la acción exitosa.
Postcondición	El cuestionario queda eliminado y ya no está disponible para los usuarios.
Excepciones	■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.47: Requisito funcional: RF-22 Eliminar cuestionario

RF-23	Revisar entregas
Descripción	Permitir al profesor revisar las entregas realizadas por los alumnos para una tarea específica.
Precondición	El profesor debe estar autenticado, tener un grupo seleccionado, existir una tarea creada y haber al menos una entrega realizada por los alumnos.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona una tarea existente. 3. El sistema muestra la lista de entregas realizadas por los alumnos. 4. El profesor selecciona una entrega para visualizar su contenido.
Postcondición	El profesor visualiza correctamente las entregas asociadas a la tarea seleccionada.
Excepciones	■ No existen entregas para la tarea seleccionada. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.48: Requisito funcional: RF-23 Revisar entregas

RF-24	Calificar entrega
Descripción	Permitir al profesor calificar una entrega realizada por un alumno para una tarea específica.
Precondición	El profesor debe estar autenticado, debe de haber un grupo seleccionado y existir una entrega asociada a la tarea seleccionada.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona una tarea. 3. El profesor selecciona una entrega concreta. 4. El sistema muestra el contenido de la entrega. 5. El profesor introduce la calificación. 6. El profesor, si lo desea, introduce una observación. 7. El profesor confirma la calificación. 8. El sistema valida la calificación introducida (Número real del 1 al 10). 9. El sistema guarda la calificación asignada, envía un mensaje del sistema a los alumnos e informa al profesor del éxito en calificar.
Postcondición	La entrega queda calificada, la calificación se almacena en el sistema y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ No existen entregas para la tarea seleccionada. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.49: Requisito funcional: RF-24 Calificar entrega

RF-25	Revisar calificaciones de entregas
Descripción	Permitir al profesor revisar las calificaciones asignadas a las entregas de una tarea específica.
Precondición	El profesor debe estar autenticado, debe de haber un grupo seleccionado y existir una entrega asociada a la tarea seleccionada.
Secuencia de pasos	1. El profesor selecciona la rama de la asignatura. 2. El profesor selecciona una tarea. 3. El sistema muestra la lista de entregas calificadas junto con sus calificaciones.
Postcondición	El profesor visualiza correctamente las calificaciones asociadas a las entregas.
Excepciones	■ No existen entregas calificadas para la tarea seleccionada. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.50: Requisito funcional: RF-25 Revisar calificaciones de entregas

RF-26	Revisar calificaciones generales del alumno
Descripción	Permitir al profesor o al alumno revisar las calificaciones generales obtenidas previamente por un alumno en el curso.
Precondición	El profesor o el alumno deben de estar autenticados, tener un grupo seleccionado y que dicho alumno tenga calificaciones generales registradas en el sistema.
Secuencia de pasos	1. El usuario accede a perfil o a administración (esta última solo si es profesor). 2. Si el usuario es alumno, el sistema identifica automáticamente al alumno autenticado. 3. Si el usuario es profesor, el sistema requiere de la selección del alumno desde la herramienta de calificaciones generales. 4. El sistema muestra las calificaciones generales registradas para el alumno. 5. El usuario revisa el historial de calificaciones.
Postcondición	El usuario visualiza correctamente las calificaciones generales del alumno.
Excepciones	■ El alumno no tiene calificaciones registradas. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.51: Requisito funcional: RF-26 Revisar calificaciones generales del alumno

RF-27	Asignar calificación general al alumno
Descripción	Permitir al profesor asignar o actualizar la calificación general de un alumno en el curso.
Precondición	El profesor debe estar autenticado y tener un grupo seleccionado.
Secuencia de pasos	1. El profesor selecciona el perfil o administración. 2. El profesor accede a la herramienta de calificaciones generales. 3. El profesor selecciona un alumno del listado. 4. El sistema muestra las calificaciones generales actuales, si existen. 5. El profesor introduce o modifica la calificación general (que podrá ser Q1, Q2, Q3, Ordinaria o extraordinaria, esta última, solo se activará en caso de tener una calificación inferior a 5 en la ordinaria). 6. El profesor confirma la asignación de la calificación. 7. El sistema valida la calificación introducida (Número entero del 1 al 10). 8. El sistema guarda la calificación general del alumno, envía un mensaje del sistema a los alumnos e informa al profesor de la operación exitosa.
Postcondición	La calificación general del alumno queda registrada o actualizada en el sistema y el alumno queda avisado de ello en los mensajes (RFT-1).
Excepciones	■ La calificación introducida no es válida. ■ No se ha elegido ningún grupo. ■ Problemas de conexión a la base de datos.

Tabla 4.52: Requisito funcional: RF-27 Asignar calificación general al alumno



RF-28	Ver mensajes
Descripción	Permitir al alumno y al profesor visualizar los mensajes recibidos en la plataforma y marcarlos como leídos si así lo desea el usuario.
Precondición	El profesor o alumno deben de estar autenticado y tener al menos un mensaje recibido.
Secuencia de pasos	1. El usuario accede a la sección de mensajes. 2. El sistema muestra la lista de mensajes recibidos. 3. De forma opcional y si el mensaje no estaba ya leído, el usuario marca el mensaje como leído. 4. El sistema actualiza el estado del mensaje.
Postcondición	El usuario puede ver sus mensajes y marcados como leídos en caso de que el usuario lo haya indicado.
Excepciones	■ No existen mensajes para el usuario. ■ Problemas de conexión a la base de datos.

Tabla 4.53: Requisito funcional: RF-28 Ver mensajes

RF-29	
Entregar tarea	
Descripción	Permitir al alumno realizar la entrega de una tarea asignada por el profesor.
Precondición	El alumno debe estar autenticado, tener un grupo seleccionado, existir una tarea activa y encontrarse dentro del plazo de entrega.
Secuencia de pasos	1. El alumno selecciona la rama de la asignatura. 2. El alumno selecciona una tarea disponible. 3. El sistema muestra los detalles de la tarea. 4. El alumno adjunta el comentario y/o archivo requerido. 5. El alumno confirma la entrega de la tarea. 6. El sistema valida que el comentario sea válido y que el archivo adjunto (si lo hubiera) sea válido. 7. El sistema registra la entrega realizada, envía un mensaje del sistema al profesor e informa al alumno de que la entrega fue exitosa.
Postcondición	La tarea queda entregada y disponible para su posterior revisión y calificación por parte del profesor y el alumno profesor avisado de ello en los mensajes (RFT-1).
Excepciones	■ No se ha elegido ningún grupo. ■ El archivo o el comentario adjunto/s no es válido. ■ Problemas de conexión a la base de datos.

Tabla 4.54: Requisito funcional: RF-29 Entregar tarea

RF-30	Rellenar y entregar cuestionario
Descripción	Permitir al alumno rellenar un cuestionario y entregarlo para su autocorrección inmediata por el sistema.
Precondición	El alumno debe estar autenticado, tener un grupo seleccionado, existir un cuestionario activo y encontrarse dentro del plazo de entrega.
Secuencia de pasos	1. El alumno selecciona la rama de teoría de la asignatura. 2. El alumno selecciona un cuestionario disponible. 3. El sistema muestra las preguntas del cuestionario. 4. El alumno responde a las preguntas. 5. El alumno confirma la entrega del cuestionario. 6. El sistema valida que todas las preguntas han sido respondidas. 7. El sistema realiza la autocorrección del cuestionario. 8. El sistema registra la entrega y la calificación obtenida.
Postcondición	El cuestionario queda entregado y autocorregido, quedando registrada la calificación obtenida por el alumno.
Excepciones	■ No se ha elegido ningún grupo. ■ No se han respondido todas las preguntas y/o las respuestas no son válidas. ■ Problemas de conexión a la base de datos.

Tabla 4.55: Requisito funcional: RF-30 Rellenar y entregar cuestionario

RF-31	Cambiar contraseña
Descripción	Permitir al alumno y al profesor cambiar su contraseña.
Precondición	El usuario debe de estar autenticado en el sistema y acceder a su perfil.
Secuencia de pasos	1. El usuario accede a la opción "Cambiar contraseña" desde su perfil. 2. El usuario introduce su contraseña actual. 3. El usuario introduce su nueva contraseña. 4. El sistema verifica que el usuario existe en la base de datos. 5. El sistema actualiza la contraseña. 6. El sistema notifica al usuario del éxito.
Postcondición	El usuario puede iniciar sesión con la nueva contraseña establecida.
Excepciones	■ La contraseña actual es incorrecta. ■ La nueva contraseña no cumple con los criterios de seguridad establecidos. ■ El sistema no puede conectarse a la base de datos o servicio de correo.

Tabla 4.56: Requisito funcional: RF-31 Cambiar contraseña

RF-32	Configurar tempo del metrónomo
Descripción	Permitir al usuario establecer la velocidad de reproducción del metrónomo mediante la selección de un valor de pulsaciones por minuto (BPM).
Precondición	El alumno debe estar autenticado, tener un grupo seleccionado y encontrarse en la rama de ritmo.
Secuencia de pasos	1. El usuario accede a la rama de ritmo. 2. El usuario modifica el valor de BPM mediante los controles disponibles. 3. El sistema actualiza la velocidad del metrónomo con el nuevo tempo seleccionado.
Postcondición	El metrónomo queda configurado con el tempo seleccionado por el usuario.
Excepciones	■ Fallo del sistema.

Tabla 4.57: Requisito funcional: Configurar tempo del metrónomo

RF-33	Iniciar y detener metrónomo
Descripción	Permitir al usuario iniciar y detener la reproducción del metrónomo para marcar un pulso constante según el tempo configurado.
Precondición	El alumno debe estar autenticado, tener un grupo seleccionado y encontrarse en la rama de ritmo.
Secuencia de pasos	1. El usuario accede a la rama de ritmo. 2. El usuario pulsa el botón de inicio. 3. El sistema comienza la reproducción de los pulsos siguiendo el tempo configurado. 4. El usuario pulsa el botón de detener. 5. El sistema finaliza la reproducción del metrónomo.
Postcondición	El metrónomo queda detenido y deja de emitir pulsos.
Excepciones	■ Fallo del sistema.

Tabla 4.58: Requisito funcional: Iniciar y detener metrónomo

RF-34	Utilizar piano de referencia
Descripción	Permitir al alumno reproducir las notas disponibles en el piano virtual para obtener referencias auditivas durante la realización de ejercicios musicales.
Precondición	El alumno debe estar autenticado, tener un grupo seleccionado y encontrarse en la rama de audición.
Secuencia de pasos	1. El usuario accede a la rama de audición. 2. El usuario selecciona una tecla del piano virtual. 3. El sistema reproduce el sonido correspondiente a la nota seleccionada. 4. El usuario puede repetir el proceso con cualquier otra tecla disponible.
Postcondición	La nota seleccionada ha sido reproducida correctamente y el usuario ha obtenido la referencia auditiva deseada.
Excepciones	■ Fallo del sistema.

Tabla 4.59: Requisito funcional: Utilizar piano de referencia

Por último, es importante añadir un requisito transversal que se dispara en varios requisitos funcionales del sistema.

RFT-1	Enviar notificaciones del sistema
Descripción	El sistema enviará notificaciones automáticas a los usuarios cuando se produzcan determinadas acciones relevantes dentro de la plataforma.
Precondición	Debe producirse una acción del sistema que tenga asociada una notificación automática.
Secuencia de pasos	1. El sistema detecta la ejecución de una acción relevante. 2. El sistema identifica a los usuarios destinatarios de la notificación. 3. El sistema genera el contenido de la notificación. 4. El sistema envía la notificación al usuario correspondiente.
Postcondición	El usuario destinatario recibe una notificación asociada a la acción realizada.
Excepciones	■ No se puede identificar al usuario destinatario. ■ La acción anterior falla. ■ Problemas de conexión a la base de datos.

Tabla 4.60: Requisito funcional transversal: RFT-1 Enviar notificaciones del sistema

4.7 REQUISITOS NO FUNCIONALES

Tras definir todas las funcionalidades del sistema, así como los requisitos que las conforman (primero, saber qué debe hacer el sistema), las investigaciones y análisis previos han llevado a definir los siguientes requisitos no funcionales que debe cumplir la solución (después, saber cómo lo debe hacer):

RNF-01	Escalabilidad y mantenibilidad
Descripción	El sistema deberá diseñarse de forma que permita su evolución y mantenimiento a lo largo del tiempo, facilitando la incorporación de nuevas funcionalidades, la modificación de las existentes y la corrección de errores sin afectar negativamente al conjunto de la aplicación.
Justificación	Proyectos de esta índole siempre van a dejar aspectos pendientes de considerar, por lo tanto, tanto en su desarrollo como en posibles fases posteriores deberá poder ser aprovechada para proyectos mayores.
Criterio de cumplimiento	El sistema presenta una arquitectura modular, con separación clara de responsabilidades y una organización del código que facilita su extensión y mantenimiento.

Tabla 4.61: Requisito no funcional: RNF-01 Escalabilidad y mantenibilidad

RNF-02	Usabilidad y accesibilidad
Descripción	El sistema deberá ser usable y accesible, permitiendo a profesores y alumnos interactuar con la aplicación de forma intuitiva, independientemente de su nivel de competencia digital, y teniendo en cuenta criterios básicos de accesibilidad.
Justificación	Los usuarios finales no tienen por qué poseer un alto nivel de conocimientos informáticos y pueden existir alumnos con dificultades cognitivas o visuales, como el daltonismo, por lo que es necesario facilitar el acceso y comprensión de la interfaz.
Criterio de cumplimiento	La interfaz utiliza elementos visuales claros, navegación sencilla, contrastes adecuados y evita sobrecargar las pantallas con información innecesaria.

Tabla 4.62: Requisito no funcional: RNF-02 Usabilidad y accesibilidad

RNF-03	Diseño visual atractivo y motivador
Descripción	El sistema deberá contar con un diseño visual atractivo, coherente y consistente, que favorezca la motivación del alumnado y mejore la experiencia de usuario durante el proceso de aprendizaje.
Justificación	El estudio y las investigaciones previas realizadas indican que un diseño visual cuidado y llamativo incrementa la motivación del alumnado, especialmente en entornos educativos digitales.
Criterio de cumplimiento	La aplicación mantiene una estética uniforme, uso adecuado del color, iconografía clara y una disposición visual que facilita la comprensión del contenido.

Tabla 4.63: Requisito no funcional: RNF-03 Diseño visual atractivo y motivador

RNF-04	Portabilidad
Descripción	El sistema deberá ser portable, permitiendo su uso en distintos dispositivos y entornos, especialmente en navegadores web modernos y dispositivos móviles.
Justificación	La portabilidad favorece la escalabilidad del sistema y permite su uso en distintos contextos educativos.
Criterio de cumplimiento	La aplicación se desarrolla utilizando tecnologías multiplataforma y es accesible desde distintos dispositivos sin necesidad de configuraciones específicas.

Tabla 4.64: Requisito no funcional: RNF-04 Portabilidad

RNF-05	Seguridad
Descripción	El sistema deberá garantizar la seguridad de los datos personales y académicos de los usuarios, controlando el acceso a la información y protegiendo la integridad de los datos almacenados.
Justificación	La aplicación está orientada a entornos educativos públicos y privados, como conservatorios y escuelas de música, que requieren un tratamiento seguro de la información del alumnado y profesorado.
Criterio de cumplimiento	El sistema implementa mecanismos de autenticación, control de roles y restricciones de acceso a las funcionalidades y datos según el tipo de usuario.

Tabla 4.65: Requisito no funcional: RNF-05 Seguridad

RNF-06	Consistencia pedagógica
Descripción	El sistema deberá mantener una coherencia pedagógica alineada con las necesidades de aprendizaje del alumnado de lenguaje musical, reflejada en la organización de contenidos, navegación y funcionalidades.
Justificación	La aplicación se estructura en ramas de aprendizaje que corresponden a las distintas áreas del lenguaje musical, separando claramente las funcionalidades educativas de las de gestión, lo que facilita un flujo de aprendizaje claro y comprensible. Asimismo, determinadas funcionalidades se limitan a aquellas ramas en las que tienen sentido desde el punto de vista académico. En concreto, la creación de cuestionarios se restringe a la rama de teoría, ya que en las ramas de entonación, ritmo y audición este tipo de evaluación no resulta adecuada pedagógicamente.
Criterio de cumplimiento	La navegación y estructura del sistema siguen una lógica pedagógica uniforme, con contenidos, tareas, cuestionarios y calificaciones organizados por ramas y grupos. Las funcionalidades disponibles en cada rama se corresponden con métodos de evaluación y aprendizaje adecuados a su naturaleza académica, evitando ofrecer opciones que no tengan sentido pedagógico.

Tabla 4.66: Requisito no funcional: RNF-06 Consistencia pedagógica

RNF-07	Adaptabilidad del contenido
Descripción	El sistema deberá permitir la adaptación del contenido educativo en función de las características de cada grupo, posibilitando que el profesor configure materiales, tareas y cuestionarios específicos según las necesidades del alumnado.
Justificación	Cada grupo puede presentar ritmos de aprendizaje y necesidades distintas, por lo que es necesario ofrecer flexibilidad al profesorado para ajustar el contenido educativo sin afectar al resto de grupos.
Criterio de cumplimiento	El sistema permite asociar contenidos y actividades de forma independiente a cada grupo, manteniendo una personalización coherente del aprendizaje.

Tabla 4.67: Requisito no funcional: RNF-07 Adaptabilidad del contenido

4.8 ARQUITECTURA DE LA SOLUCIÓN

4.8.1 Aspectos comunes de la implementación

Aunque el backend y el frontend desempeñan funciones claramente diferenciadas dentro del sistema, ambos se han desarrollado siguiendo una misma filosofía de diseño y un conjunto de principios comunes que garantizan la coherencia de la solución software. Desde el inicio del proyecto se buscó construir una arquitectura modular en la que cada aplicación asumiera únicamente las responsabilidades propias de su ámbito, manteniendo un bajo acoplamiento entre módulos y favoreciendo la reutilización de componentes.

Esta aproximación permite que ambos módulos evolucionen de forma independiente sin comprometer el funcionamiento global del sistema. Al mismo tiempo, facilita las tareas de mantenimiento, simplifica la incorporación de nuevas funcionalidades y mejora la escalabilidad de la aplicación, ya que cada parte puede modificarse o ampliarse sin necesidad de realizar cambios significativos en el resto del proyecto.

Aunque cada aplicación utiliza tecnologías diferentes, ambas comparten una organización interna basada en la separación de responsabilidades. Tanto el backend como el frontend estructuran el código en componentes especializados, agrupando funcionalidades relacionadas dentro de módulos independientes y evitando la dispersión de la lógica de la aplicación. Este criterio de organización favorece un desarrollo más ordenado y facilita la comprensión del código durante todo el ciclo de vida del proyecto.

Además, el desarrollo de ambos módulos se ha realizado bajo un mismo conjunto de objetivos de calidad, priorizando aspectos como la mantenibilidad, la reutilización de código, la facilidad para realizar pruebas automatizadas y la automatización de los procesos de construcción y despliegue. Como consecuencia, backend y frontend no deben entenderse como aplicaciones independientes, sino como dos elementos complementarios que colaboran continuamente para ofrecer al usuario una experiencia homogénea y transparente.

Principios de diseño comunes

Uno de los principales criterios seguidos durante el desarrollo ha sido la aplicación del principio de separación de responsabilidades. Tanto el backend como el frontend distribuyen su funcionalidad en distintos módulos, cada uno encargado de una tarea concreta dentro del sistema. Esta organización permite reducir el acoplamiento entre componentes, incrementar la cohesión interna y facilitar la reutilización de código.

En ambos casos se adopta una organización inspirada en el patrón Modelo-Vista-Controlador (MVC). Aunque la implementación concreta difiere debido a la naturaleza de cada aplicación, el objetivo perseguido es el mismo: separar claramente la representación de los datos, la lógica de procesamiento y los mecanismos encargados de la interacción con otros componentes del sistema. Esta separación permite desarrollar y mantener cada capa de forma prácticamente independiente.

La modularidad también se refleja en la utilización de componentes especializados que encapsulan funcionalidades concretas. El backend distribuye la lógica mediante modelos, controladores, rutas y middlewares, mientras que el frontend organiza la aplicación mediante páginas, componentes, servicios, modelos y guards. Aunque la estructura no es idéntica, ambas siguen una filosofía común basada en asignar una única responsabilidad a cada elemento del sistema.

Otro aspecto compartido es la reutilización de funcionalidades comunes. En ambos módulos existen componentes que son utilizados desde diferentes partes de la aplicación con el objetivo de evitar duplicación de código y simplificar futuras modificaciones. Este enfoque reduce la complejidad del proyecto y favorece la mantenibilidad a medida que aumenta el número de funcionalidades implementadas.

Finalmente, ambos módulos se han diseñado para facilitar su evolución futura. La utilización de interfaces bien definidas entre componentes, junto con una clara separación entre las distintas capas de la aplicación, permite incorporar nuevas funcionalidades o modificar las existentes con un impacto reducido sobre el resto del sistema.

Comunicación e integración entre módulos

El funcionamiento del sistema se basa en una arquitectura cliente-servidor donde el frontend actúa como cliente de la API REST implementada por el backend. Toda la comunicación entre ambos módulos se realiza mediante peticiones HTTP y respuestas en formato JSON, permitiendo mantener completamente desacopladas la lógica de negocio y la interfaz de usuario.

Este desacoplamiento aporta una gran flexibilidad al sistema, ya que el backend puede evolucionar internamente sin afectar al funcionamiento del frontend siempre que se mantenga la interfaz pública de la API. Del mismo modo, la aplicación cliente puede modificarse o incluso sustituirse por otra implementación capaz de consumir los mismos servicios.

Las operaciones realizadas por el usuario siguen un flujo de comunicación común independiente de la funcionalidad concreta ejecutada. En términos generales, cualquier acción iniciada desde la interfaz genera una petición hacia el backend, donde se realizan las validaciones necesarias, se ejecuta la lógica de negocio y se accede a la base de datos cuando resulta necesario. Finalmente, el resultado obte-



nido es devuelto nuevamente al frontend, encargado de actualizar la información mostrada al usuario y gestionar la interacción correspondiente.

La Figura 4.7 representa este flujo general de funcionamiento, mostrando la colaboración existente entre los distintos elementos que intervienen durante el procesamiento de una petición.

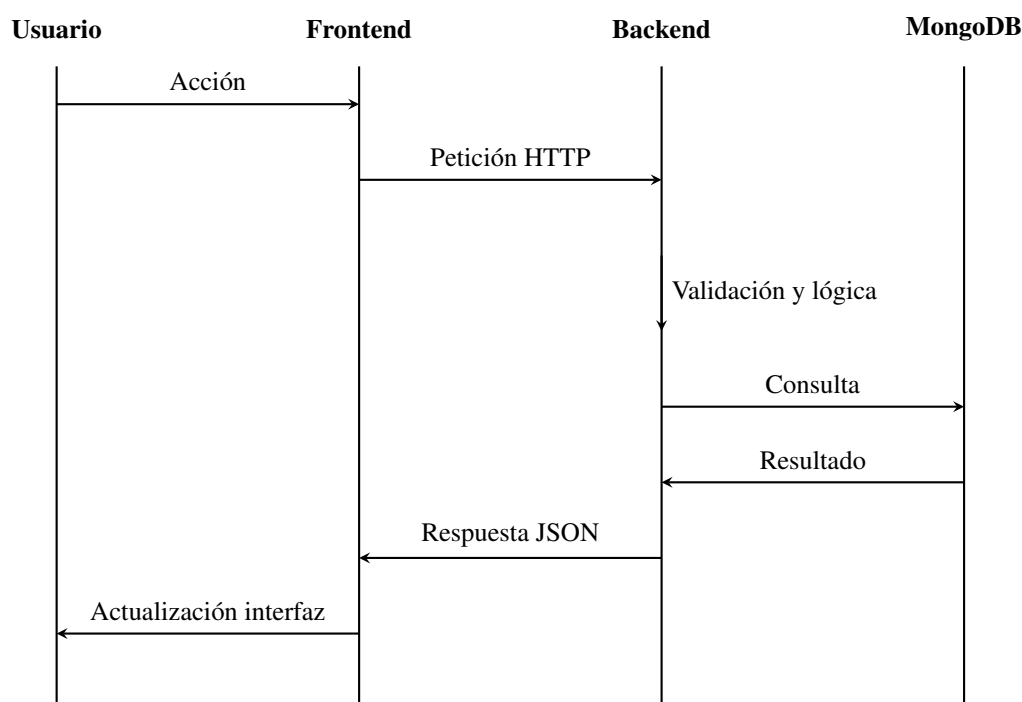


Figura 4.7: Flujo general de comunicación entre frontend y backend durante una operación del sistema.

Este flujo constituye la base sobre la que se implementan todas las funcionalidades de la aplicación. Independientemente de que se trate del inicio de sesión, la gestión de calificaciones, el intercambio de mensajes o la administración de grupos, todas las operaciones siguen el mismo esquema de comunicación, garantizando un comportamiento homogéneo y facilitando tanto el desarrollo como el mantenimiento del sistema.

La utilización de una interfaz de comunicación única también simplifica el proceso de pruebas y depuración. Al encontrarse claramente definidas las responsabilidades de cada módulo, resulta posible validar de forma independiente el correcto funcionamiento del backend y del frontend antes de verificar la integración completa del sistema. Asimismo, este diseño facilita la incorporación de nuevos clientes capaces de consumir la API existente sin necesidad de modificar la lógica de negocio implementada en el servidor.

Seguridad y gestión de la autenticación

Uno de los aspectos más importantes compartidos por ambos módulos es la gestión de la seguridad. La aplicación implementa un mecanismo de autenticación basado en JSON Web Tokens (JWT), que permite mantener sesiones sin estado (*stateless*) y proteger el acceso a los recursos restringidos de forma eficiente.

La autenticación constituye un claro ejemplo de colaboración entre el backend y el frontend, ya que ambos módulos participan activamente en el proceso. El backend es responsable de validar las credenciales del usuario, generar el token de acceso y verificar su validez en cada petición recibida. Por su parte, el frontend gestiona el almacenamiento del token, controla el estado de autenticación del usuario y lo incorpora automáticamente en las solicitudes dirigidas a la API.

Gracias a esta distribución de responsabilidades, el servidor mantiene el control sobre la seguridad de la aplicación mientras que el cliente proporciona una experiencia de uso transparente, evitando que el usuario tenga que autenticarse continuamente durante una misma sesión.

El funcionamiento conjunto de este mecanismo puede resumirse mediante el próximo pseudocódigo

Proceso general de autenticación

1. El usuario introduce sus credenciales.
2. El frontend:
 - Validar el formulario.
 - Enviar usuario y contraseña al backend.
3. El backend:
 - Buscar el usuario.
 - Comprobar las credenciales.
 - Generar un token JWT.
 - Devolver el token al cliente.
4. El frontend:
 - Almacenar el token.
 - Actualizar el estado de autenticación.
5. Para cada petición protegida:
 - Recuperar el token almacenado.
 - Incorporarlo a la cabecera HTTP.
6. El backend:
 - Verificar el token recibido.
 - Autorizar o denegar el acceso.
7. Si la operación es válida:
 - Ejecutar la lógica solicitada.
 - Devolver la respuesta al frontend.

Este proceso representa el funcionamiento general del mecanismo de autenticación utilizado en toda la aplicación. Los detalles específicos relacionados con la generación y validación del token en el backend, así como su gestión dentro del frontend, se desarrollan posteriormente en los apartados correspondientes a cada uno de los módulos.

Otro aspecto compartido es la validación de la información intercambiada entre ambos módulos. Aunque la comprobación definitiva de las reglas de negocio corresponde al backend, el frontend incorpora validaciones orientadas a mejorar la experiencia de usuario, evitando el envío de datos incorrectos y proporcionando mensajes de error inmediatos. Esta estrategia permite reducir el número de peticiones innecesarias y mejora la interacción del usuario con la aplicación sin comprometer la integridad de la información almacenada.

Calidad del software y despliegue

Además de compartir criterios arquitectónicos y mecanismos de comunicación, ambos módulos han sido desarrollados siguiendo una estrategia común orientada a garantizar la calidad del software durante todo su ciclo de vida.

En primer lugar, tanto el backend como el frontend incorporan pruebas automatizadas destinadas a verificar el correcto funcionamiento de sus componentes antes de su integración en el sistema. Aunque las herramientas empleadas difieren debido a las tecnologías utilizadas en cada aplicación, ambos módulos aplican una filosofía similar basada en la validación temprana de errores y en la comprobación sistemática de las funcionalidades implementadas. Esta estrategia facilita la detección de regresiones, incrementa la fiabilidad del software y simplifica el mantenimiento conforme aumenta el tamaño del proyecto.

De igual forma, durante el desarrollo se ha buscado mantener una estructura de código homogénea y fácilmente mantenible. La utilización de componentes especializados, la encapsulación de funcionalidades comunes y la clara separación entre las distintas capas de la aplicación favorecen la reutilización de código y reducen la complejidad asociada a futuras modificaciones.

Otro aspecto común es la automatización de los procesos de construcción y despliegue. Tanto el backend como el frontend utilizan mecanismos de integración y despliegue continuo (CI/CD) que permiten automatizar la generación de nuevas versiones, reduciendo la intervención manual y minimizando la posibilidad de introducir errores durante la publicación de la aplicación.

Aunque cada módulo emplea herramientas de despliegue diferentes, ambos siguen un flujo de trabajo similar basado en la construcción automática de la aplicación, la comprobación de su correcto funcionamiento y su posterior publicación en la plataforma correspondiente. Esta estrategia permite mantener sincronizadas ambas aplicaciones y facilita la distribución continua de nuevas funcionalidades y correcciones.

En conjunto, todos los aspectos descritos ponen de manifiesto que backend y frontend no constituyen desarrollos aislados, sino dos partes complementarias de una misma solución software. Ambos comparten criterios de diseño, mecanismos de comunicación, estrategias de seguridad y procedimientos de calidad que permiten ofrecer una aplicación coherente, robusta y preparada para evolucionar conforme aumenten las necesidades del proyecto.

Una vez presentados estos aspectos comunes, en los apartados siguientes se aborda de forma independiente el desarrollo del backend y del frontend, profundizando en las decisiones de diseño e implementación específicas adoptadas en cada uno de ellos.

4.8.2 Backend

Una vez descritos los aspectos comunes de la implementación del sistema, este apartado aborda el desarrollo específico del backend. Este módulo constituye el núcleo funcional de la aplicación y es el responsable de implementar la lógica de negocio, gestionar la persistencia de los datos y proporcionar los servicios consumidos por el frontend mediante la API REST.

A lo largo de esta sección se describen las principales decisiones de implementación adoptadas, la estructura interna del proyecto, el modelo de datos utilizado y los mecanismos específicos desarrollados para satisfacer los requisitos funcionales definidos durante el análisis del sistema.

Arquitectura y organización del backend

La aplicación backend se ha desarrollado utilizando **Node.js** como entorno de ejecución y **Express** como framework para la construcción de la API REST. Esta combinación proporciona un entorno ligero y eficiente para el desarrollo de aplicaciones web, facilitando la implementación de servicios independientes y la gestión de un elevado número de peticiones concurrentes.

La estructura del proyecto sigue las convenciones habituales del ecosistema Express, organizando el código en distintos directorios según la responsabilidad de cada elemento. Esta distribución permite mantener una clara separación entre las diferentes capas de la aplicación y facilita la incorporación de nuevas funcionalidades conforme evoluciona el proyecto.

Los principales elementos que componen la arquitectura del backend son los siguientes:

- **Modelos:** representan las entidades persistentes del sistema y definen tanto la estructura de los documentos almacenados como las validaciones asociadas a cada uno de ellos.
- **Controladores:** implementan la lógica específica de cada operación expuesta por la API, coordinando el acceso a los modelos y gestionando las respuestas enviadas al cliente.
- **Rutas:** definen los distintos endpoints disponibles y establecen la correspondencia entre cada petición HTTP y el controlador encargado de procesarla.
- **Middlewares:** encapsulan funcionalidades transversales utilizadas por múltiples rutas, como la autenticación de usuarios, la validación de permisos o la gestión centralizada de errores.

Esta organización facilita el mantenimiento del código y permite modificar o ampliar cada componente sin afectar significativamente al resto de la aplicación.

La persistencia de la información se realiza mediante **MongoDB**, una base de datos documental que se adapta adecuadamente a la naturaleza de los datos gestionados por el sistema. Su modelo flexible facilita la evolución de las estructuras almacenadas conforme aparecen nuevas necesidades funcionales y simplifica la representación de entidades con relaciones de distinta complejidad.

Además de la organización del código, el backend incorpora una estructura de servicios claramente orientada a la implementación de la lógica de negocio. Cada controlador concentra exclusivamente las operaciones relacionadas con un dominio funcional concreto, evitando la dispersión de responsabilidades y facilitando tanto la reutilización de código como la realización de pruebas unitarias sobre cada funcionalidad.

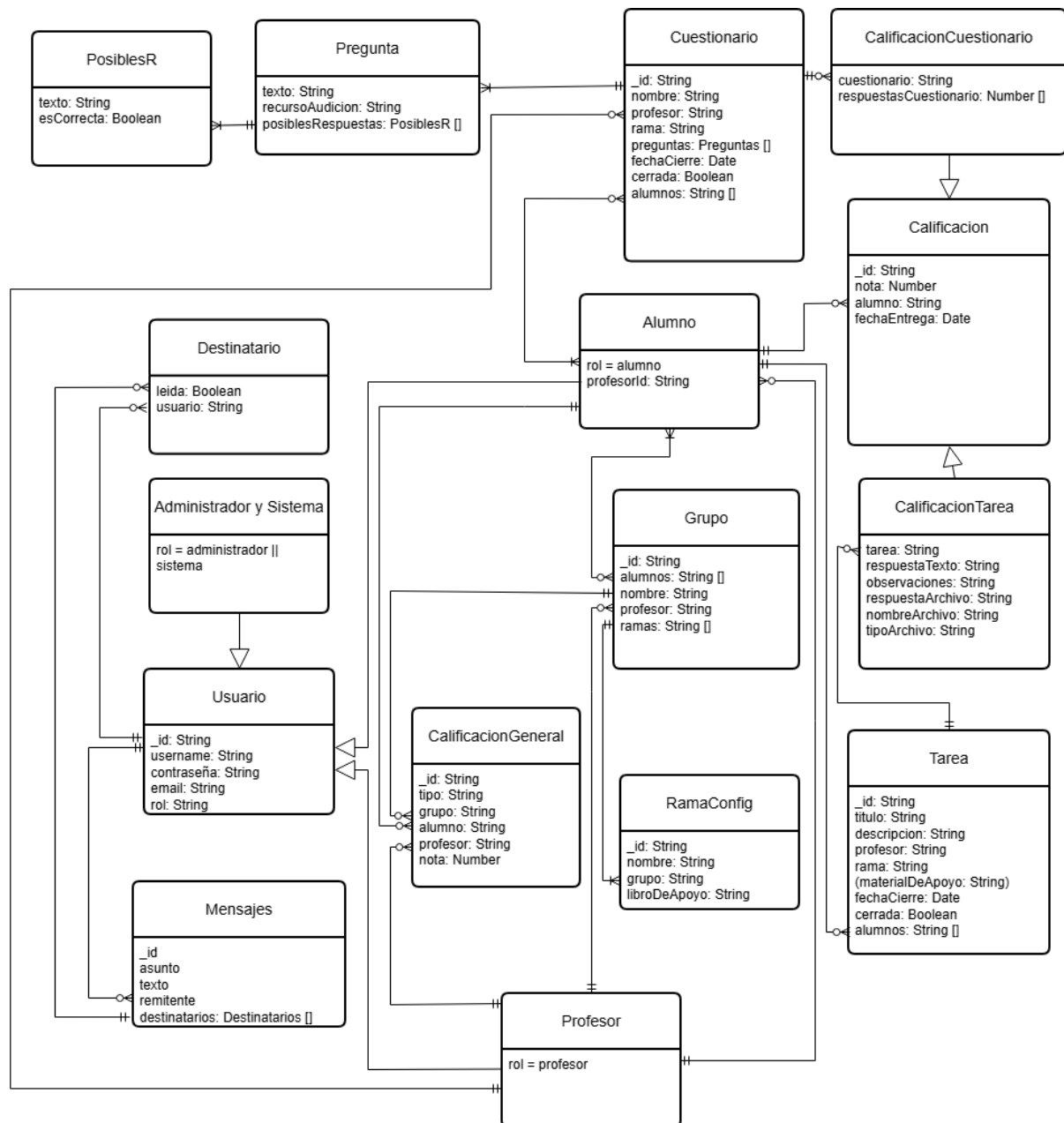
Modelado de datos

El diseño de la persistencia del sistema parte del modelo conceptual obtenido durante el análisis de requisitos, adaptándolo posteriormente al paradigma documental utilizado por MongoDB.

A diferencia de un sistema basado en bases de datos relacionales, donde las entidades y relaciones se representan mediante tablas y claves externas, en este proyecto se ha optado por una estructura documental que permite almacenar información relacionada dentro de un mismo documento cuando resulta conveniente y mantener referencias entre documentos cuando la naturaleza de los datos así lo requiere.

Esta aproximación proporciona una mayor flexibilidad durante la evolución del sistema, permitiendo modificar la estructura de determinadas entidades sin afectar al resto de colecciones ni requerir migraciones complejas del esquema de datos.

La Figura 4.8 muestra el modelo conceptual utilizado durante el diseño del sistema, donde se representan las principales entidades manejadas por la aplicación y las relaciones existentes entre ellas.



Profesor

esor

ual de datos del sistema

a los distintos perfiles de usuario, los grupos aca-
rios, las calificaciones y el sistema de mensajería.
e encargado de definir su estructura, restricciones
os.

a que posteriormente se implementan las opera-
ón de información desarrolladas por la lógica de

69

Profesor

esor

ual de datos del sistema

a los distintos perfiles de usuario, los grupos aca-
rios, las calificaciones y el sistema de mensajería.
e encargado de definir su estructura, restricciones
os.

a que posteriormente se implementan las opera-
ón de información desarrolladas por la lógica de

69

Profesor

esor

ual de datos del sistema

a los distintos perfiles de usuario, los grupos aca-
rios, las calificaciones y el sistema de mensajería.
e encargado de definir su estructura, restricciones
os.

a que posteriormente se implementan las opera-
ón de información desarrolladas por la lógica de

69

Implementación de la lógica de negocio

La principal responsabilidad del backend consiste en implementar las reglas de negocio que gobiernan el funcionamiento del sistema. Todas las operaciones expuestas mediante la API siguen un flujo de ejecución similar: recepción de la petición, validación de la información recibida, aplicación de las restricciones funcionales, acceso a la base de datos y generación de la respuesta correspondiente.

Aunque este patrón es común a todas las funcionalidades implementadas, cada controlador incorpora las comprobaciones específicas asociadas a su dominio funcional, garantizando el cumplimiento de las restricciones definidas durante la fase de análisis.

Como ejemplo representativo se presenta el proceso encargado de crear o actualizar las calificaciones generales de los alumnos. Esta operación ilustra la forma en que el backend combina las validaciones de entrada, la aplicación de reglas de negocio y la actualización de la información persistente.

El flujo general de esta funcionalidad puede resumirse mediante este pseudocódigo.

```
Crear o actualizar calificación general

1. Validar los datos de entrada:
   - Tipo de calificación.
   - Rango permitido de la nota.
   - Existencia del alumno, profesor y grupo.

2. Si la calificación es extraordinaria:
   - Comprobar que existe una convocatoria ordinaria.
   - Verificar que la nota ordinaria es inferior a 5.

3. Crear o actualizar la calificación.

4. Registrar la información en la base de datos.

5. Generar una notificación para el alumno.

6. Devolver el resultado de la operación.

7. Gestionar posibles errores de validación o ejecución.
```

Este ejemplo pone de manifiesto cómo el backend concentra toda la lógica asociada al dominio de la aplicación, garantizando que las operaciones realizadas respeten las restricciones funcionales definidas y mantengan la consistencia de la información almacenada. Del mismo modo, refleja la interacción existente entre los distintos componentes del backend, desde la recepción de la petición hasta la persistencia de los datos y la generación de la respuesta enviada al cliente.

Integración con servicios externos

Además de la implementación de la lógica de negocio propia del sistema, el backend integra diferentes servicios externos que permiten ampliar sus funcionalidades sin necesidad de desarrollar soluciones específicas para cada caso. Esta estrategia facilita el mantenimiento del proyecto y permite delegar determinadas tareas en plataformas especializadas.

En el sistema desarrollado, la principal integración externa corresponde al servicio de envío de correos electrónicos mediante la API de **Mailjet**. Esta funcionalidad se emplea durante el proceso de creación de nuevos usuarios para comunicar automáticamente las credenciales de acceso, evitando la necesidad de realizar esta tarea manualmente.

El funcionamiento general de esta integración puede resumirse mediante el siguiente pseudocódigo.

```
Enviar correo de credenciales

1. Construir el mensaje:
   - Remitente.
   - Destinatario.
   - Asunto.
   - Contenido.

2. Enviar la petición a Mailjet.

3. Si el envío es correcto:
   - Confirmar la operación.

4. Si ocurre un error:
   - Capturar la excepción.
   - Registrar el error.
   - Informar del fallo.
```

La utilización de un servicio externo especializado permite aumentar la fiabilidad del envío de correos electrónicos y simplifica considerablemente la implementación de esta funcionalidad dentro del backend.

Servicios de soporte

Además de las funcionalidades propias de la lógica de negocio, el backend incorpora una serie de servicios de soporte que facilitan el desarrollo, mantenimiento y operación del sistema. Entre ellos destacan los mecanismos de seguridad implementados para proteger la API y la documentación automática de los servicios ofrecidos.

Seguridad El mecanismo de autenticación empleado por el sistema se ha descrito previamente en el apartado dedicado a los aspectos comunes de la implementación, donde se presenta el flujo completo de autenticación y la colaboración existente entre el frontend y el backend.

Desde el punto de vista del servidor, la seguridad se implementa mediante componentes específicos encargados de gestionar la autenticación y autorización de las peticiones recibidas. Tras verificar las credenciales del usuario durante el proceso de inicio de sesión, el backend genera un token JWT que será utilizado posteriormente para acceder a los recursos protegidos de la aplicación.

Asimismo, las rutas privadas incorporan un middleware de autenticación encargado de validar la integridad y vigencia del token antes de permitir la ejecución del controlador correspondiente. Esta estrategia centraliza el control de acceso en un único punto de la aplicación, simplificando el mantenimiento y garantizando un comportamiento homogéneo para todos los servicios protegidos.

Documentación de la API Con el objetivo de facilitar el desarrollo, mantenimiento y validación del backend, la API REST se encuentra documentada mediante **Swagger**. Esta herramienta genera automáticamente una interfaz web interactiva a partir de la documentación asociada a cada endpoint, permitiendo consultar la información disponible y realizar pruebas directamente desde el navegador tal y como se puede observar en la Figura 4.9.

La documentación generada incluye información detallada sobre los recursos disponibles, los métodos HTTP soportados, los parámetros de entrada, los posibles códigos de respuesta y los modelos de datos utilizados por cada operación.

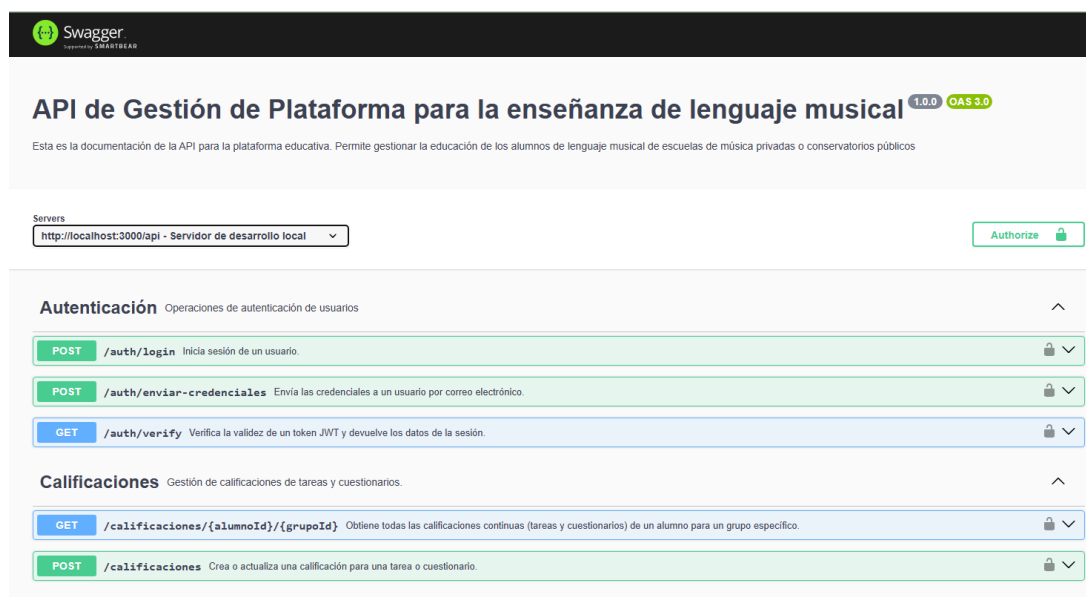


Figura 4.9: Interfaz de documentación de la API generada mediante Swagger.

Al mantenerse sincronizada con el código fuente, esta documentación constituye una referencia útil tanto durante el desarrollo como en las tareas de prueba y mantenimiento del sistema.

Testing

Con el objetivo de validar específicamente el comportamiento del backend, se ha desarrollado un conjunto de pruebas automatizadas destinadas a comprobar tanto la lógica de negocio como el correcto funcionamiento de los distintos servicios expuestos por la API.

Para ello se han empleado dos herramientas principales:

- **Jest**, utilizado para la implementación de pruebas unitarias sobre funciones y controladores.
- **Supertest**, empleado para realizar peticiones HTTP simuladas y verificar el comportamiento de los distintos endpoints de la API.

La combinación de ambas herramientas permite validar tanto el funcionamiento interno de los distintos componentes como su integración dentro del sistema.

Como ejemplo representativo se presenta el conjunto de pruebas desarrollado para el endpoint encargado de crear o actualizar las calificaciones generales de los alumnos.

Pruebas del endpoint POST /calificaciones-generales

1. Crear una calificación válida.
 - Verificar respuesta correcta.
2. Introducir una nota fuera del rango permitido.
 - Comprobar error de validación.
3. Crear una extraordinaria sin ordinaria previa.
 - Verificar rechazo de la operación.
4. Crear una extraordinaria con ordinaria ≥ 5 .
 - Verificar rechazo de la operación.

Estas pruebas permiten comprobar que las reglas de negocio implementadas por el backend se aplican correctamente y que la API responde de forma consistente tanto en situaciones normales como ante condiciones de error.

Despliegue

El despliegue del backend se realiza mediante un flujo automatizado de integración y despliegue continuo que garantiza la publicación de versiones estables y reduce la intervención manual durante el proceso.

La integración continua se implementa mediante **GitHub Actions**, responsable de ejecutar automáticamente el proceso de construcción del proyecto tras cada actualización del repositorio. Una vez completadas las verificaciones correspondientes, el despliegue se realiza mediante **Render**, plataforma que monitoriza el repositorio y publica automáticamente la versión más reciente de la aplicación.

Este proceso reduce el riesgo de errores durante el despliegue, facilita la distribución continua de nuevas versiones y proporciona una trazabilidad completa de las distintas fases del proceso de integración.

✓ fix backend test corregido #107

Summary

All jobs

✓ build-frontend
✓ **build-backend**
✓ deploy-frontend

Run details

Usage

Workflow file

build-backend

succeeded 4 minutes ago in 32s

> ✓ Set up job
> ✓ Checkout code
> ✓ Set up Node.js
> ✓ Install dependencies
> ✓ Run backend tests (jest)
> ✓ Post Set up Node.js
> ✓ Post Checkout code
> ✓ Complete job

```
45 Test Suites: 10 passed, 10 total
46 Tests:      137 passed, 137 total
47 Snapshots:  0 total
48 Time:       10.708 s
49 Ran all test suites.
```

(a) Proceso de construcción del backend en GitHub Actions.

(b) Ejecución de las pruebas automatizadas del backend.

Figura 4.10: Pipeline de integración y despliegue continuo del backend.

La automatización de este proceso permite mantener el backend permanentemente sincronizado con el repositorio del proyecto y facilita la incorporación de nuevas funcionalidades durante la evolución del sistema.

4.8.3 Frontend

Una vez descritos los aspectos comunes de la implementación del sistema y el desarrollo específico del backend, en este apartado se aborda la implementación del frontend. Este módulo constituye la interfaz de usuario de la aplicación y es el encargado de proporcionar los mecanismos necesarios para la interacción con el sistema, facilitando el acceso a las funcionalidades implementadas por el backend de una forma intuitiva, eficiente y adaptada a distintos dispositivos.

A lo largo de esta sección se describen la organización interna del proyecto, las principales decisiones de implementación adoptadas, la construcción de la interfaz de usuario y las estrategias utilizadas para garantizar una experiencia de uso coherente y mantenible.

Arquitectura y organización del frontend

La aplicación frontend se ha desarrollado utilizando **Angular** como framework principal e **Ionic** como conjunto de componentes destinados al desarrollo de interfaces híbridas. Esta combinación permite mantener una única base de código capaz de ejecutarse tanto en navegadores web como en dispositivos móviles, reduciendo el esfuerzo de desarrollo y garantizando una experiencia de usuario uniforme en las diferentes plataformas soportadas.

La estructura del proyecto sigue las convenciones establecidas por Angular, organizando el código en distintos módulos según la responsabilidad de cada componente. Esta organización facilita el mantenimiento del proyecto y permite localizar rápidamente cada uno de los elementos que forman parte de la aplicación.

Los principales directorios utilizados son los siguientes:

- **Pages:** contienen las distintas vistas que conforman la aplicación y representan los diferentes apartados accesibles mediante el sistema de navegación.
- **Components:** agrupan componentes reutilizables empleados por varias páginas, permitiendo construir interfaces complejas a partir de elementos independientes.
- **Services:** centralizan la comunicación con el backend y encapsulan funcionalidades compartidas por distintos componentes de la aplicación.
- **Guards:** implementan los mecanismos de protección de rutas y control de acceso a las distintas vistas.
- **Models:** definen las estructuras de datos utilizadas por la aplicación y facilitan el tratamiento tipado de la información recibida desde el backend.

Esta organización permite mantener una clara separación entre los distintos elementos de la aplicación y facilita la incorporación de nuevas funcionalidades sin modificar significativamente la estructura existente.

Además de la organización del código, Angular proporciona un conjunto de herramientas que simplifican aspectos como la navegación, la inyección de dependencias, el tratamiento de datos asíncronos y la comunicación entre componentes, contribuyendo a construir una aplicación modular y fácilmente mantenible.

Implementación

Una vez definida la estructura general del proyecto, se procedió a la implementación de los distintos elementos que componen la aplicación. El desarrollo del frontend se ha basado principalmente en la construcción de componentes independientes encargados de gestionar tanto la representación de la información como la interacción del usuario con el sistema.

Las diferentes funcionalidades implementadas comparten un patrón de funcionamiento similar: obtención de la información necesaria mediante los servicios de la aplicación, actualización del estado interno del componente y representación dinámica de los datos utilizando los componentes proporcionados por Ionic.

Estructura del proyecto Cada funcionalidad desarrollada se organiza alrededor de una página principal encargada de coordinar la interacción con el usuario. Estas páginas hacen uso de componentes

reutilizables y servicios especializados que encapsulan la lógica necesaria para acceder a la información y actualizar la interfaz cuando se producen cambios en el estado de la aplicación.

Esta distribución permite mantener una elevada cohesión dentro de cada componente y reducir las dependencias entre las distintas partes del sistema.

Páginas, componentes y controladores Las diferentes vistas de la aplicación se implementan como *Standalone Components*, una de las funcionalidades incorporadas en las versiones recientes de Angular. Este enfoque elimina la necesidad de declarar módulos específicos para cada componente y permite definir localmente todas las dependencias requeridas por cada página.

Como ejemplo representativo, el siguiente pseudocódigo resume el funcionamiento general de uno de los componentes principales de la aplicación.

```
Componente Angular standalone

1. Declarar los módulos necesarios.

2. Inyectar los servicios utilizados.

3. Inicializar el componente:
   - Obtener el usuario.
   - Obtener el grupo seleccionado.
   - Cargar la información asociada.

4. Escuchar cambios en la navegación.

5. Gestionar la interacción del usuario.

6. Solicitar información a los servicios.

7. Actualizar la interfaz.
```

Este patrón se repite en la mayor parte de las páginas desarrolladas, variando únicamente los servicios utilizados y las operaciones específicas implementadas en cada una de ellas.

Entre los aspectos más relevantes de esta implementación destacan:

- La utilización de **Standalone Components**, que simplifica la organización del proyecto y reduce el número de dependencias globales.
- El empleo del sistema de **inyección de dependencias** de Angular para acceder a servicios, mecanismos de navegación y otros elementos compartidos.
- La utilización de los distintos métodos del **ciclo de vida** de Angular para inicializar la información mostrada y mantener sincronizado el estado de cada componente.

- La comunicación asíncrona mediante **Observables**, permitiendo actualizar automáticamente la interfaz cuando se producen cambios en la información recibida desde el backend.
- La utilización de componentes proporcionados por **Ionic**, facilitando la construcción de interfaces adaptadas tanto a dispositivos móviles como a navegadores web.

Este enfoque permite desarrollar componentes independientes, fácilmente reutilizables y preparados para evolucionar conforme aumentan las funcionalidades implementadas por la aplicación.

Manejo de rutas y navegación La navegación entre las distintas vistas de la aplicación se implementa mediante el sistema de enrutado proporcionado por Angular, integrado con los componentes de navegación de Ionic. Esta combinación permite construir una navegación fluida entre páginas, manteniendo una experiencia homogénea tanto en dispositivos móviles como en navegadores web.

Cada ruta se asocia a un componente específico y puede incorporar mecanismos adicionales de protección o redirección cuando resulta necesario. De esta forma, la navegación queda completamente desacoplada de la lógica implementada por cada página, facilitando la incorporación de nuevas vistas conforme evoluciona la aplicación.

Como ejemplo representativo, el siguiente pseudocódigo resume el funcionamiento general del sistema de protección de rutas implementado mediante *Guards*.

Control de acceso a una ruta

1. Comprobar si existe una sesión activa.
2. Si el usuario no está autenticado:
 - Redirigir a la página de inicio de sesión.
3. Si el usuario está autenticado:
 - Comprobar sus permisos.
4. Permitir o denegar el acceso según el rol del usuario.

Este mecanismo centraliza la gestión de la navegación protegida y evita que cada componente tenga que implementar individualmente las comprobaciones de acceso.

Gestión del estado La aplicación mantiene la información compartida mediante servicios de Angular que actúan como punto centralizado de acceso a los datos utilizados por los distintos componentes.

Este enfoque evita la duplicación de información entre páginas y facilita que cualquier cambio realizado sobre el estado de la aplicación se propague automáticamente al resto de componentes interesados.

Uno de los servicios más representativos es el encargado de gestionar la información del usuario autenticado y mantener sincronizado el estado de la sesión durante toda la ejecución de la aplicación.

El funcionamiento general de este servicio puede resumirse mediante el siguiente pseudocódigo.

```
Servicio de gestión del estado

1. Inicializar la información almacenada.

2. Publicar el estado actual.

3. Actualizar el estado cuando cambien
   los datos recibidos.

4. Permitir que otros componentes
   consulten la información compartida.

5. Notificar automáticamente
   cualquier modificación.
```

Gracias a esta estrategia, los distintos componentes permanecen sincronizados sin necesidad de establecer dependencias directas entre ellos.

Integración con el backend La comunicación con el backend se realiza mediante servicios especializados que encapsulan todas las peticiones HTTP necesarias para acceder a los recursos de la aplicación.

Cada servicio agrupa las operaciones correspondientes a un dominio funcional concreto, facilitando su reutilización desde distintos componentes y evitando duplicar el código responsable de la comunicación con la API.

La Figura 4.7, presentada en el apartado de aspectos comunes, ilustra el flujo general seguido durante la comunicación entre ambos módulos. Desde el punto de vista del frontend, dicho proceso se materializa mediante servicios que envían las solicitudes correspondientes, procesan las respuestas recibidas y actualizan automáticamente la información mostrada al usuario.

El siguiente pseudocódigo resume el funcionamiento general de este mecanismo.

```
Comunicación con el backend

1. Recibir la solicitud del componente.

2. Construir la petición HTTP.

3. Enviar la solicitud.

4. Esperar la respuesta.
```

5. Procesar el resultado.
6. Actualizar el estado.
7. Refrescar la interfaz.

La utilización de servicios especializados simplifica el mantenimiento del código y permite modificar la implementación de la comunicación con el backend sin afectar al resto de componentes de la aplicación.

Diseño y experiencia de usuario

Además de la implementación funcional, durante el desarrollo del frontend se ha prestado especial atención al diseño de la interfaz y a la experiencia ofrecida al usuario. El objetivo ha sido construir una aplicación intuitiva, consistente y adaptable a los diferentes dispositivos desde los que puede accederse al sistema.

La utilización de Ionic proporciona un amplio conjunto de componentes visuales preparados para dispositivos móviles y navegadores web, permitiendo desarrollar interfaces modernas sin necesidad de implementar manualmente muchos de los elementos habituales de una aplicación.

Componentes de la interfaz La construcción de las distintas pantallas se basa principalmente en la utilización de componentes proporcionados por Ionic, entre los que destacan cabeceras, listas, tarjetas, botones, formularios, menús laterales, pestañas, modales y sistemas de notificación.

Estos componentes permiten mantener una identidad visual homogénea en toda la aplicación y favorecen la reutilización de elementos comunes entre las distintas vistas.

Especialmente relevante resulta el uso de componentes modales, que permiten realizar determinadas operaciones sin abandonar la página actual, mejorando la continuidad de la interacción y reduciendo el número de cambios de contexto durante el uso de la aplicación.

Diseño responsive La naturaleza híbrida de Ionic facilita la adaptación automática de la interfaz a diferentes tamaños de pantalla sin necesidad de desarrollar versiones independientes de la aplicación.

Esta adaptabilidad se consigue mediante el uso del sistema de rejillas proporcionado por Ionic, junto con componentes capaces de modificar automáticamente su distribución en función del espacio disponible.

Como consecuencia, la aplicación mantiene una presentación coherente tanto en dispositivos móviles como en ordenadores de escritorio, garantizando una experiencia de usuario uniforme.

La Figura 4.11 muestra un ejemplo de esta adaptación utilizando la pantalla de inicio de sesión en diferentes dispositivos.

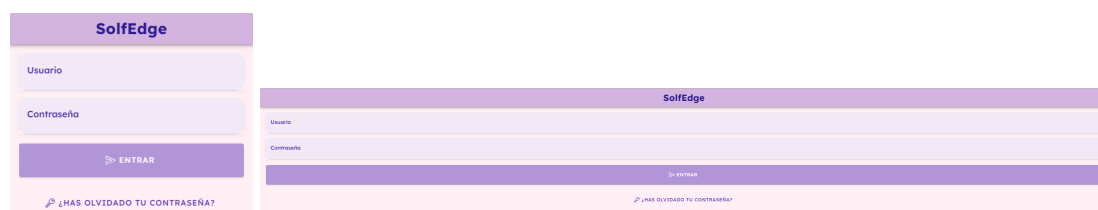


Figura 4.11: Vista de inicio de sesión en dispositivos móviles y escritorio.

Usabilidad y accesibilidad Durante el diseño de la interfaz se han seguido criterios orientados a facilitar el aprendizaje y reducir la complejidad de las tareas más habituales.

Para ello se han utilizado estructuras de navegación consistentes, componentes reutilizables, mensajes de retroalimentación inmediatos y mecanismos de validación que ayudan al usuario durante la interacción con la aplicación.

Asimismo, la utilización de los componentes de Ionic favorece el cumplimiento de criterios básicos de accesibilidad, incorporando soporte para atributos semánticos, compatibilidad con tecnologías asistivas y una adecuada organización de los distintos elementos visuales.

Las Figuras 4.12, 4.13 y 4.14 muestran algunos ejemplos representativos de la interfaz desarrollada, donde puede observarse la utilización de menús adaptativos, componentes reutilizables y una organización visual orientada a mejorar la experiencia de usuario.

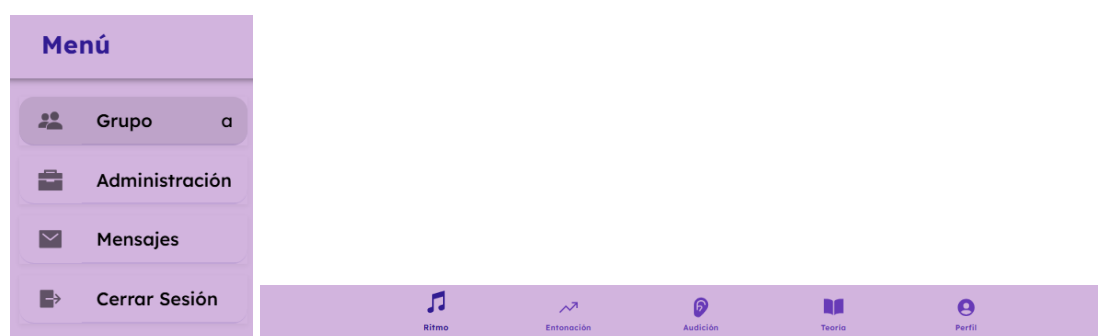


Figura 4.12: Menú lateral y barra de pestañas utilizadas en la aplicación.

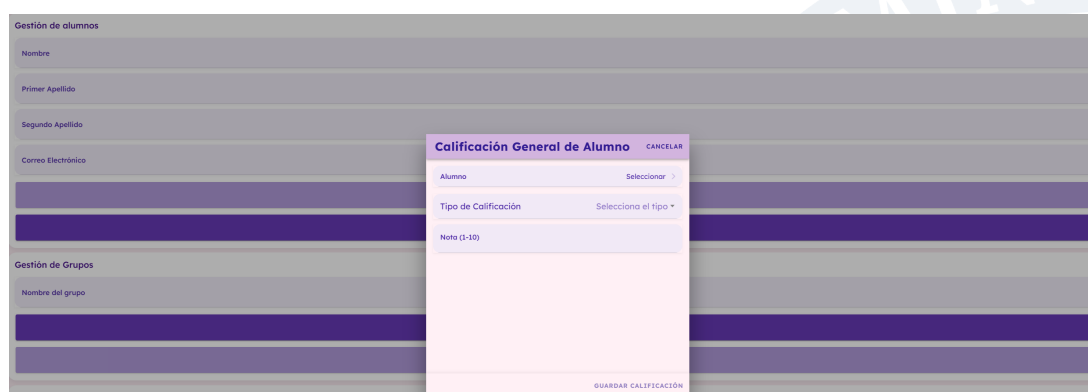


Figura 4.13: Ejemplo de utilización de componentes modales en la gestión de calificaciones.



Figura 4.14: Vista representativa de la interfaz desarrollada.

Testing

Con el objetivo de validar el correcto funcionamiento del frontend, se ha desarrollado un conjunto de pruebas automatizadas orientadas a verificar tanto el comportamiento de los componentes como su interacción con los distintos servicios de la aplicación.

Para ello se han utilizado las herramientas propias del ecosistema Angular:

- **Karma**, empleado como entorno de ejecución de las pruebas, permitiendo su lanzamiento tanto en navegadores reales como en modo *headless*.
- **Jasmine**, utilizado para definir los casos de prueba, realizar aserciones y simular el comportamiento de dependencias externas mediante *spies*.

La combinación de ambas herramientas facilita la validación automática de los componentes desarrollados y permite detectar posibles regresiones conforme evoluciona la aplicación.

Como ejemplo representativo se muestra el conjunto de pruebas desarrollado para el componente Tab5Page, encargado de gestionar distintas funcionalidades relacionadas con la interacción del usuario.

```
Pruebas del componente Tab5Page

1. Verificar la creación del componente.

2. Abrir el modal de mensajes.

3. Abrir el modal de calificaciones
   cuando existe un grupo seleccionado.

4. Comprobar que el modal no se abre
   si no existe un grupo activo.

5. Verificar la interacción
   con los servicios simulados.
```

Estas pruebas permiten comprobar tanto la correcta inicialización de los componentes como la interacción con los servicios utilizados por la aplicación, garantizando un comportamiento estable ante distintos escenarios de uso.

Despliegue

El frontend se despliega mediante un proceso automatizado que genera una versión optimizada de la aplicación preparada para su publicación en producción.

La construcción del proyecto se realiza utilizando Angular CLI en modo de producción, proceso durante el cual se aplican distintas optimizaciones destinadas a reducir el tamaño de la aplicación y mejorar su rendimiento.

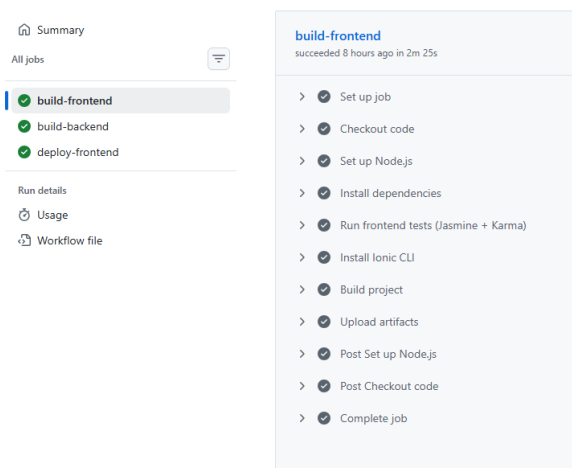
Entre las principales optimizaciones realizadas destacan:

- **Minificación** del código HTML, CSS y TypeScript.
- **Tree Shaking**, eliminando dependencias no utilizadas.
- **Compilación Ahead-Of-Time (AOT)**, que reduce los tiempos de carga de la aplicación.

Una vez generado el paquete de distribución, el despliegue se realiza mediante **GitHub Pages**, plataforma utilizada para alojar la versión web del proyecto.

El proceso de publicación consiste en generar el directorio de distribución del proyecto y actualizar automáticamente la rama configurada para GitHub Pages, desde donde la aplicación queda disponible para los usuarios finales.

✓ fix backend test corregido #107



(a) Proceso de construcción del frontend.

```
198 Chrome Headless 143.0.0.0 (Linux 0.0.0): Executed 179 of 179 SUCCESS
199 TOTAL: 179 SUCCESS
200 ✓ Browser application bundle generation complete.
201 ✓ Browser application bundle generation complete.
```

(b) Publicación automática mediante GitHub Pages.

Figura 4.15: Proceso de despliegue del frontend.

Este proceso permite mantener la aplicación permanentemente actualizada y facilita la distribución de nuevas versiones con una intervención manual mínima, simplificando las tareas de mantenimiento y evolución del sistema.

4.8.4 Integración continua del sistema

Una vez desarrollados de forma independiente el backend y el frontend, resulta necesario coordinar su construcción y publicación mediante un proceso automatizado que garantice la coherencia entre ambos módulos. Para ello se ha implementado un flujo de integración continua (CI) que centraliza las tareas de compilación, validación y despliegue de todo el sistema utilizando GitHub Actions como plataforma de automatización.

A diferencia de los apartados anteriores, donde se describen los procesos específicos de despliegue de cada aplicación, esta sección se centra en el funcionamiento conjunto del pipeline y en la coordinación de las distintas tareas necesarias para mantener sincronizadas las versiones del frontend y del backend.

Coordinación del pipeline

El flujo de integración continua se encuentra definido mediante un único fichero de configuración de GitHub Actions encargado de orquestar las diferentes etapas del proceso. Cada vez que se produce una actualización sobre la rama principal del repositorio, el pipeline inicia automáticamente la ejecución de las tareas necesarias para verificar el estado del proyecto y generar las versiones correspondientes de ambas aplicaciones.

La automatización de este proceso permite que todas las operaciones se ejecuten de forma repetible y controlada, evitando errores derivados de intervenciones manuales y garantizando que el código publicado corresponda siempre a una versión previamente validada.

El funcionamiento general del pipeline puede resumirse mediante el próximo pseudocódigo.

```
Pipeline de integración continua

1. Detectar una actualización del repositorio.

2. Iniciar el workflow de GitHub Actions.

3. Construir el frontend:
   - Descargar el código.
   - Instalar dependencias.
   - Ejecutar pruebas.
   - Generar el build de producción.

4. Construir el backend:
   - Descargar el código.
   - Instalar dependencias.
   - Ejecutar pruebas.

5. Si ambos procesos finalizan correctamente:
   - Publicar el frontend.
   - Desplegar el backend.
```


6. Finalizar el pipeline.

Este flujo refleja la coordinación existente entre ambos módulos durante el proceso de integración continua. Aunque cada aplicación dispone de sus propios mecanismos de construcción y despliegue, la ejecución centralizada del pipeline permite mantener ambas versiones sincronizadas y facilita la gestión del proyecto como una única aplicación.

Automatización del flujo de trabajo

El pipeline implementado se compone de varios trabajos independientes que se ejecutan siguiendo una secuencia determinada. En primer lugar se realizan las tareas de construcción del frontend y del backend, verificando que ambos proyectos pueden compilar correctamente y que las pruebas automatizadas finalizan sin errores.

Una vez completadas estas etapas, el pipeline continúa con las tareas de publicación correspondientes a cada módulo. El frontend se publica automáticamente mediante GitHub Pages utilizando los artefactos generados durante el proceso de construcción, mientras que el backend actualiza la versión desplegada en Render a partir de la última revisión disponible en el repositorio.

La separación de estas tareas en diferentes trabajos facilita el mantenimiento del pipeline y permite modificar de forma independiente el proceso de construcción o despliegue de cualquiera de los módulos sin afectar al funcionamiento global del sistema.

Además, GitHub Actions proporciona información detallada sobre la ejecución de cada una de las etapas, incluyendo registros de ejecución, tiempos empleados y posibles incidencias detectadas durante el proceso. Esta información resulta especialmente útil durante las tareas de mantenimiento, ya que simplifica la identificación del origen de posibles errores y permite verificar rápidamente el estado de cada despliegue realizado.

Resultado del proceso

La integración continua implementada proporciona una visión unificada del ciclo de construcción y despliegue de todo el proyecto, coordinando las distintas herramientas empleadas por el frontend y el backend bajo un único flujo de trabajo.

Esta aproximación garantiza que ambas aplicaciones evolucionen de forma sincronizada y que cada nueva versión publicada haya superado previamente las verificaciones correspondientes. Como consecuencia, se reduce el riesgo de inconsistencias entre módulos y se simplifica la distribución de nuevas funcionalidades durante la evolución del sistema.

Asimismo, la utilización de un pipeline automatizado facilita la reproducibilidad del proceso de despliegue y proporciona una mayor trazabilidad sobre las distintas versiones del proyecto, permitiendo conocer en todo momento el estado de cada ejecución y detectar rápidamente posibles incidencias.

En conjunto, este mecanismo de integración continua complementa los procesos específicos descritos para el backend y el frontend, proporcionando una visión global del ciclo de construcción, validación y publicación de todo el sistema.

5 FLUJOS DE TRABAJO

En los capítulos anteriores se ha presentado el proceso seguido para el análisis, diseño e implementación de la solución informática desarrollada. No obstante, la descripción individual de las funcionalidades no permite apreciar completamente el funcionamiento de la aplicación ni la interacción existente entre los diferentes actores que participan en ella.

Por este motivo, en este capítulo se describen los principales flujos de trabajo soportados por la plataforma, mostrando la secuencia de acciones que realizan profesores y alumnos durante el uso de las funcionalidades más representativas del sistema. El objetivo es ofrecer una visión global del comportamiento de la aplicación y de la comunicación existente entre sus distintos módulos.

Cada flujo se acompaña de diferentes capturas de pantalla que ilustran los momentos más relevantes del proceso, tales como la creación de recursos, la interacción por parte del alumnado, la confirmación de operaciones realizadas o la consulta de los resultados obtenidos.

5.1 FLUJO DE TRABAJO DE CUESTIONARIOS

Los cuestionarios constituyen uno de los principales mecanismos de evaluación continua disponibles en la plataforma. A través de esta funcionalidad, el profesorado puede diseñar actividades compuestas por preguntas musicales que posteriormente serán resueltas por el alumnado, permitiendo realizar una evaluación prácticamente inmediata de los conocimientos adquiridos.

El flujo completo comprende tres etapas claramente diferenciadas: la creación del cuestionario por parte del profesor, la realización del cuestionario por parte del alumno y la consulta de los resultados obtenidos una vez finalizada la actividad.

Los requisitos implicados en este flujo son: RF19 (figura 4.44), RF28 (figura 4.53), RF30 (figura 4.55) y RFT1 (figura 4.60), tal y como se ha descrito en el capítulo de análisis de requisitos.

5.1.1 Creación del cuestionario

El flujo comienza cuando el profesor accede al apartado de cuestionarios desde la aplicación. Desde esta sección dispone de la posibilidad de consultar los cuestionarios existentes, modificarlos, eliminarlos o crear uno nuevo.

Para crear un cuestionario, el profesor selecciona la opción correspondiente e introduce la información necesaria para su configuración. Entre los datos solicitados se encuentran el título del cuestionario, los alumnos destinatarios y la fecha límite para su realización. Posteriormente, el profesor incorpora las preguntas que compondrán la actividad junto con sus correspondientes respuestas correctas.

Una vez completada toda la información, el sistema valida los datos introducidos y registra el nuevo cuestionario, mostrándose un mensaje de confirmación indicando que la operación se ha realizado correctamente.

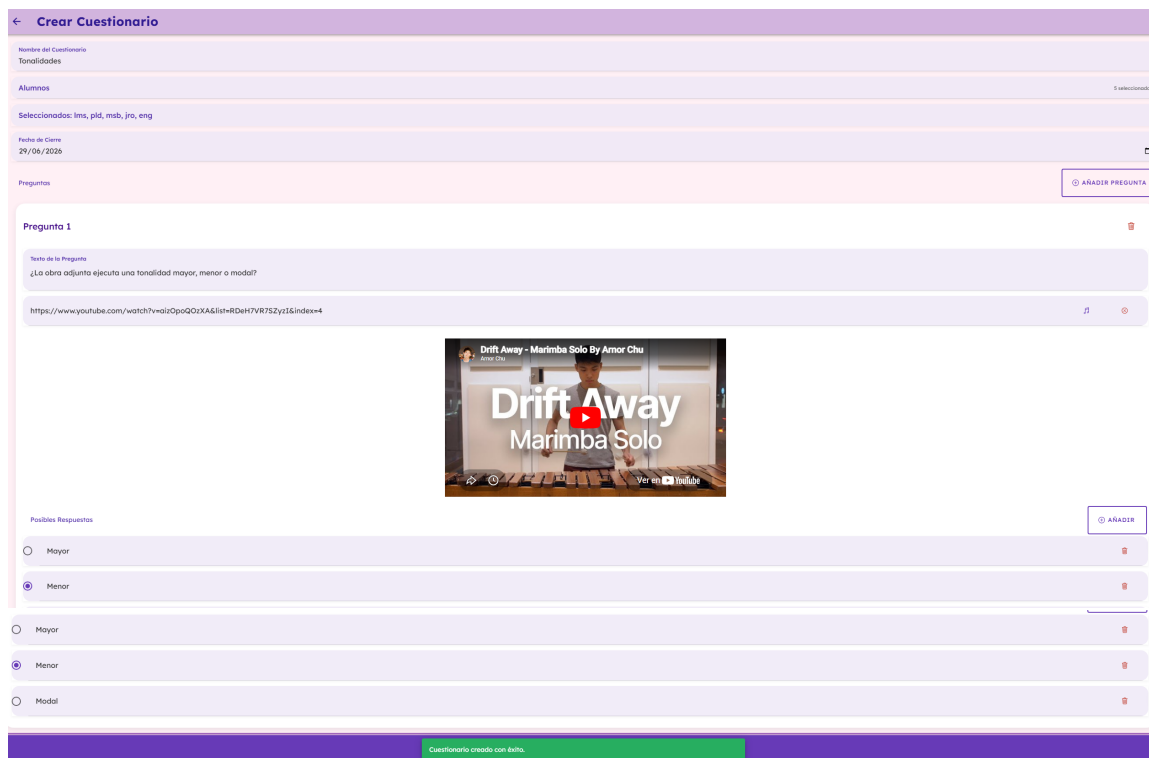


Figura 5.1: Creación de un nuevo cuestionario.

Tras la creación del cuestionario, este pasa a estar disponible para todos los alumnos pertenecientes al grupo seleccionado, los cuales recibirán un mensaje de notificación.

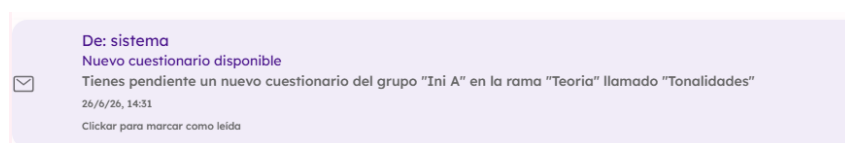


Figura 5.2: Mensaje de notificación al alumno tras la creación del cuestionario.

5.1.2 Resolución del cuestionario

Cuando un alumno accede a la sección de cuestionarios, la aplicación muestra únicamente aquellos cuestionarios que han sido publicados para el grupo al que pertenece.

Una vez seleccionado uno de ellos, comienza el proceso de resolución. Durante esta fase el alumno puede escuchar los audios asociados a cada pregunta, analizar el contenido musical y seleccionar la respuesta que considere correcta. En aquellos ejercicios en los que el profesor haya habilitado pistas, el sistema permite consultarlas para facilitar la resolución de la actividad.

El alumno puede avanzar entre las distintas preguntas hasta completar el cuestionario en su totalidad. Finalmente, confirma el envío de las respuestas para que estas sean almacenadas y evaluadas por el sistema, obteniendo la valoración al instante.

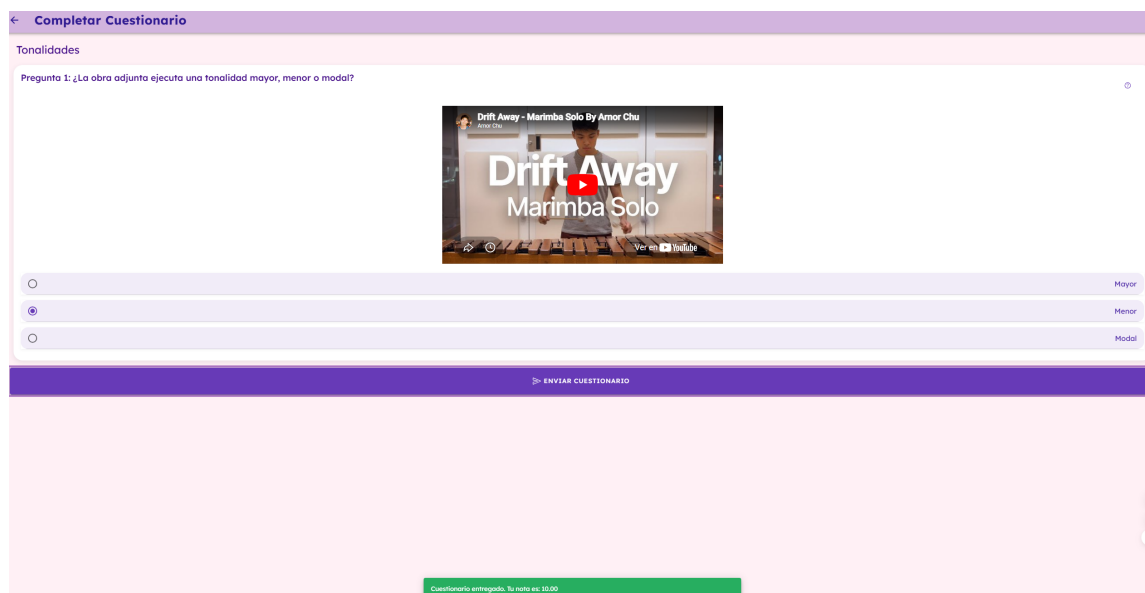


Figura 5.3: Resolución de un cuestionario por parte del alumno utilizando las pistas disponibles.

5.1.3 Consulta de resultados

Una vez enviado el cuestionario, como se ha mencionado anteriormente, el sistema procesa las respuestas proporcionadas por el alumno y calcula automáticamente la calificación obtenida en función de los criterios de corrección establecidos. Este proceso permite ofrecer una retroalimentación inmediata, facilitando que el alumno conozca su rendimiento sin necesidad de esperar a la intervención del profesor desde su perfil.

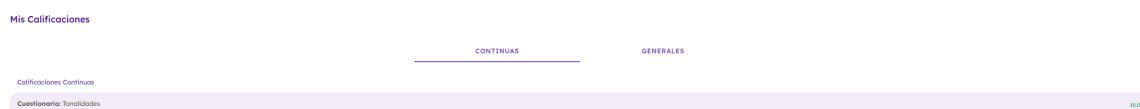


Figura 5.4: Consulta de un cuestionario realizado y revisión de las respuestas.

Con este proceso finaliza el flujo completo asociado a la gestión de cuestionarios, quedando registradas las respuestas y la calificación obtenida por el alumno para su posterior consulta.

5.2 FLUJO DE TRABAJO DE TAREAS

A diferencia de los cuestionarios, cuyo proceso de corrección es automático, las tareas permiten al profesorado plantear actividades que requieren una evaluación individualizada. De esta forma, el profesor puede valorar ejercicios cuya resolución no puede determinarse únicamente mediante respuestas cerradas, proporcionando además observaciones personalizadas que sirvan de orientación al alumnado.

El flujo de trabajo de las tareas comprende cuatro fases principales: creación de la tarea, entrega por parte del alumno, evaluación realizada por el profesor y consulta de la calificación obtenida.

Los requisitos implicados en este flujo son: RF-14 (figura 4.39), RF-23 (figura 4.48), RF-24 (figura 4.49), RF-25 (figura 4.50), RF-28 (figura 4.53), RF-29 (figura 4.54), y RFT-1 (figura 4.60), tal y como se ha descrito en el capítulo de análisis de requisitos.

5.2.1 Creación de la tarea

El profesor accede al apartado de tareas y selecciona la opción destinada a crear una nueva actividad.

Durante este proceso debe cumplimentar la información necesaria para su publicación, indicando el título de la tarea, una descripción con las instrucciones correspondientes, el grupo al que irá dirigida y la fecha límite de entrega.

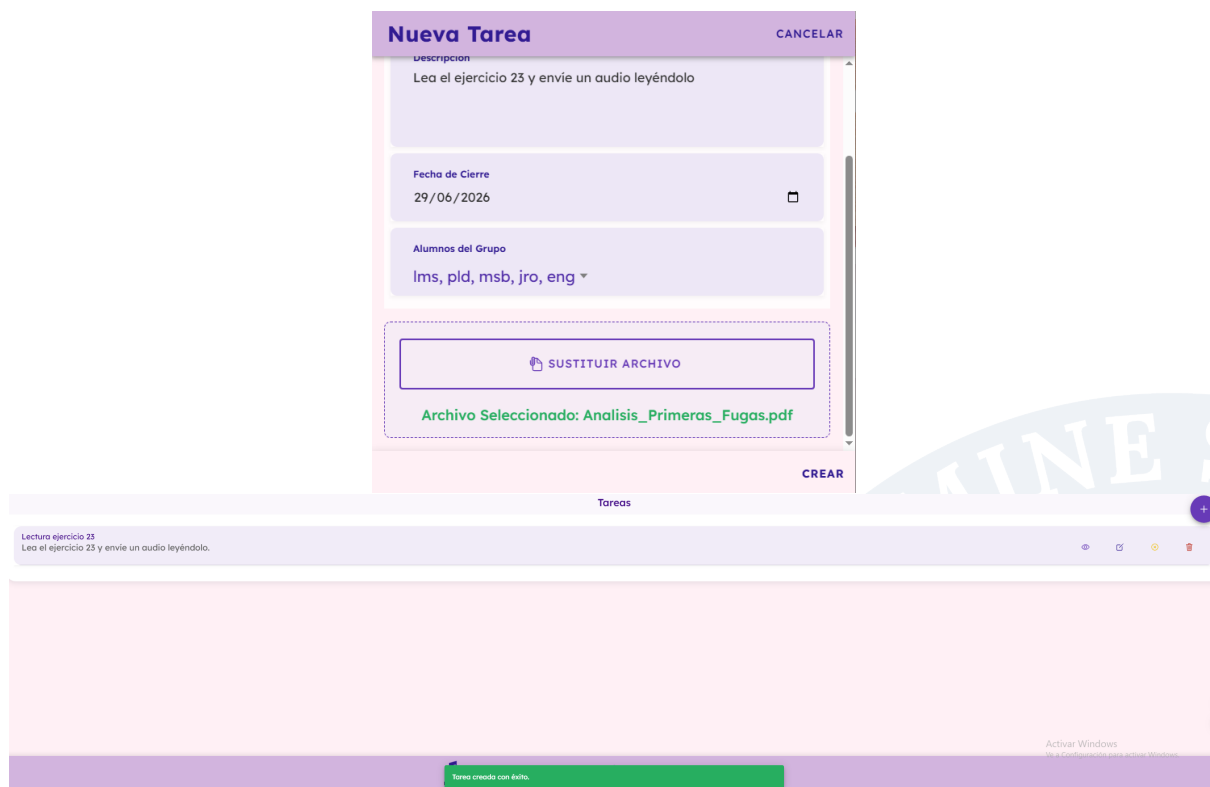


Figura 5.5: Creación de una nueva tarea.

Una vez introducidos todos los datos, la plataforma valida la información proporcionada y registra la nueva tarea, enviando un mensaje de notificación al alumno.

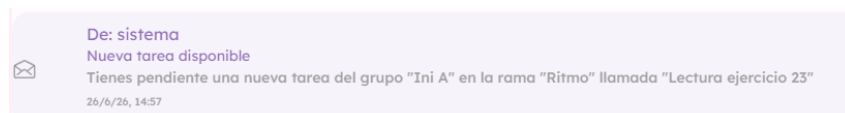


Figura 5.6: Confirmación de la creación de la tarea.

Una vez creada, la tarea pasa automáticamente a estar disponible para todos los alumnos pertenecientes al grupo seleccionado.

5.2.2 Entrega de la tarea

El alumno accede al apartado de tareas, donde puede visualizar todas aquellas actividades que han sido publicadas para su grupo.

Tras seleccionar una tarea concreta, consulta las instrucciones proporcionadas por el profesor y procede a realizar la entrega correspondiente utilizando el mecanismo habilitado por la aplicación.

Una vez finalizado el proceso de entrega, el alumno confirma la operación y el sistema almacena la información recibida, notificando que la entrega se ha realizado correctamente.



Figura 5.7: Entrega de una tarea por parte del alumno.

Tras completar el envío, el sistema muestra un mensaje confirmando que la entrega ha quedado registrada correctamente.

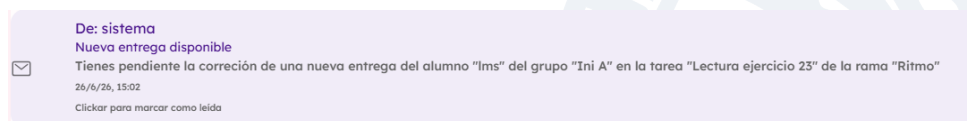


Figura 5.8: Mensaje de notificación al profesor tras la entrega de la tarea.

5.2.3 Evaluación de la tarea

Una vez finalizado el plazo de entrega o cuando el profesor lo considere oportuno, este puede acceder al listado de entregas asociadas a una tarea concreta. Desde esta vista se muestran todos los alumnos que han realizado la entrega, permitiendo consultar el contenido enviado por cada uno de ellos.

Tras revisar la entrega, el profesor asigna una calificación numérica y, si lo considera necesario, incorpora observaciones o comentarios destinados a proporcionar una retroalimentación personalizada al alumno. Este mecanismo permite no solo evaluar el trabajo realizado, sino también orientar al alumnado acerca de los aspectos susceptibles de mejora.

Cuando la evaluación finaliza, el sistema registra la información introducida y ya se puede comprobar las calificaciones desde la tarea.



Figura 5.9: Evaluación de una tarea por parte del profesor.

Lectura ejercicio 23

Descripción: Lee el ejercicio 23 y envíe un audio leyéndolo.

Profesor: ogp

Fecha de entrega:

Entregas de Alumnos

lms

Entregado: 28/6/26, 20:25

Calificado: 28/6/26, 20:25

23

Figura 5.10: Calificaciones en detalles de tarea.

5.2.4 Consulta de la calificación

Una vez corregida la tarea, el alumno puede acceder nuevamente a la actividad para consultar la evaluación realizada por el profesor.

En esta pantalla la aplicación muestra la calificación obtenida junto con las observaciones introducidas durante el proceso de corrección, permitiendo al alumno conocer tanto el resultado alcanzado como los aspectos destacados por el profesor durante la evaluación.

De esta forma, la funcionalidad de tareas no se limita únicamente al proceso de entrega, sino que proporciona un mecanismo completo de seguimiento del aprendizaje mediante la incorporación de comentarios personalizados.

Mis Calificaciones

	CONTINUAS	GENERALES
Calificaciones Continuas		
Tareas: Lectura ejercicio 23		1.00
Cuestionarios: Tonaltidades		10.00
Tareas: Leer lección 23		7.82

Figura 5.11: Consulta de la tarea evaluada por parte del alumno.

Con esta última fase concluye el flujo de trabajo asociado a la gestión de tareas, quedando registrada tanto la entrega realizada como la evaluación efectuada por el profesor.

5.3 FLUJO DE TRABAJO DE CALIFICACIONES

Además de las evaluaciones individuales realizadas mediante cuestionarios y tareas, la aplicación incorpora un módulo específico destinado a la calificación general del alumno donde, además, se podrán revisar calificaciones generales anteriores.

Este flujo comprende dos fases, la calificación general por parte del profesor y la consulta de las calificaciones por parte del alumno.

Los requisitos implicados en este flujo son: RF-26 (figura 4.51), RF-27 (figura 4.52), RF-28 (figura 4.53) y RFT-1 (figura 4.60), tal y como se ha descrito en el capítulo de análisis de requisitos.

5.3.1 Calificación general por parte del profesor

El profesor puede asignar una calificación general a cada alumno pudiendo revisar en el momento las calificaciones generales anteriores.

Calificación General de Alumno
CANCELAR

Alumno lms >

Tipo de Calificación Q2 ▾

Nota (1-10) 8

Calificaciones existentes para lms en este grupo:
 - Q1: 7

GUARDAR CALIFICACIÓN

Figura 5.12: Calificación general por parte del profesor.



5.3.2 Consulta de calificaciones por parte del alumno

El alumnado dispone igualmente de acceso al apartado de calificaciones generales desde el perfil, donde puede consultar en cualquier momento el resultado obtenido.

Gracias a esta funcionalidad, el alumno dispone en todo momento de una visión clara de su evolución académica.

Mis Calificaciones

CONTINUAS

GENERALES

Calificaciones Generales

Primer Trimestre	
Segundo Trimestre	
Tercer Trimestre	Aún no calificado
Ordinaria	Aún no calificado
Extraordinaria	Aún no calificado

Figura 5.13: Consulta de las calificaciones por parte del alumno.

5.4 CONCLUSIÓN

Los flujos de trabajo presentados en este capítulo muestran el funcionamiento integrado de las principales funcionalidades implementadas durante el desarrollo del proyecto. A través de ellos puede observarse la interacción continua entre profesores y alumnos, así como el papel desempeñado por la plataforma durante todas las etapas del proceso de enseñanza y aprendizaje.

La combinación de funcionalidades como la creación de cuestionarios, la gestión de tareas y el seguimiento de las calificaciones permite ofrecer un entorno unificado donde el profesorado puede organizar y evaluar la actividad académica, mientras que el alumnado dispone de herramientas que facilitan tanto la realización de las actividades propuestas como el seguimiento de su propio progreso.

En conjunto, estos flujos reflejan el objetivo principal perseguido durante el desarrollo de la aplicación: proporcionar una solución informática que facilite la enseñanza del lenguaje musical mediante una plataforma intuitiva, organizada y adaptada a las necesidades detectadas durante las fases iniciales de investigación.

6 CONCLUSIONES Y LÍNEAS DE TRABAJO FUTURO

En este capítulo se presentan las conclusiones finales del proyecto, evaluando el grado de cumplimiento de los objetivos planteados inicialmente, así como una reflexión sobre las posibles líneas de trabajo futuro que permitirían ampliar, mejorar y consolidar la solución desarrollada.

6.1 CONCLUSIONES

El objetivo principal de este proyecto consistía en **resolver la problemática relacionada con la falta de recursos informáticos específicos orientados a la enseñanza del lenguaje musical**. A lo largo del desarrollo de la aplicación se ha trabajado de forma progresiva para dar respuesta a esta necesidad, apoyándose en tecnologías actuales, gratuitas y ampliamente utilizadas, y poniendo el foco tanto en profesores como en alumnos.

En relación con los subobjetivos definidos inicialmente, se pueden extraer las siguientes conclusiones:

- **Entender las necesidades de profesores y alumnos de lenguaje musical:** Este objetivo se ha alcanzado a través de un proceso sistemático de análisis que ha incluido la investigación en centros educativos, la recopilación de información mediante encuestas y entrevistas, así como el estudio bibliográfico. Este trabajo previo ha permitido identificar con precisión las dinámicas habituales de la enseñanza del lenguaje musical, las principales dificultades detectadas y las carencias existentes en las herramientas digitales disponibles. Como resultado directo de este análisis, la aplicación incorpora funcionalidades alineadas con estas necesidades, tales como la gestión de grupos, calificaciones, tareas, cuestionarios y mensajería, ofreciendo un soporte integral tanto para la labor docente como para el seguimiento y la participación activa del alumnado.
- **Aprender sobre la relación existente entre la informática y el lenguaje musical:** Las fases de estudio, análisis y diseño del proyecto han permitido profundizar en la intersección entre ambas disciplinas, abordando la representación digital de conceptos musicales, la estructuración pedagógica de contenidos y la adaptación de metodologías tradicionales a entornos software interactivos. Este aprendizaje ha sido determinante para el planteamiento de la arquitectura funcional de la aplicación, la definición de los distintos módulos y la elección de las tecnologías empleadas, consolidando una base conceptual sólida sobre la que se ha construido toda la solución.

- **Definir una solución informática que cubra las necesidades detectadas:** A partir del análisis previo, se ha definido una solución integral basada en una arquitectura cliente-servidor que integra un backend modular y un frontend híbrido multiplataforma. La especificación funcional, la definición de requisitos y el diseño arquitectónico han permitido establecer una estructura coherente, escalable y mantenible, capaz de dar respuesta a las necesidades identificadas. Tal y como se detalla en los capítulos 4.8.2, 4.8.3 y 4.8.4, la arquitectura adoptada y la implementación técnica confirman la viabilidad, robustez y consistencia de la solución propuesta.
- **Desarrollar la solución:** Este objetivo se ha materializado mediante la implementación completa del sistema, abarcando el desarrollo del backend, la construcción del frontend híbrido, la integración de mecanismos de seguridad, la automatización de pruebas y la implantación de un flujo de integración y despliegue continuo. El resultado es una aplicación funcional, estable y usable, validada mediante pruebas unitarias y de integración, y preparada para su despliegue en entornos reales, cumpliendo con los requisitos establecidos durante las fases iniciales del proyecto.
- **Llevar a cabo el proceso de desarrollo con tecnologías totalmente gratuitas:** A lo largo de todo el proyecto se han empleado exclusivamente tecnologías gratuitas y de código abierto, tanto en el desarrollo como en el despliegue, incluyendo frameworks, herramientas de testing y plataformas de integración continua. Este enfoque ha permitido demostrar la viabilidad de una solución completa sin costes de licencias, si bien también ha puesto de manifiesto ciertas limitaciones en términos de escalabilidad y despliegue en producción, que constituyen una base realista para futuras líneas de mejora. En conjunto, el objetivo se considera plenamente alcanzado.

En conjunto, el proyecto ha logrado cumplir de manera satisfactoria el objetivo principal y los subobjetivos planteados, estableciendo una base sólida para una herramienta informática orientada a la enseñanza del lenguaje musical, con un alto potencial de evolución, mejora y adaptación a contextos educativos reales.

6.2 LÍNEAS DE TRABAJO FUTURO

A pesar de los resultados obtenidos, existen múltiples líneas de trabajo futuro que permitirían ampliar el alcance del proyecto y mejorar su aplicabilidad en contextos reales.

6.2.1 Plan de implantación y despliegue en entornos reales

Una de las principales líneas de trabajo futuro consiste en la definición de un plan de implantación formal de la aplicación en centros educativos. El uso de tecnologías gratuitas, aunque ventajoso desde el punto de vista económico, presenta limitaciones en términos de escalabilidad, persistencia de datos y gestión avanzada de configuraciones.

Sería necesario estudiar aspectos como:

- La adaptación de los entornos de despliegue para soportar un mayor número de usuarios.

- La gestión de archivos de configuración y variables de entorno específicas para cada centro.
- La posible migración a infraestructuras más robustas manteniendo, en la medida de lo posible, el uso de herramientas gratuitas o de bajo coste.

Este trabajo permitiría acercar la aplicación a un uso real en entornos educativos.

6.2.2 Extensión de funcionalidades e innovación

Otra línea clara de evolución es la ampliación de las funcionalidades actuales, tanto para mejorar la calidad del desarrollo como para ofrecer características innovadoras y atractivas para los usuarios, algunos ejemplos son:

- **Integración de inteligencia artificial:** Desarrollo de pistas o ayudas automáticas en cuestionarios y ejercicios, adaptadas al progreso del alumno mediante técnicas de IA, favoreciendo un aprendizaje más personalizado.
- **Gestión masiva de usuarios y grupos mediante archivos:** Incorporación de funcionalidades que permitan dar de alta usuarios y a grupos a partir de archivos con un formato definido (por ejemplo CSV), evitando la creación manual y agilizando la administración por parte del profesorado o los centros.
- **Borrado de usuarios y grupos:** Actualmente no se contempla la opción desde dentro de la aplicación para el borrado de datos por lo que sería necesaria asistencia técnica para ello, por lo tanto, es interesante permitir el borrado de usuarios y grupos y, con ellos, su material asociado (tareas, cuestionarios, grupos, calificaciones, mensajes...)

Estas mejoras reforzarían el carácter innovador de la aplicación y su adecuación a contextos educativos reales.

6.2.3 Acceso sin autenticación y apertura a usuarios externos

Una línea más de trabajo futura relevante consiste en extender la aplicación para ofrecer funcionalidades accesibles a usuarios no autenticados o no vinculados a ninguna entidad educativa. Esto podría incluir:

- Acceso a contenidos informativos o demostrativos sin necesidad de iniciar sesión.
- Registro de usuarios independientes, no asociados a un centro, que puedan utilizar parte de la aplicación de forma autónoma.

Esta apertura permitiría aumentar el alcance de la aplicación, facilitar su difusión y servir como punto de entrada para nuevos usuarios interesados en el aprendizaje del lenguaje musical.



6.2.4 Creación de la aplicación móvil

Por último, aprovechando el desarrollo híbrido realizado, extender la aplicación a sistemas Android y/o iOS adaptando la aplicación donde sea necesario para su correcta ejecución en dispositivos móviles. Esto permitiría a los usuarios acceder a la plataforma desde cualquier lugar y en cualquier momento, aumentando la flexibilidad y comodidad en el uso de la aplicación.

En conclusión, el proyecto constituye una base sólida sobre la que construir futuras mejoras, demostrando tanto la viabilidad técnica como el potencial educativo de la solución desarrollada.

7 ANEXO

En este anexo se recoge información complementaria que respalda la implementación descrita a lo largo de la memoria, aportando evidencias técnicas y visuales del desarrollo realizado sin interrumpir el hilo principal del documento. Concretamente, se incluyen el acceso al código fuente del proyecto, la estructura general de los proyectos frontend y backend, así como capturas adicionales de la aplicación en funcionamiento.

7.1 REPOSITORIO DEL PROYECTO

El código fuente completo del sistema desarrollado se encuentra disponible en un repositorio público de GitHub, lo que permite consultar la implementación real del frontend y del backend, así como la evolución del proyecto durante su desarrollo.

El repositorio incluye:

- Código fuente del frontend desarrollado con Ionic y Angular.
- Código fuente del backend basado en una API REST.
- Ficheros de configuración necesarios para la ejecución y despliegue.
- Historial de commits que refleja el proceso de desarrollo.

El acceso al repositorio se realiza a través del siguiente enlace:

`https://github.com/selugc4/SolfEdge`

Este enlace permite verificar de forma directa los aspectos técnicos descritos en la memoria y facilita la reproducibilidad del proyecto en entornos de desarrollo locales.

7.2 ESTRUCTURA DEL PROYECTO

A continuación se presenta una visión general de la organización de los proyectos frontend y backend, mostrando la separación de responsabilidades y la modularidad adoptada.



7.2.1 Estructura del backend

El backend se organiza siguiendo una arquitectura modular orientada a servicios REST:

```
backend/  
|-- controllers/  
|-- models/  
|-- routes/  
|-- middleware/  
|-- services/  
`-- *.config  
`-- index.js
```

Esta disposición favorece el mantenimiento del código y la extensión futura de la aplicación, permitiendo añadir nuevas funcionalidades de forma controlada.

7.2.2 Estructura del frontend

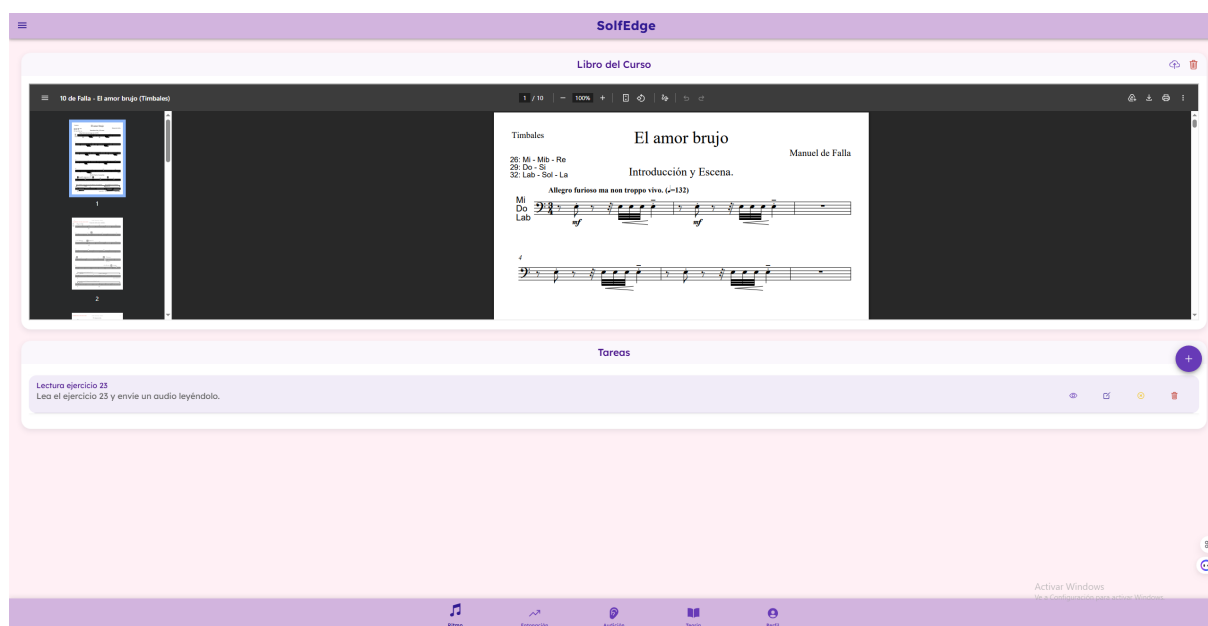
El frontend sigue la estructura habitual de un proyecto Ionic + Angular, organizada en torno a componentes, páginas y servicios respetando así el MVC (Modelo-Vista-Controlador):

```
frontend/  
|-- src/  
|   |-- app/  
|   |-- |-- pages/  
|   |-- |-- components/  
|   |-- |-- services/  
|   |-- |-- guards/  
|   |-- |-- models/  
|   |-- `-- app-routing.module.ts  
|   |-- assets/  
|   |-- environments/  
|   |-- theme/  
|   `-- index.html  
`-- angular.json
```

Esta estructura facilita la escalabilidad del proyecto y la reutilización de componentes, manteniendo una clara separación entre lógica de presentación, lógica de negocio y navegación.

7.3 CAPTURAS ADICIONALES DE LA APLICACIÓN

En este apartado se incluyen capturas adicionales de la aplicación en funcionamiento que complementan las mostradas anteriormente. Estas imágenes permiten apreciar otras vistas y flujos relevantes del sistema, reforzando la comprensión global de la aplicación desarrollada. En este caso, en la Figura 7.1 se puede observar la vista de la rama de ritmo (con metrónomo incorporado para el alumno), en la Figura 7.2 la vista del profesor y del administrador de la propia administración y en la Figura 7.3 la vista de mensajería.



(a) Vista de profesor con libro y tareas



(b) Vista de alumno sin libro y sin tareas

Figura 7.1: Vista de cada rama de la aplicación



SolfEdge

Gestionar profesores

Nombre

Primer Apellido

Segundo Apellido

Correo Electrónico

CREAR PROFESOR

GESTIONAR PROFESORES

Importar Usuarios y Grupos (CSV)

El archivo CSV debe contener las columnas: tipo, ref. Para usuarios: nombre, apellido1, apellido2, email, rol (alumno/profesor), profesor_ref (solo si es alumno). Para grupos: nombre_grupo, profesor_ref, alumnos_ref (refs separadas por ;).

SELECCIONAR ARCHIVO CSV

SUBIR CSV

(a) Administración de administrador

SolfEdge

Gestión de alumnos

Nombre

Primer Apellido

Segundo Apellido

Correo Electrónico

CREAR ALUMNO

REVISAR ALUMNOS

Gestión de Grupos

Nombre del grupo

SELECCIONAR ALUMNOS (0)

CREAR GRUPO

Gestión y Comunicación

ENVIAR MENSAJE AL GRUPO

ADJUNTAR CALIFICACIÓN GENERAL

GESTIONAR GRUPO ACTUAL

Utilice estas herramientas para gestionar las notas, enviar mensajes o modificar los detalles del grupo seleccionado.

Activar Windows
Ve a Configuración para activar Windows.

(b) Administración de profesor

Figura 7.2: Vista de administración

Mensajes

De: sistema
Calificación de tono (solfedge)
La calificación obtenida en calidad es 7
3/3/20, 10:55
Click para marcar como leído

De: sistema
Calificación general de (Q2)
La calificación obtenida en Q2 es 5
16/3/20, 10:55

De: sistema
Calificación general de (Q2)
La calificación obtenida en Q2 es 6
16/3/20, 14:20

De: sistema
Calificación general de (Extraordinaria)
La calificación obtenida en Extraordinaria es 7
16/3/20, 10:55

Figura 7.3: Vista de mensajería

Estas capturas ilustran la variedad de vistas y componentes que conforman la aplicación, evidenciando su complejidad funcional, su coherencia visual y su correcta adaptación a distintos flujos de uso.

7.4 ENTORNOS Y HERRAMIENTAS DE DESARROLLO

Además de las tecnologías principales descritas a lo largo de la memoria, durante el desarrollo del proyecto se han empleado diversas herramientas de apoyo que han facilitado la implementación, depuración y verificación del sistema. Estas herramientas no constituyen un elemento central en la justificación técnica del proyecto, pero sí han sido relevantes para el flujo de trabajo diario y la productividad durante el desarrollo.

7.4.1 Entorno de desarrollo

El desarrollo tanto del frontend como del backend se ha realizado principalmente utilizando el editor de código **Visual Studio Code**. Este entorno ha permitido trabajar de forma eficiente gracias a su ligereza, extensibilidad y buen soporte para las tecnologías empleadas en el proyecto.

Entre las extensiones más utilizadas se encuentran:

- Extensiones de soporte para **Angular** y **TypeScript** como **GeminiCLI** y **Copilot** que facilitan la navegación por el código, el autocompletado y la detección temprana de errores.
- Extensiones para **ESLint**, empleadas para mantener un estilo de código consistente y legible.
- Extensiones para **Git**, que permiten gestionar el control de versiones directamente desde el editor.

Estas herramientas han contribuido a mejorar la calidad del código y a reducir errores durante el proceso de desarrollo, sin influir directamente en la arquitectura o las decisiones técnicas del sistema.

7.4.2 Gestión y visualización de la base de datos

Para la gestión y exploración de la base de datos se ha utilizado **MongoDB Compass**, una herramienta gráfica que permite interactuar con bases de datos MongoDB de forma visual. Su uso ha sido especialmente útil durante las fases de desarrollo y depuración, permitiendo:

- Visualizar y verificar el contenido de las colecciones.
- Comprobar la estructura de los documentos almacenados.
- Realizar consultas rápidas para validar el comportamiento del backend.

Cabe destacar que MongoDB Compass se ha empleado únicamente como herramienta de apoyo, sin formar parte del sistema en producción ni afectar a la lógica de la aplicación.

7.4.3 Herramientas de apoyo adicionales

Durante el desarrollo también se han utilizado otras herramientas auxiliares, entre ellas:

- **Navegadores web con herramientas de desarrollo**, para depurar el frontend, inspeccionar el DOM y analizar el comportamiento responsive de la interfaz como Chrome que permite revisar la vista como si fuera desde móvil.
- **Terminal y gestores de paquetes** (npm), empleados para la ejecución de scripts, instalación de dependencias y lanzamiento de servidores locales.

Estas herramientas han permitido verificar el correcto funcionamiento del sistema durante las distintas fases del desarrollo, contribuyendo a un proceso de implementación más controlado y eficiente. En conjunto, el uso de estos entornos y herramientas ha facilitado el desarrollo del proyecto sin condicionar las decisiones arquitectónicas ni los objetivos planteados, sirviendo como soporte técnico durante todo el ciclo de vida de la aplicación.

BIBLIOGRAFÍA

- [1] UNIR, “Enseñar música con el método kodaly,” 2021. [Online]. Available: <https://www.unir.net/revista/musica/metodo-kodaly/>
- [2] R. S. Pressman, *Software Engineering: A Practitioner’s Approach*, 7th ed. McGraw-Hill, 2010. [Online]. Available: https://www.mlsu.ac.in/econtents/16_EBOOK-7th_ed_software_engineering_a_practitioners_approach_by_roger_s._pressman_.pdf
- [3] Universidad Europea, “Lenguaje musical: qué es, cómo enseñarlo y fichas útiles,” Aug. 2025. [Online]. Available: <https://universidadeuropea.com/blog/lenguaje-musical/>
- [4] Musicorcheas, “Elementos básicos de una partitura,” 2020. [Online]. Available: <https://musicorcheas.blogspot.com/2020/09/elementos-basicos-de-una-partitura.html>
- [5] Instituto Nacional de Tecnologías Educativas y de Formación del Profesorado (INTEF), “Laboratorio de música. Creación de sintonías radiofónicas con tecnologías digitales.” [Online]. Available: https://intef.es/experiencias_edu/laboratorio-de-musica/
- [6] UNIR, “La importancia de la pedagogía musical en el desarrollo del alumnado,” 2024. [Online]. Available: <https://www.unir.net/revista/musica/pedagogo-musical/>
- [7] MuseScore, “MuseScore Studio Handbook.” [Online]. Available: <https://musescore.org/es/handbook>
- [8] Noteflight, “Music Notation User Guide.” [Online]. Available: <https://www.noteflight.com/guide>
- [9] Audacity Team, “Audacity Manual.” [Online]. Available: <https://manual.audacityteam.org/>
- [10] Kahoot!, “Getting Started Guide.” [Online]. Available: <https://kahoot.com/library/getting-started-guide/>
- [11] Estudio Música, “Estudio música.” [Online]. Available: <https://estudiomusica.com/>
- [12] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Prentice Hall, 2007. [Online]. Available: https://vowi.fsinf.at/images/b/bc/TU_Wien-Verteilte_Systeme_VO_%28G%C3%B6schka%29_-_Tannenbaum-distributed_systems_principles_and_paradigms_2nd_edition.pdf
- [13] Postman, “What Is a REST API? Examples, Uses & Challenges,” 2025. [Online]. Available: <https://blog.postman.com/rest-api-examples/>

- [14] Ionic Team, “Ionic Framework Documentation.” [Online]. Available: <https://ionicframework.com/docs>
- [15] Angular Team, “Angular Documentation.” [Online]. Available: <https://angular.dev/overview>
- [16] ESIC Business & Marketing School, “Modelo-Vista-Controlador: qué es, cómo funciona y ventajas,” 2025. [Online]. Available: <https://www.esic.edu/rethink/tecnologia/modelo-vista-controlador-que-es-como-funciona-y-ventajas-c>
- [17] World Wide Web Consortium (W3C), “World Wide Web Consortium.” [Online]. Available: <https://www.w3.org/>
- [18] Jasmine Team, “Jasmine Testing Framework Documentation.” [Online]. Available: <https://jasmine.github.io/>
- [19] Karma Core Team, “Karma Testing Documentation.” [Online]. Available: <https://karma-runner.github.io/6.4/index.html>
- [20] Node.js Foundation, “Node.js Documentation.” [Online]. Available: <https://nodejs.org/docs/latest/api/>
- [21] Express.js, “Express Framework Documentation.” [Online]. Available: <https://expressjs.com/es/>
- [22] JWT.io, “Introduction to JSON Web Tokens.” [Online]. Available: <https://www.jwt.io/introduction#what-is-json-web-token>
- [23] OWASP Foundation, “OWASP Foundation.” [Online]. Available: <https://owasp.org/>
- [24] SmartBear, “Swagger Documentation.” [Online]. Available: <https://swagger.io/docs/>
- [25] Jest Team, “Jest Documentation.” [Online]. Available: <https://jestjs.io/docs/getting-started>
- [26] npm, Inc., “Supertest Package.” [Online]. Available: <https://www.npmjs.com/package/supertest>
- [27] MongoDB, Inc., “MongoDB Documentation.” [Online]. Available: <https://www.mongodb.com/docs/>
- [28] Mailjet, “Mailjet Email Guides.” [Online]. Available: <https://dev.mailjet.com/email/guides/>
- [29] yavisht, “GenratrAPI Repository.” [Online]. Available: <https://github.com/yavisht/GenratrAPI>
- [30] GitHub, “GitHub Documentation.” [Online]. Available: <https://docs.github.com/es>
- [31] Render, “Render Documentation.” [Online]. Available: <https://render.com/docs>
- [32] Princeton University Library, “Literature Mapping: Overview,” 2023. [Online]. Available: <https://libguides.princeton.edu/litmapping>

- [33] I. Rodríguez Heras, “El lenguaje musical a través de las TIC en Educación Primaria,” 2022. [Online]. Available: <https://uvadoc.uva.es/bitstream/handle/10324/56944/TFG-G5671.pdf>
- [34] I. Gorbunova and H. Hiner, “Music Computer Technologies and Interactive Systems of Education in Digital Age School,” in *Proceedings of the International Conference Communicative Strategies of Information Society (CSIS 2018)*, 2019. [Online]. Available: <https://www.atlantis-pess.com/proceedings/csis-18/55913802>
- [35] M. García Aguirre, “Aplicación informática para la introducción del lenguaje musical (LeeMúsica) en dispositivos móviles. Niveles 1 y 2 (3 años),” 2017. [Online]. Available: <https://uvadoc.uva.es/bitstream/10324/22384/1/TFG-G2315.pdf>
- [36] S. Marić, “Online gaming to learn music and English language in music and ballet school solfeggio education,” *Hellenic Journal of Music, Education and Culture*, vol. 6, no. 2, p. 54, 2015. [Online]. Available: <https://hejmec.eu/journal/index.php/HeJMEC/article/view/58/54>
- [37] L. Parshina, “Multimedia technologies as a tool for teaching supervision over the students’ skills (within the course on “music theory training”),” *Life Science Journal*, vol. 11, no. 6, pp. 547–551, 2014. [Online]. Available: https://www.lifesciencesite.com/ljsj/life1106/081_24754life110614_547_551.pdf

El proyecto ha consistido en el estudio e investigación de las diferentes herramientas, tecnologías o cualquier aspecto que pudiera relacionar la asignatura de lenguaje musical con la ingeniería informática, así como la definición, desarrollo y despliegue de una solución informática para resolver la carencia de recursos informáticos en el proceso de aprendizaje del lenguaje musical.

The project has consisted of the study and research of different tools, technologies, and any other aspects that could relate the subject of music theory to computer engineering, as well as the definition, development, and deployment of a software solution aimed at addressing the lack of digital resources in the process of learning music theory.

